



Universidad Internacional de la Rioja (UNIR)

Escuela Superior de Ingeniería y Tecnología

Máster en Inteligencia Artificial

Optimización de modelos de lenguaje compactos para la síntesis de programas orientada a la resolución de ARC-AGI-1

Trabajo Fin de Estudios

presentado por: Asier Marin Fernandez

Dirigido por: Fernando Antonio Rufo Jimenez

Ciudad: Valencia

Fecha: 22 de julio de 2026

Índice de Contenidos

Resumen	VIII
Abstract	IX
1. Introducción	1
1.1. Contexto	1
1.2. Justificación	2
1.3. Objetivos	2
2. Estado del Arte	4
2.1. Definición de la inteligencia	4
2.2. La inteligencia desde un punto de vista filosófico	4
2.3. Núcleos del conocimiento	5
2.4. Sistemas de pensamiento	6
2.5. Algoritmos de búsqueda y optimización	7
2.5.1. Búsqueda en anchura	8
2.5.2. Búsqueda de coste uniforme	8
2.5.3. Búsqueda en profundidad	9
2.5.4. Búsqueda A*	10
2.5.5. Algoritmo de Monte Carlo aplicado a Búsquedas	10
2.5.6. Síntesis de programas	13
2.5.7. Búsqueda Guiada por Redes Neuronales	13
2.6. Abstract and Reasoning Corpus	14
2.6.1. Espectro de la generalización	15
2.6.2. La inteligencia desde la perspectiva de la adquisición de habilidades	17
2.6.3. ARC-AGI-1	20
2.6.4. Núcleos de conocimiento aplicado en ARC-AGI	21
2.6.5. Repositorio de datos	24
2.6.6. ARC-AGI-2	25
2.6.7. ARC-AGI-3	26
2.6.8. Comparativa entre sistemas ARC-AGI	29
2.7. Primeros enfoques para resolver ARC-AGI	29

2.7.1. Domain Specific Languages (DSLs) y síntesis de programas	29
2.7.2. Enfoque Neurosimbólico	30
2.8. Grandes Modelos de Lenguaje en ARC-AGI-1	31
2.9. Modelos basados en la Neurodiversidad	32
2.9.1. Inducción de Síntesis de programas mediante Redes Neuronales Efi- cientes	34
2.10. Afinamiento eficiente de modelos para ARC-AGI	37
2.11. Enfoques de modelos sin preentrenamiento	39
2.12. Modelos basados en Transformers compactos	40
2.13. Modelos compactos basados en bucles de refinamiento iterativo	41
2.13.1. Modelos de razonamiento jerárquico	41
2.13.2. Modelos de razonamiento recursivo con redes neuronales compactas	42
2.14. Estado actual con perspectiva de eficiencia computacional	44
3. Metodología	48
3.1. Enfoque Científico y Método de Investigación	48
3.2. Metodología de Desarrollo y Gestión del Proyecto	48
3.3. Ciclo de Vida del Modelo basado en MLOps	49
4. Infraestructura y herramientas	50
4.1. Infraestructura de computación	50
4.2. Entorno de desarrollo	51
4.3. Herramientas externas	53
5. Desarrollo	54
5.0.1. Análisis exploratorio de datos	57
5.1. Sistema de referencia	61
5.1.1. Pruebas iniciales	65
5.1.2. Codificador Multitensor	66
5.1.3. Arquitectura de decodificación	68
5.1.4. Limitaciones	77
5.1.5. Núcleos de conocimiento	79
5.2. Ejes de mejora	82
5.2.1. Eje H: Reducción de saturación de procesos en paralelo	83

5.2.2. Eje D: Eficiencia computacional y aprovechamiento de recursos . . .	86
5.2.3. Eje B: Mejora de robustez del espacio latente y multi-semilla	88
5.2.4. Problemas encontrados	97
6. Resultados	99
7. Ética del proyecto	101
8. Conclusiones y Trabajo Futuro	102
Referencias	103

Índice de Ilustraciones

2.1. Representación árbol de nodos búsqueda en anchura	8
2.2. Representación árbol de nodos búsqueda en profundidad	9
2.3. Pasos algoritmo MCTS	12
2.4. Modelo jerárquico de habilidades cognitivas y su mapeo del espectro de generalización	17
2.5. Posición del problema, un sistema inteligente que genera programas de ha- bilidades	18
2.6. Tarea donde el objetivo es completar un patrón de un área dada	21
2.7. Prueba de ruido	22
2.8. El objetivo rojo debe “moverse” junto al objeto azul hasta hacer “contacto”	22
2.9. Tarea donde el objetivo es contar el objeto que aparece más veces	23
2.10. Tarea donde el objetivo es generar una línea diagonal hasta rebotar al hacer contacto con el obstáculo rojo	24
2.11. Tarea ARC-AGI 2: cbebaa4b	26
2.12. Tarea ARC-AGI 3: ls20	28
2.13. Arquitectura VLM	36
2.14. Flujo de proceso de pre-entrenamiento e inferencia	38
2.15. Algoritmo de compresión de CompressARC	40
2.16. Arquitectura TRM con proceso recursivo y eficiente en cuanto a parámetros	43
2.17. Evolución de la puntuación de ARC-AGI desde 2020	44
2.18. Comparativa de la progresión entre ARC-AGI-1 y ARC-AGI-2	45
2.19. Comparativa del coste por tarea \$ entre diferentes soluciones	46
2.20. Comparativa del coste por tarea entre modelos	47
4.1. Representación semántica y relacional del uso de la herramienta Codebase Memory como <i>indexer</i> de código.	53
5.1. Comparativa del coste por tarea entre modelos compactos	56
5.2. Comparativa del coste por tarea modelos compactos	57
5.3. Ejemplo de tarea desde el editor de código	58
5.4. Valor asignado a cada color	59

5.5. Tarea 045e512c	59
5.6. Tarea 06df4c85	60
5.7. Tarea 0dfd9992	60
5.8. Tarea 05a7bcf2	61
5.9. Arquitectura <i>CompressARC</i>	63
5.10. Ejemplos de colapso posterior. Tareas 6cdd2623, ecdecbb3, 2bee17df y 007bbfb7	89
5.11. Arquitectura <i>CompressARC</i> con ejes de mejora H/B/D.	90
5.12. Panel de monitorización del progreso y comportamiento del sistema.	97
6.1. Comparativa del coste por tarea modelos compactos	100

Índice de Tablas

2.1. Comparación de las versiones de ARC-AGI.	29
4.1. Rendimiento por nivel de precisión y formato.	51
4.2. Librerías de entorno en Python, versión y abreviación.	52
5.1. Combinaciones dimensionales del sistema Multitensor.	67
5.2. Resultados comparativos de la línea base y la versión optimizada del Eje H en 12 tareas de entrenamiento en paralelo.	86
5.3. Resultados de la comparación final del Eje D sobre 50 tareas con 1500 iteraciones por tarea.	88
6.1. Resultados finales tras la ejecución del modelo base respecto a ejes de mejora.	99

Resumen

Este trabajo diseña y optimiza *SuperCompressARC*, un modelo compacto orientado a la síntesis de programas y a la resolución del *benchmark* ARC-AGI-1, con prioridad en la eficiencia computacional frente al escalado de modelos de lenguaje. Se aplica una metodología empírico-analítica y cuantitativa, con desarrollo iterativo basado en prácticas ágiles y MLOps. La evaluación compara el modelo base con mejoras de *hardware*, memoria y arquitectura, incluyendo entrenamiento específico por tarea, estabilización del espacio latente y varias semillas. En el conjunto de entrenamiento, la métrica *pass@2* aumenta del 34,75 % al 44 %; en evaluación, del 20 % al 25,25 %. Además, el tiempo de ejecución de evaluación se reduce de 138 a 64,3 horas y el coste aproximado por tarea de 0,086 a 0,040 dólares. Se concluye que los modelos compactos son viables bajo restricciones de *hardware*, aunque la profundidad fija y la generalización limitada siguen siendo retos.

Palabras clave: ARC-AGI-1; modelos de lenguaje compactos; síntesis de programas; razonamiento abstracto; eficiencia computacional.

Abstract

This work designs and optimizes *SuperCompressARC*, a compact model for program synthesis and solving the ARC-AGI-1 *benchmark*, prioritizing computational efficiency over scaling language models. An empirical-analytical and quantitative methodology is applied, combining iterative development with agile and MLOps practices. The evaluation compares the baseline with hardware, memory, and architectural improvements, including task-specific training, latent-space stabilization, and multiple random seeds. On the training set, *pass@2* improves from 34.75% to 44%; on evaluation, from 20% to 25.25%. Evaluation runtime also decreases from 138 to 64.3 hours, while the approximate cost per task falls from 0.086 to 0.040 dollars. The results indicate that compact models are viable under hardware constraints, although fixed reasoning depth and limited generalization remain challenges.

Keywords: ARC-AGI-1; compact language models; program synthesis; abstract reasoning; computational efficiency.

1. Introducción

Actualmente existe una tendencia ascendente en el campo de inteligencia artificial. El paradigma del aprendizaje profundo ha demostrado ser una herramienta del presente y del futuro. El *software* es un elemento fundamental en la sociedad actual, y con la evolución del propio *software* también evolucionan los distintos campos tecnológicos, entre ellos el de la inteligencia artificial.

La inteligencia artificial es un campo de estudio que se centra en la creación de sistemas capaces de realizar tareas que normalmente requieren inteligencia humana, como el aprendizaje, el razonamiento, la resolución de problemas, la comprensión del lenguaje natural y la percepción visual. En los últimos años, ha habido un aumento significativo en el interés y el desarrollo de la inteligencia artificial, lo que ha llevado a avances notables en áreas como el aprendizaje automático, el procesamiento del lenguaje natural y la visión por computadora.

Cada vez se le da más importancia a las herramientas basadas en inteligencia artificial y se llega a un público más amplio. En este contexto, los mecanismos para que esta tecnología pueda ser consumida por el público general se vuelven cada vez más importantes. En este sentido, la eficiencia en el consumo de estos recursos tecnológicos cada vez es más relevante. Con la llegada de más modelos de lenguaje, la necesidad de encontrar formas de medir la capacidad de generalización de estos modelos se vuelve con el tiempo más importante.

Es aquí donde entra en juego el *Abstraction and Reasoning Corpus (ARC-AGI)*, un *benchmark* diseñado específicamente para evaluar la inteligencia de los sistemas y su capacidad para adquirir nuevas habilidades de forma eficiente a partir de unos pocos ejemplos, basándose en los principios del conocimiento central innato humano. El estándar ARC-AGI es una respuesta a la necesidad de medir la inteligencia, permitiendo la comparación entre sistemas y humanos.

1.1. Contexto

A pesar de los avances recientes, los Grandes Modelos de Lenguaje siguen teniendo dificultades para resolver este tipo de tareas, ya que sus capacidades se derivan principalmente de volúmenes masivos de datos de pre-entrenamiento y memorización probabilística. Esto evidencia que la hipótesis de escalar los modelos sin límites insospechados no es suficiente

para alcanzar el razonamiento abstracto. Aumentar ciegamente la cantidad de parámetros no dota automáticamente al sistema de la capacidad de resolver tareas que requieren un razonamiento abstracto más novedoso.

ARC-AGI representa el indicador estándar por excelencia de la industria para medir el progreso real hacia la inteligencia general artificial a través de la generalización amplia y la síntesis de programas.

1.2. Justificación

Abordar el problema de ARC-AGI desde la perspectiva de los Modelos de Lenguaje Pequeños o SMLs y la optimización en tiempo de inferencia supone un cambio de paradigma necesario: pasar del escalado por fuerza bruta a la optimización inteligente de los parámetros. Limitar deliberadamente la capacidad del modelo evita la sobre-generalización y fuerza al sistema a concentrarse exclusivamente en descubrir transformaciones algorítmicas robustas y reglas recursivas, en lugar de memorizar estadísticas superficiales.

La viabilidad de este enfoque está respaldada por investigaciones muy recientes. Arquitecturas como el *Hierarchical Reasoning Model* y el *Tiny Recursive Model* han logrado superar el rendimiento de modelos masivos en ARC-AGI utilizando redes minúsculas de apenas 7 a 27 millones de parámetros, gracias a la aplicación de pasos de computación recursiva en un espacio latente. Asimismo, enfoques revolucionarios como *CompressARC* han demostrado que es posible resolver un alto porcentaje de *puzzles* sin absolutamente ningún pre-entrenamiento, utilizando apenas 76.000 parámetros y minimizando la longitud de descripción puramente en el tiempo de inferencia (*Test-Time Training*).

Dadas las restricciones de capacidad de cómputo disponibles para este proyecto, apostar por estos paradigmas (bucles de refinamiento, síntesis de programas y entrenamiento en tiempo de prueba) no solo es una necesidad técnica, sino la estrategia científica más prometedora y eficiente para abordar el razonamiento complejo sin la saturación de recursos inherente a los modelos de lenguaje a gran escala.

1.3. Objetivos

El objetivo principal es diseñar y desarrollar un modelo compacto (SLM) que maximice la puntuación en la resolución del *benchmark* ARC-AGI-1, priorizando estrictamente la eficiencia computacional y el rendimiento por parámetro frente a los modelos de lenguaje

a gran escala.

En cuanto a objetivos particulares se enumeran los siguientes:

- Desarrollar e implementar bucles de refinamiento recursivo que permitan al modelo actualizar de manera iterativa sus estados latentes y mejorar progresivamente sus predicciones, simulando la síntesis de programas.
- Explorar y aplicar técnicas de adaptación en tiempo de prueba *Test-Time Training* y *Test-Time Compute* para que el modelo aprenda y descubra abstracciones algorítmicas directamente sobre las tareas objetivo, reduciendo la dependencia del pre-entrenamiento.
- Integrar mecanismos de tiempo de cálculo adaptativo o *early-stopping* que permitan al sistema abortar el bucle de razonamiento iterativo una vez haya alcanzado una solución convergente, optimizando drásticamente el consumo de memoria y el tiempo de ejecución.
- Implementar estrategias de consistencia y ensamblado *top-k* que operen sobre diversas variantes o aumentos de los datos de entrada para seleccionar las predicciones más robustas.
- Realizar una comparativa exhaustiva de eficiencia que evalúe cualitativamente la relación entre la precisión lograda en el *benchmark* y el coste computacional empleado, como el uso de VRAM, y precisión en FLOPs, validando la viabilidad de la inteligencia algorítmica en entornos de recursos restringidos.

2. Estado del Arte

2.1. Definición de la inteligencia

La definición de inteligencia no es una tarea sencilla. A lo largo de la historia, filósofos, científicos y expertos en diversas disciplinas han propuesto diferentes definiciones de inteligencia, cada una con su propia perspectiva y enfoque. A pesar de numerosos intentos, no existe una definición estándar de inteligencia.

La inteligencia se ha descrito de manera aproximada pero no se ha llegado a una definición concreta. Según la Real Academia Española, la inteligencia es la “capacidad de entender o comprender”, “capacidad de resolver problemas” y “conocimiento, comprensión, acto de entender”. Sin embargo, esta definición es bastante amplia y no captura completamente la complejidad de lo que implica la inteligencia.

Para crear una definición única se debe tener en cuenta varios factores. La inteligencia es una propiedad que tiene en consideración la interacción con uno o diversos entornos. La inteligencia también se relaciona con la capacidad del agente para tener éxito u obtener ganancias al respecto de alguna meta. De la misma forma, también depende de cómo el agente está capacitado en diferentes objetivos y entornos. Con base en referencias de diversos autores, en este trabajo se adoptará el estándar de definición más utilizada a día de hoy.

“Intelligence measures an agent’s ability to achieve goals in a wide range of environments.”

(Legg, Hutter, y cols., 2007, p. 9, párr. 6)

2.2. La inteligencia desde un punto de vista filosófico

Autores como Descartes, Kant o Hume han reflexionado sobre la naturaleza de la inteligencia y la mente humana. Descartes, por ejemplo, propuso la idea de que la mente y el cuerpo son entidades separadas, lo que se conoce como dualismo (González, 2011). Kant, por su parte, argumentó que la inteligencia es una facultad innata que nos permite organizar y comprender el mundo a través de categorías y conceptos (Waxman, 2014). Hume, en cambio, se centró en la experiencia y la percepción como base de la inteligencia

humana.

Aristóteles también realiza varias reflexiones sobre lo que hoy se entiende por inteligencia. En concreto, sus reflexiones se desarrollan fundamentalmente a través de la naturaleza del intelecto, la facultad de inteligir y el pensamiento.

“El intelecto -siendo impasible- ha de ser capaz de recibir la forma, es decir, ha de ser en potencia tal como la forma pero sin ser ella misma y será respecto de lo inteligible algo análogo a lo que es la facultad sensitiva respecto de lo sensible.”

(Aristotle y Boeri, 2010, p. 230, párr. 1)

2.3. Núcleos del conocimiento

En la teoría se ha considerado que el conocimiento humano se basa en un único sistema de aprendizaje de propósito general. Investigaciones recientes del departamento de Psicología de la Universidad de Harvard han propuesto una teoría alternativa que sugiere que el conocimiento humano se basa en una serie de núcleos o módulos innatos de conocimiento, cada uno especializado en un dominio específico, como el espacio, los objetos, las personas, los números y el lenguaje (Spelke y Kinzler, 2007).

Cada sistema se define por un conjunto de principios y un conjunto de fronteras cognitivas. Estas fronteras actúan como un sello de identidad que ayuda a diversos investigadores a demostrar que existe un sistema cognitivo único (Spelke y Kinzler, 2007). Por ejemplo, el sistema numérico central, se evidencia con representaciones numéricas imprecisas, esta imprecisión crece de forma lineal a medida que aumenta el número representado.

1. **Sistema de representación de objetos inanimados:** Es la capacidad de percibir límites de objetos, la forma completa o parcial de objetos ocultos y percibir cuándo y hacia donde se moverán los objetos.
2. **Sistema de representación de agentes y sus acciones:** Es la capacidad de percibir agentes animados, sus acciones y sus intenciones, incluso sin lenguaje.
3. **Sistema de representación numérica:** Es la capacidad de representar cantidades numéricas, incluso sin lenguaje, y de realizar operaciones aritméticas básicas.

4. **Sistema de representación geométrica del espacio:** Es la capacidad de representar el espacio y las relaciones espaciales entre objetos.
5. **Sistema de representación del lenguaje y social:** Es la capacidad de comprender y utilizar el lenguaje, así como de interactuar socialmente con otros individuos.

2.4. Sistemas de pensamiento

El concepto de sistemas de pensamiento fue originalmente propuesto por los psicólogos Keith Stanovich y Richard West, quienes describieron dos sistemas de pensamiento: el sistema 1, que es rápido, automático e intuitivo, y el sistema 2, que es lento, deliberado y analítico. Posteriormente fue popularizado por el ganador del Nobel Daniel Kahneman en su libro *Thinking, Fast and Slow* (2011).

- **Sistema 1 (Pensamiento intuitivo o rápido):** Este es el sistema de pensamiento más rápido, automático e intuitivo. Funciona de manera automática y es responsable de nuestras «reacciones instintivas», juicios rápidos y habilidades aprendidas que se han vuelto algo natural, como reconocer la cara de un amigo o resolver $2+2$. En el contexto de la inteligencia artificial, esto es análogo a la transducción, donde un modelo realiza directamente una predicción global basada en patrones.
- **Sistema 2 (Pensamiento analítico o lento):** Este es nuestro modo de pensamiento lento, deliberado y lógico. Requiere atención consciente y se utiliza para tareas complejas, como resolver un problema matemático difícil, sopesar ventajas e inconvenientes o aprender una nueva habilidad. En el ámbito de la inteligencia artificial, esto se corresponde con la inducción o la síntesis de programas, donde un sistema construye deliberadamente un programa lógico paso a paso para llegar a una solución.

La clave de estos conceptos es que el pensamiento humano hace uso de una combinación de ambos sistemas. Tiene la capacidad de usar un pensamiento más transductivo o inductivo dependiendo de la tarea y situación. Para la mayoría de tareas diarias es más eficiente el uso de un pensamiento más transductivo, pero cuando se requiere un razonamiento más complejo o intensivo, el pensamiento inductivo es más adecuado.

2.5. Algoritmos de búsqueda y optimización

Teóricamente, casi cualquier problema puede ser resuelto por fuerza bruta, pero en la realidad el coste computacional y tiempo hacen inabarcable este enfoque. El problema y el espacio son características fundamentales para determinar la estrategia de búsqueda y optimización más adecuada. En este sentido, existen diferentes algoritmos de búsqueda y optimización que se pueden aplicar en función de las características del problema.

Tradicionalmente los algoritmos de búsqueda se han clasificado en dos grandes categorías, los algoritmos de búsqueda no informada y los algoritmos de búsqueda informada. Los algoritmos de búsqueda no informada, son aquellos algoritmos que no están guiados o simplemente están basados en fuerza bruta. No existe información sobre el dominio del problema, sólo representación. Estos algoritmos intentan explorar todas las opciones posibles en cada nodo. Para buscar una solución óptima se imponen una serie de restricciones o reglas con el objetivo de evitar caer en bucles o ciclos de la estructura de los datos. Algunos ejemplos de algoritmos de búsqueda no informada son: búsqueda en anchura (BFS), búsqueda en profundidad (DFS), búsqueda de coste uniforme, búsqueda iterativa o limitada en profundidad, entre otros.

Los algoritmos no informados pueden ser mejorados proporcionando más información sobre el problema. Búsqueda informada significa que el algoritmo tiene un contexto específico y la heurística ayuda a representar el contexto. La heurística puede ser una serie de reglas para evaluar un estado y puede medir su rendimiento. De esta manera, no existe un camino fijo y los nodos son visitados de acuerdo al coste heurístico. Los costes son calculados a medida que se recorre el árbol de cada nodo visitado y se añade a la cola. El objetivo es simple, ignorar los árboles cuyo coste es mayor que los visitados, ahorrando coste computacional. La heurística se puede definir como la capacidad de realizar innovaciones para conseguir alcanzar los objetivos.

“Un proceso que puede resolver un problema dado, pero que no ofrece ninguna garantía de que lo hará, se llama una heurística para ese problema.”

(Newell, Shaw, y Simon, 1962)

La heurística puede reducir drásticamente el coste computacional de problemas que son resueltos con algoritmos no informados, pero también puede llevar a soluciones subóptimas. Algunos algoritmos de búsqueda informados son; búsqueda el primero el mejor (BFA),

búsqueda codiciosa el primero el mejor (GBFS), búsqueda A*, búsqueda de haz (Beam Search), entre otros.

2.5.1. Búsqueda en anchura

Explora todas las opciones de un determinado nivel de profundidad antes de pasar al siguiente nivel. Es necesario visitar todos los nodos hijos de un determinado nivel y finalizar al llegar al objetivo. Para implementarlo se usa una cola FIFO o *First In First Out*, los nodos de un nivel son procesados y sus nodos hijos son añadidos a la cola. Puede ser óptimo si todas las acciones tienen el mismo coste y es completo siempre que el espacio sea discreto o finito.

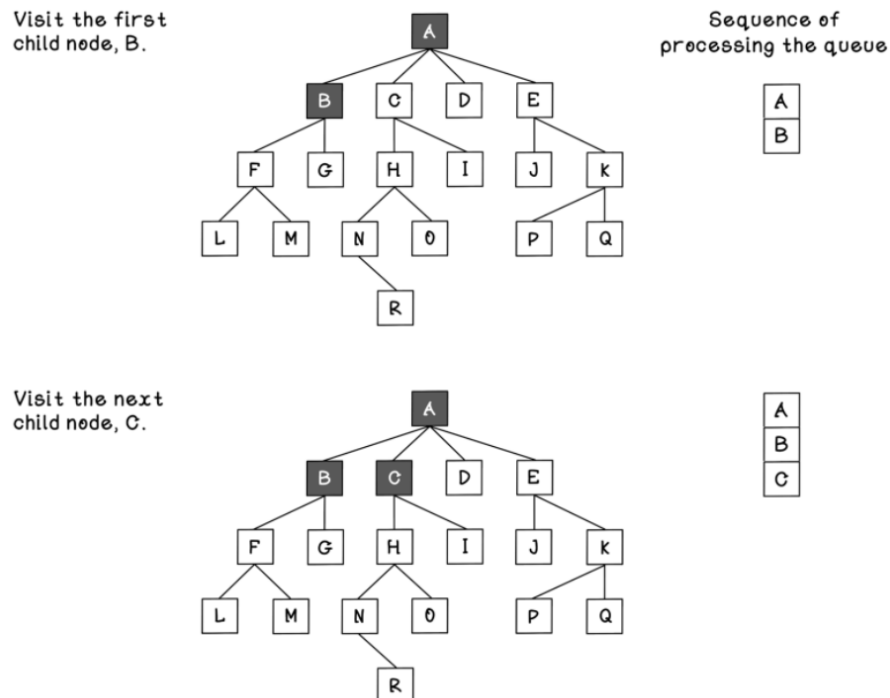


Figura 2.1: Representación árbol de nodos búsqueda en anchura.

Fuente: Adaptado de “Grokking Artificial Intelligence Algorithms: Understand and apply the core algorithms of deep learning and artificial intelligence” (Hurbans, 2020)

2.5.2. Búsqueda de coste uniforme

Se trata de una variación de la búsqueda en anchura, pero en este caso se tiene en cuenta el coste de cada acción. El nodo con el menor coste acumulado es expandido primero. Para implementarlo se usa una cola de prioridad, donde los nodos con menor coste acumulado

tienen mayor prioridad. Es óptimo siempre que el coste de las acciones sea no negativo y es completo siempre que el espacio sea discreto o finito.

En caso de que el coste sea el mismo en todos los nodos, la búsqueda de coste uniforme se comporta exactamente igual que la búsqueda en anchura, ya que el orden de expansión de los nodos será el mismo. Sin embargo, si los costes varían, la búsqueda de coste uniforme puede ser más eficiente al expandir primero los nodos con menor coste acumulado, lo que puede llevar a encontrar una solución óptima más rápidamente que la búsqueda en anchura.

2.5.3. Búsqueda en profundidad

Explora un nodo hijo antes de explorar los nodos hermanos. Repite el proceso para el resto de hijos que todavía no ha visitado. Para implementarlo se usa una pila LIFO ó *Last In First Out*, el nodo más recientemente visitado es el primero en ser procesado.

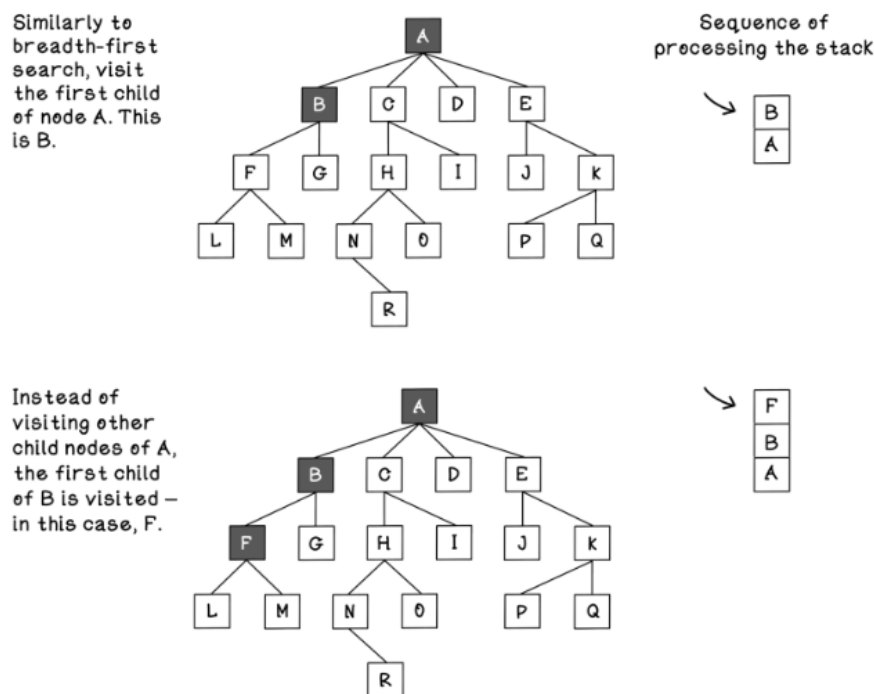


Figura 2.2: Representación árbol de nodos búsqueda en profundidad.

Fuente: Adaptado de “Grokking Artificial Intelligence Algorithms: Understand and apply the core algorithms of deep learning and artificial intelligence” (Hurbans, 2020)

No es completo en caso de un entorno no finito. Tampoco es óptimo en caso de entorno no finito. Es aconsejable utilizar la búsqueda en profundidad de manera limitada, se considera que los nodos por debajo del límite no tienen hijos.

También existe la alternativa al uso del algoritmo en profundidad iterativa, incrementando el límite del nivel de profundidad de manera iterativa, hasta que se alcance el objetivo. Es completo y óptimo siempre que el coste de las acciones sea el mismo.

2.5.4. Búsqueda A*

Existen distintas variantes de algoritmos de búsqueda informadas *Best Search*. La mayoría de estos algoritmos emplean una función heurística para estimar el coste de la ruta a nivel computacional más eficiente. Por ejemplo, el algoritmo el primero el mejor codicioso (GBFS) es el caso más básico, se expande aquel nodo que parece más cerca del objetivo según la estimación heurística.

La búsqueda A* es el más utilizado de esta categoría. Se consideran tanto el coste acumulado desde el nodo raíz, como el coste estimado mediante heurística hasta el objetivo. Es completo siempre que la heurística sea admisible. También se puede considerar óptimo y presenta una complejidad temporal y espacial de $O(b^d)$, donde b es el factor de ramificación y d es la profundidad de la solución óptima. Para que la heurística sea admisible, nunca debe sobrestimar el coste real y debe ser consistente. Se asume que el coste de cualquier transición es mayor que 0.

La combinación de búsquedas y la satisfacción de restricciones es una técnica habitual. La idea de CSP es representar problemas mediante la declaración de restricciones sobre un dominio y encontrar soluciones que satisfagan esas restricciones.

2.5.5. Algoritmo de Monte Carlo aplicado a Búsquedas

En diversos problemas o juegos, se puede construir, al menos en teoría, un espacio que se asemeja a un árbol de estados completo que puede contener todos los resultados posibles. Un algoritmo como Minimax se basa precisamente en la idea de evaluar la bondad de las jugadas de etiquetas de estados que se propagan hasta un estado actual. El objetivo es decidir cuál es el camino o árbol que conduce a mejores resultados al jugador. El problema que se encuentra con el algoritmo Minimax y derivados es que el espacio de búsqueda puede ser tan grande que resulta absolutamente inabarcable. En todos los algoritmos se han proporcionado métodos para mitigar el problema mediante acotaciones de la profundidad de búsqueda hacia adelante ó directamente la poda de ramificaciones para evitar visitar aquellos nodos que parecen resultar menos prometedores.

Como respuesta a este tipo de aproximaciones hace unos años surgió la técnica de *Monte*

Carlo Tree Search (Búsqueda en Árboles con Monte Carlo) que ofrece algunas ventajas inherentes. Se puede aplicar en espacios de búsqueda con muchas ramificaciones, espacios que antes eran inabarcables. No requiere conocimiento específico del juego y no siempre tiene por qué encontrar una solución óptima. *Upper Confidence Bound applied to Trees* (UCBT) es la variante más común que trata de equilibrar la exploración y explotación de nodos. La idea general es simular jugadas al azar desde una posición actual para realizar una muestra de cómo se comportan las distintas simulaciones. El algoritmo del límite superior de confianza (UCB) trata de equilibrar la exploración y explotación de nodos. Para que UCBT no genere un gran número de simulaciones aleatorias y minimizar riesgos, la variante UCT hace uso de estadísticas de bondad disponibles para todos los estados sucesores. MCTS cuenta con diversas fases:

- **Selección:** Se parte del nodo raíz y en cada nivel se elige la rama más prometedora utilizando una función de utilidad como UCT. Se continúa descendiendo hasta alcanzar un nodo terminal o un nodo que aún tiene acciones sin explorar.
- **Expansión:** Si el nodo seleccionado no está completamente explorado, se elige una de sus acciones no visitadas. Se crea un nuevo nodo hijo y se añade al árbol.
- **Simulación:** Desde el nuevo nodo se simula una partida o secuencia completa de acciones hasta llegar a un estado final. La simulación puede ser aleatoria o guiada por heurísticas. El resultado proporciona una recompensa o valoración de la rama explorada.
- **Retro-propagación:** La recompensa obtenida se propaga hacia atrás por todos los nodos recorridos durante la iteración. Se actualizan sus estadísticas y esta información se utilizará en futuras selecciones para decidir qué ramas explorar.

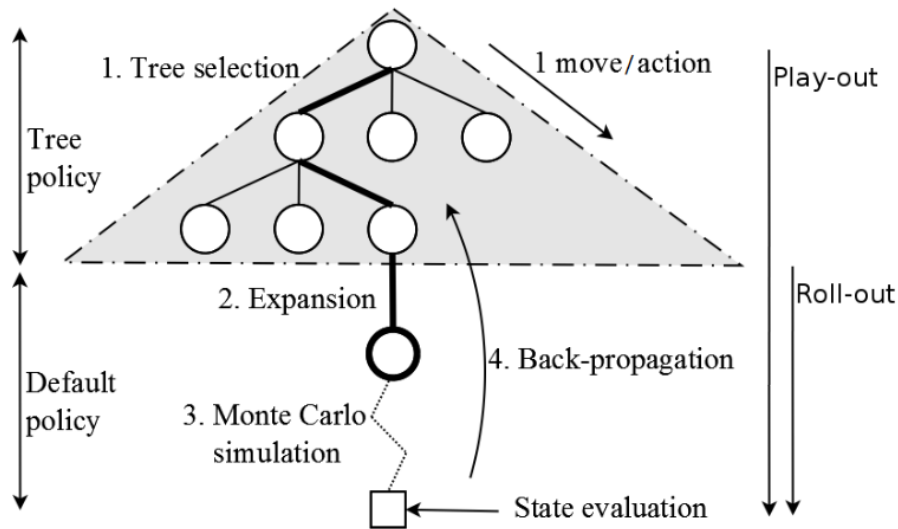


Figura 2.3: Pasos del algoritmo MCTS.

Fuente: Adaptado de “Knowledge-based fast evolutionary MCTS for general video game playing” (Perez, Samothrakis, y Lucas, 2014)

MCTS selecciona la rama más prometedora, expande una acción nueva, simula el resultado y actualiza las estadísticas del camino recorrido para mejorar futuras decisiones. Se pueden definir tres estrategias fundamentales que intervienen en distintas fases del algoritmo. Durante la fase de selección es necesario decidir qué nodo del árbol debe explorarse a continuación. Esta decisión puede tomarse utilizando diferentes criterios, como elegir el nodo con mayor recompensa acumulada, el que haya participado en un mayor número de simulaciones exitosas o, más habitualmente, aplicar una función que equilibre exploración y explotación, como UCB. En la fase de expansión se determina qué nuevo nodo añadir al árbol (Perez y cols., 2014). Aunque lo más común es seleccionar una acción disponible de forma aleatoria, también puede aprovecharse conocimiento específico del problema para guiar esta elección. Finalmente, en la fase de simulación se genera una secuencia completa de acciones desde el nuevo nodo hasta alcanzar un estado final. Estas simulaciones pueden realizarse de forma aleatoria o incorporar heurísticas y conocimiento del dominio para obtener estimaciones más precisas.

Actualmente existen variantes como Adaptive Branching MCTS (AB-MCTS) que introduce un mecanismo de ramificación adaptativa para mejorar la eficiencia en espacios de búsqueda con alta ramificación, o Sakana AI, un sistema de inteligencia artificial que utiliza MCTS para resolver *puzzles* del *benchmark* ARC-AGI.

2.5.6. Síntesis de programas

La síntesis de programas es un área de investigación que se centra en la generación automática de programas a partir de especificaciones o ejemplos. El objetivo es crear programas que cumplan con ciertos requisitos o resuelvan problemas específicos sin necesidad de escribir código manualmente.

Existen diferentes enfoques para la síntesis de programas, como la síntesis basada en ejemplos, donde se proporcionan ejemplos de entrada y salida para que el sistema genere un programa que cumpla con esos ejemplos. Otro enfoque es la síntesis basada en especificaciones, donde se proporcionan especificaciones formales del comportamiento deseado y el sistema genera un programa que satisfaga dichas especificaciones.

2.5.7. Búsqueda Guiada por Redes Neuronales

Históricamente, los algoritmos de búsqueda clásicos como A* o MCTS, funcionaban explorando sistemáticamente las posibles acciones futuras. Sin embargo, en dominios complejos como el ajedrez, o la síntesis de programas para ARC-AGI, el número de ramificaciones posibles crece de forma exponencial en cada paso (Świechowski, Godlewski, Sawicki, y Mańdziuk, 2023), generando una explosión combinatoria que hace imposible la resolución por fuerza bruta dentro de un límite de tiempo o memoria manejable.

Por otro lado, las redes neuronales profundas son excelentes reconociendo patrones e intuiciones de un solo vistazo, pero carecen de la capacidad algorítmica para planificar deliberadamente a largo plazo o corregir sus propios errores secuenciales. La búsqueda guiada por redes neuronales soluciona las carencias de ambos mundos inyectando la “intuición” del aprendizaje profundo dentro de la estructura lógica de los árboles de búsqueda. Esto se materializó entrenando redes neuronales para que actuaran como entes evaluadores dentro de algoritmos como MCTS.

En su versión clásica, MCTS depende de simulaciones masivas para estimar la calidad de cada rama. Sin embargo, en los enfoques híbridos modernos se incorporan redes neuronales que guían de forma explícita la exploración:

- **Red de Política (*Policy Network*):** En lugar de que el árbol de búsqueda explore todas las acciones legales posibles por igual, la red neuronal predice una distribución de probabilidad sobre cuáles son las acciones más prometedoras. Esto reduce drásticamente la amplitud de la búsqueda, podando inmediatamente las ramas inútiles.

- **Red de Valor (*Value Network* / *Q-function*):** En lugar de tener que simular el juego o el programa hasta su conclusión final para saber si es exitoso, la red neuronal evalúa la calidad o “bondad” del estado intermedio actual. Esto reduce drásticamente la profundidad de la búsqueda.

Gracias a estas capacidades de generalización y abstracción, el modelo híbrido logra converger hacia la solución correcta a una velocidad de órdenes de magnitud mayor que la búsqueda clásica, haciendo tratables problemas que antes se consideraban imposibles para las máquinas.

2.6. Abstract and Reasoning Corpus

Los sistemas de evaluación de la inteligencia artificial han ido evolucionando a lo largo del tiempo. La investigación de esta área se ha centrado históricamente en comprobar si las máquinas realizan bien tareas específicas, siendo una evaluación orientada a tareas y no una evaluación de la inteligencia intrínseca.

“[AI is] the science of making machines capable of performing tasks that would require intelligence if done by [humans].”

(Minsky, 1968)

Tras los métodos tradicionales de evaluación de algoritmos, la IA ha pasado a una evaluación de “caja negra” basada en el comportamiento que involucra la discriminación humana (comparación directa con humanos) y pruebas de rendimiento en problemas. Se iniciaron evaluaciones metodológicas de dudosa validez, donde los investigadores a menudo seleccionaban los conjuntos de datos que mejor les convenían y ajustaban parámetros en exceso provocando sobreajustes. Esto llevó a situaciones donde muchos resultados publicados eran reproducibles pero difícilmente replicables (Hernández-Orallo, 2017). En consecuencia, la aparente mejora del modelo desaparecía en cuanto cambiaba ligeramente el dominio de los datos. Además, existía una fuerte dispersión de los métodos en las diferentes disciplinas.

Para los sistemas de inteligencia artificial de propósito general era necesario un nuevo enfoque orientado a habilidades. Este enfoque se basa en la evaluación de capacidades cognitivas en lugar de tareas para las que fueron diseñados originalmente (Hernández-Orallo,

2017). Para garantizar la validez de este enfoque, las pruebas de evaluación requieren directrices estrictas, como definir con antelación los conjuntos de datos para entrenamiento y evaluación, y evitar el sobre-ajuste a conjuntos de datos específicos.

“The field of artificial intelligence has been very successful in developing artificial systems that perform these tasks without featuring intelligence.”

(Hernández-Orallo, 2017)

Para avanzar hacia sistemas de inteligencia artificial más inteligentes y similares a los humanos es necesario un método de evaluación de la inteligencia que compare dos o más sistemas entre sí y con seres humanos. Es aquí donde entra en juego el Abstract and Reasoning Corpus (ARC-AGI), un *benchmark* diseñado para respetar todas las directrices necesarias. ARC-AGI está construido con un conjunto de prioridades para ser lo más parecido posible a la capacidad de razonamiento innata del ser humano (Chollet, 2019b).

Los modelos que existen en la actualidad funcionan correctamente en tareas específicas, pero todavía están limitados y requieren grandes cantidades de datos. En situaciones en las que se desvían ligeramente de sus datos de entrenamiento, son incapaces de afrontar tareas que requieren un razonamiento novedoso. ARC-AGI también está pensado para medir el progreso de manera precisa y cuantitativa de la inteligencia. En este sentido, las definiciones que se han dado en secciones anteriores pueden ayudar a comprender la estructura de las pruebas de evaluación de ARC-AGI. Las tareas de evaluación no tienen por qué ser conocidas de antemano. Para conseguir la generalización, el agente debe ser capaz de aprender y gestionar nuevas tareas (Chollet, 2019b).

“We have been conceptualizing and evaluating intelligence in the context of AI research: crystallized skill on one hand, skill-acquisition ability on the other.”

(Chollet, 2019b)

2.6.1. Espectro de la generalización

La generalización es un concepto que ha llegado de la mano del aprendizaje automático, desarrollado originalmente para caracterizar el buen rendimiento de un modelo estadístico en datos de entrada que no forman parte de su conjunto de datos de entrenamiento. Este concepto ha seguido formando parte del debate y se puede definir como la “habilidad” de

un modelo para gestionar una situación o varias situaciones (o tareas) que no se ha encontrado previamente. Nótese que el concepto de la frase “situación que no se ha encontrado previamente” puede ser tremendamente ambiguo y Chollet (2019b) propone un espectro de generalización para aclarar esta ambigüedad. En este espectro se pueden encontrar diferentes tipos de generalización, como la generalización sistémica (centrada en el sistema) o la generalización consciente-desarrollada (capacidad de manejar situaciones que ni el sistema ni su creador humano habían encontrado antes), en las que no se profundizará en este trabajo. En cambio, se profundizará en la generalización de habilidades, que resulta más útil para cuantificar los grados de generalización.

- **Ausencia de generalización:** Sistemas donde no existe incertidumbre ni novedad. Entorno cerrado como un algoritmo que juega al “tres en raya”, iterando exhaustivamente sobre todas las jugadas posibles. No hay adaptación posible.
- **Generalización local o robustez:** Es la capacidad de un sistema para manejar nuevos ejemplos dentro de una distribución de datos conocida para una sola tarea o un conjunto bien definido de ellas. Este es el nivel en el que ha operado históricamente el aprendizaje profundo moderno. El autor (Chollet, 2019b) hace referencia a un ejemplo de clasificador de gatos en el que se clasifican gatos que el modelo no había visto antes.
- **Generalización amplia o flexibilidad:** Es la capacidad de manejar una categoría amplia de tareas y entornos relacionados sin intervención humana adicional, incluyendo situaciones imprevistas por los creadores (Chollet, 2019b). Un ejemplo de esto puede ser el coche autónomo de nivel 5.
- **Generalización extrema o generalidad:** Sistemas abiertos capaces de manejar tareas enteramente nuevas que solo comparten características abstractas con las ya conocidas y que son aplicables a cualquier dominio. También se puede denominar inteligencia general.

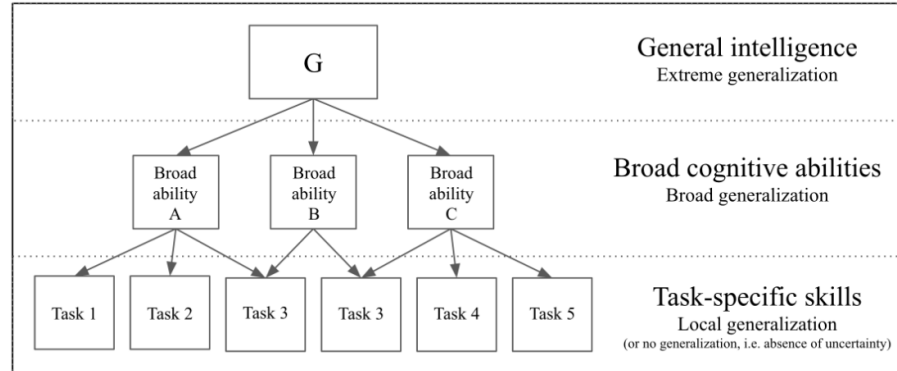


Figura 2.4: Modelo jerárquico de habilidades cognitivas y su mapeo del espectro de generalización.

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

2.6.2. La inteligencia desde la perspectiva de la adquisición de habilidades

Existen dos perspectivas fundamentales sobre la mente humana: una la considera una colección de habilidades que se han grabado en nuestro ADN durante la evolución y la otra la concibe como una mente en blanco. François Chollet, en el manifiesto “On the Measure of Intelligence” (Chollet, 2019b), sugiere que los humanos no nacen con una mente vacía que aprende cualquier conocimiento desde cero, pero tampoco son una colección de instintos. Se puede decir que nacemos con algunos conocimientos previos innatos (Spelke y Kinzler, 2007).

Para que un *benchmark* como ARC pueda ser una medida real de la inteligencia, no puede depender de conocimientos adquiridos y sus tareas deben poder resolverse utilizando únicamente los principios de los núcleos de conocimiento. De este modo, se puede medir la eficiencia con la que un agente utiliza sus conocimientos innatos para resolver problemas que no ha conocido antes.

François Chollet define la inteligencia no como un programa estático, sino como un proceso de síntesis de programas. Para establecer esta definición, hay que detallar en una primera instancia la posición del problema y el proceso técnico de una tarea según su marco formal. La arquitectura del sistema se compone de tres entidades principales:

- **El Sistema Inteligente (IS):** Es el agente que genera un programa de habilidad (*SP*) basándose en la experiencia recibida.

- **La Tarea (T):** Es el entorno que plantea un problema. La tarea dispone de una función de puntuación (éxito o fracaso) y de una función que devuelve retroalimentación al sistema inteligente.
- **El programa de habilidad (SP):** Artefacto producido por (IS), un programa ejecutable que trata de resolver la tarea y devuelve una respuesta. Representa la capacidad del sistema para gestionar el entorno o problema dado. Un sistema es más inteligente si proporciona programas de alta habilidad utilizando la menor cantidad de recursos posible en tareas de elevada dificultad de generalización.

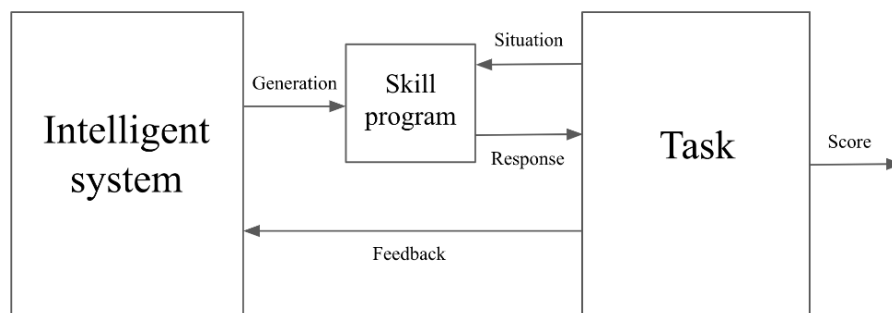


Figura 2.5: Posición del problema, un sistema inteligente que genera programas de habilidades que interactúan con una tarea.

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

El proceso de la resolución de una tarea se podría dividir en dos fases:

1. **Fase de entrenamiento o adquisición de habilidades:** Ciclo interactivo donde el sistema inteligente (IS) aprende a generar un programa de habilidad (SP) a través de la experiencia con la tarea (T). En esta fase, el sistema recibe retroalimentación sobre su desempeño y ajusta su programa de habilidad en consecuencia.
 - a. Fase de generación. Se presenta una situación de entrenamiento.
 - b. Fase de Producción. El IS genera un programa de habilidad (SP).
 - c. Fase de Acción. El SP interactúa con la tarea (T) para producir una respuesta.
 - d. Fase de Retroalimentación. La tarea evalúa la respuesta, se asigna una puntuación y se genera un *feedback*.
 - e. Fase de Actualización. El sistema inteligente (IS) recibe ese *feedback* para actualizar su estado o estados internos, mejorando su comprensión del problema

para una siguiente iteración.

2. Fase de evaluación o prueba de generalización: Una vez terminada la fase de entrenamiento, el *IS* entrega la versión definitiva de su programa de habilidad. En esta fase el programa se enfrenta a situaciones o entornos que no ha percibido anteriormente.

a. Fase de Desafío. Se presenta una situación nueva (test) que implica incertidumbre.

b. Fase de Ejecución. El programa de habilidad produce respuestas sin posibilidad de recibir retroalimentación.

c. Fase de Medición. Suma de la puntuación obtenida. Si el programa ha podido gestionar las situaciones nuevas, significa que se ha demostrado el principio de generalización.

En conclusión, ¿qué se puede esperar de un *benchmark* de inteligencia ideal? Un *benchmark* ideal debe ser explícito sobre su alcance, demostrar el éxito en las pruebas y presentar una correlación estadística con el éxito en un entorno real. Los resultados deben ser reproducibles, tal como señalaba José Hernández-Orallo (2017). Además, se debería medir la capacidad de generalización, es decir, medir la capacidad de gestionar situaciones que ni el sistema ni el desarrollador ha visto anteriormente. Esto indica que el conjunto de tareas dedicadas a la evaluación debe ser estrictamente privado para evitar que se inyecte la solución en el código o en los propios datos de entrenamiento (contaminación). Otro punto relevante que acompaña a este aspecto es el diseño para evitar trampas estadísticas que no requieran una verdadera comprensión abstracta.

El control de la experiencia debe ser fundamental. No se debe “comprar” el rendimiento de un *benchmark* mediante un conjunto ilimitado de datos de entrenamiento. Si un sistema inteligente necesita millones de datos de entrenamiento para generar programas de habilidad consistentes que un humano puede captar con apenas unos pocos ejemplos, significa que el sistema es ineficiente y no inteligente, aunque logre la misma puntuación. La cantidad de datos de entrenamiento debería estar estrictamente limitada. Por último, el *benchmark* debe funcionar de la misma forma para humanos y máquinas, de manera justa, asumiendo que ambos agentes poseen los mismos núcleos de conocimiento y teniendo en cuenta el tiempo y el volumen de entrenamiento de un humano.

2.6.3. ARC-AGI-1

Con el objetivo de dar respuesta a todas las necesidades mencionadas en las secciones anteriores, nace la propuesta del *benchmark* Abstraction and Reasoning Corpus (ARC), un repositorio de datos destinado a servir como referencia para medir la inteligencia general. En esta sección se van a explorar los objetivos y el diseño de este repositorio.

ARC-AGI como *benchmark* de síntesis de programas tiene diferentes objetivos finales:

- Mantener un formato cercano o simple para que pueda ser solucionable por humanos.
- Centrar el foco en medir la generalización y no solo las habilidades. La generalización se debe medir de manera cuantitativa. Presentar tareas altamente abstractas que deben ser entendidas por el ser humano con pocos ejemplos.
- Control cuantitativo de la experiencia proporcionada (datos de entrenamiento limitados). Los datos de entrenamiento tendrán una cantidad fija.
- Describir el conjunto completo de núcleos de conocimiento y comparar de manera “precisa” la inteligencia humana con la de los agentes de inteligencia artificial.

En la imagen siguiente 2.6 se muestra un ejemplo de una tarea de ARC-AGI-1. En esta tarea, el objetivo es completar un patrón de un área dada. La solución de esta tarea pasa por generar una cuadrícula de salida que corresponde con la entrada a su izquierda. Para resolver esta tarea, el agente debe ser capaz de identificar el patrón subyacente en la entrada y generar la salida correcta, demostrando así su capacidad para generalizar a partir de los ejemplos proporcionados.

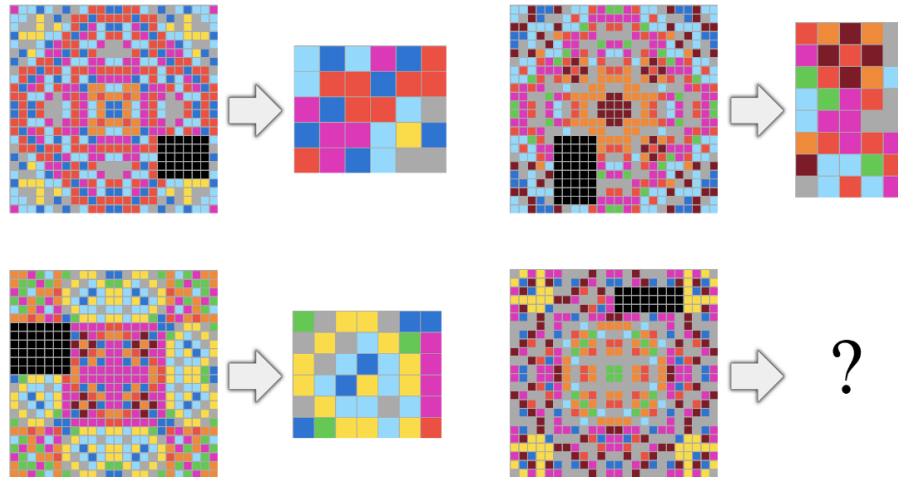


Figura 2.6: Tarea donde el objetivo es completar un patrón de un área dada. Para solucionar esta tarea se debe generar una cuadrícula de salida que se corresponda con la entrada situada a su izquierda.

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

2.6.4. Núcleos de conocimiento aplicado en ARC-AGI

Todos los problemas planteados están diseñados específicamente para abordar los núcleos de conocimiento. Los núcleos de conocimiento de ARC están planteados para ser lo más parecido posible a los núcleos de conocimiento del ser humano.

Objetualidad

Núcleo de conocimiento centrado en la capacidad de interpretación del entorno mediante el análisis de las diferentes entidades. Se fundamenta en el principio de cohesión y persistencia de los objetos. Se analiza y se separa la cuadrícula de la entrada en diferentes objetos basándose en valores o criterios de continuidad como los bloques del mismo color.

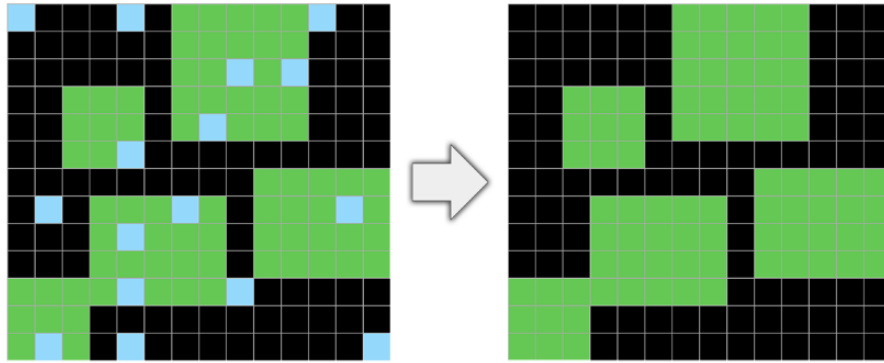


Figura 2.7: Prueba de ruido (*denoising*).

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

Generalmente, los objetos presentes en la cuadrícula de entrada persisten en la salida, a menudo con transformación geométrica. A pesar del ruido visual o la inclusión provocada por otros objetos, muchos de los objetos persisten.

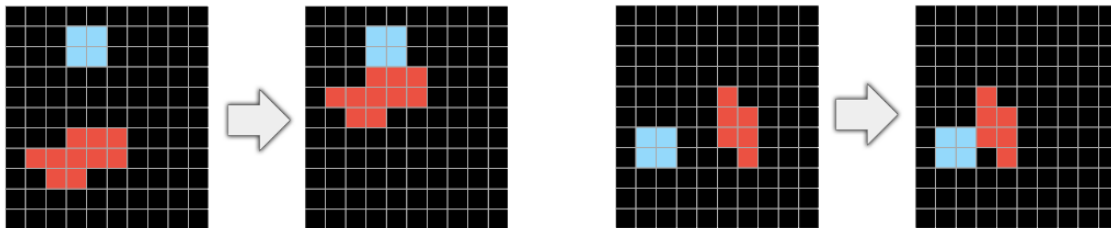


Figura 2.8: El objetivo rojo debe “moverse” junto al objeto azul hasta hacer “contacto”.

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

La influencia mediante el contacto es otro punto especialmente relevante. Muchas tareas requieren entender ciertas interacciones físicas, como un objeto que se desplaza hasta tocar a otro o una línea que se dibuja y rebota al entrar en contacto con un obstáculo.

Orientación a objetivos

Las cuadrículas de entrada y salida suelen ser comprendidas por los humanos como los estados iniciales y finales de un proceso que conlleva intencionalidad. Las transformaciones buscan alcanzar una meta y resulta ser una heurística útil para resolver *puzzles*.

Números y conteo

Diversas tareas de ARC implican contar y clasificar objetos. Las tareas pueden tener distintos tamaños y es especialmente relevante la comparación de magnitudes. No es sólo contar objetos o “puntos”, es también identificar aquellos símbolos que aparecen en más ocasiones (conurrencia). Son necesarias nociones básicas de suma y resta en cantidades pequeñas. En ARC las cantidades utilizadas no superan el número 9. Esto refleja una capacidad innata de los humanos para la representación numérica. En secciones anteriores se ha explorado esta capacidad como uno de los núcleos de conocimiento innato. El ser humano es capaz de usarla sin formación previa, aunque con limitaciones en la cantidad de objetos que puede contar.

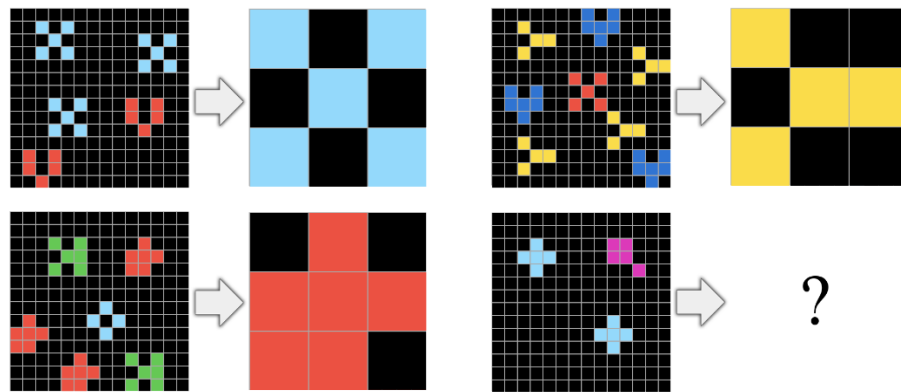


Figura 2.9: Tarea donde el objetivo es contar el objeto que aparece más veces.

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

Geometría y Topología básica

La orientación espacial es un aspecto fundamental como núcleo de conocimiento. Se engloban aquellas tareas de reconocimiento de formas para la comprensión de líneas u otras figuras geométricas. El agente debe ser capaz de estimar con mayor probabilidad la aparición de formas regulares frente a otras formas más complejas.

Las transformaciones espaciales, como las simetrías, las rotaciones y el escalado, deben ser comprendidas y aplicables. En diversas tareas también se requieren relaciones espaciales basadas en nociones topológicas, como contener o estar contenido y situarse fuera o dentro de un perímetro delimitado. El trazado de líneas, la conexión de puntos (proyecciones ortogonales) o la copia de diversos objetos repetidamente también se incluyen en este

núcleo de conocimiento.

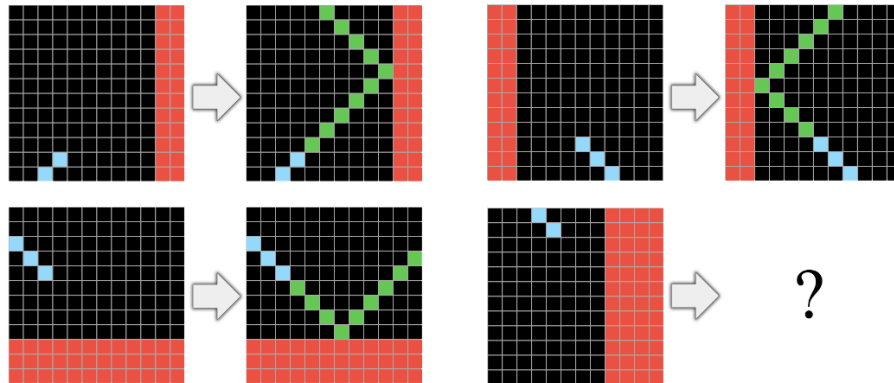


Figura 2.10: Tarea donde el objetivo es generar una línea diagonal hasta rebotar al hacer contacto con el obstáculo rojo.

Fuente: Adaptado de “On the Measure of Intelligence” (Chollet, 2019b)

2.6.5. Repositorio de datos

Los datos de ARC-AGI se pueden encontrar en el repositorio de GitHub (Chollet, 2019a). Los datos están contenidos en dos subdirectorios diferentes:

- **Training:** Contiene 400 tareas de entrenamiento. Cada tarea contiene todo el contenido en un archivo; por lo tanto, son 400 archivos. Todos los archivos de este directorio sirven para el entrenamiento del modelo y la adquisición de habilidades.
 - a. Pares “train”: Típicamente suele tener tres o más pares de “arrays”. Cada par de “arrays” contiene un “array” de “input” y otro de “output”.
 - b. Pares “test”: Típicamente suele tener un par de “arrays”. El par de “arrays” contiene un “array” de “input” y otro de “output”.
- **Evaluation:** Contiene 400 tareas de evaluación. Cada tarea contiene todo el contenido en un archivo, por lo tanto, son otros 400 archivos.

Los archivos están en formato JSON. Cada archivo contiene un diccionario con dos claves: “train” y “test”. El valor de cada clave es una lista de pares de “arrays”. Cada par de “arrays” contiene un “array” de “input” y otro de “output”. Los “arrays” son matrices bidimensionales de números enteros que representan los estados iniciales y finales de una tarea.

En la Ecuación (2.1) se muestran una matriz de entrada X y su correspondiente matriz de salida Y :

$$X = \begin{pmatrix} 0 & 7 & 7 \\ 7 & 7 & 7 \\ 0 & 7 & 7 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & 0 & 0 & 0 & 7 & 7 & 0 & 7 & 7 \\ 0 & 0 & 0 & 7 & 7 & 7 & 7 & 7 & 7 \\ 0 & 0 & 0 & 0 & 7 & 7 & 0 & 7 & 7 \\ 0 & 7 & 7 & 0 & 7 & 7 & 0 & 7 & 7 \\ 7 & 7 & 7 & 7 & 7 & 7 & 7 & 7 & 7 \\ 0 & 7 & 7 & 0 & 7 & 7 & 0 & 7 & 7 \\ 0 & 0 & 0 & 0 & 7 & 7 & 0 & 7 & 7 \\ 0 & 0 & 0 & 7 & 7 & 7 & 7 & 7 & 7 \\ 0 & 0 & 0 & 0 & 7 & 7 & 0 & 7 & 7 \end{pmatrix} \quad (2.1)$$

2.6.6. ARC-AGI-2

La introducción de ARC-AGI-2 en 2025 supone una evolución crítica impulsada por los avances observados y por los modos de fallo identificados en los sistemas de inteligencia artificial sobre el conjunto de datos original. Aunque los sistemas de IA más avanzados podían alcanzar puntuaciones elevadas en ARC-AGI-1, a menudo mediante búsquedas por fuerza bruta, ARC-AGI-2 fue diseñado para ser menos vulnerable a este tipo de métodos. Además, se creó específicamente para evaluar debilidades conocidas en los sistemas modernos de razonamiento de IA.

ARC-AGI-2 propone nuevos desafíos conceptuales. Basándose en los fallos observados en los modelos de IA de vanguardia, ARC-AGI-2 incorpora tareas destinadas a evaluar capacidades de razonamiento nuevas y más complejas:

- **Interpretación simbólica:** tareas en las que los símbolos visuales deben interpretarse como portadores de un significado semántico más allá de su forma. Por ejemplo, cuando una figura representa una acción.
- **Razonamiento composicional:** tareas que requieren descubrir y aplicar simultáneamente múltiples reglas que interactúan entre sí.
- **Aplicación contextual de reglas:** tareas en las que la regla correcta depende del contexto específico dentro de la cuadrícula, superando la simple identificación de patrones globales superficiales.

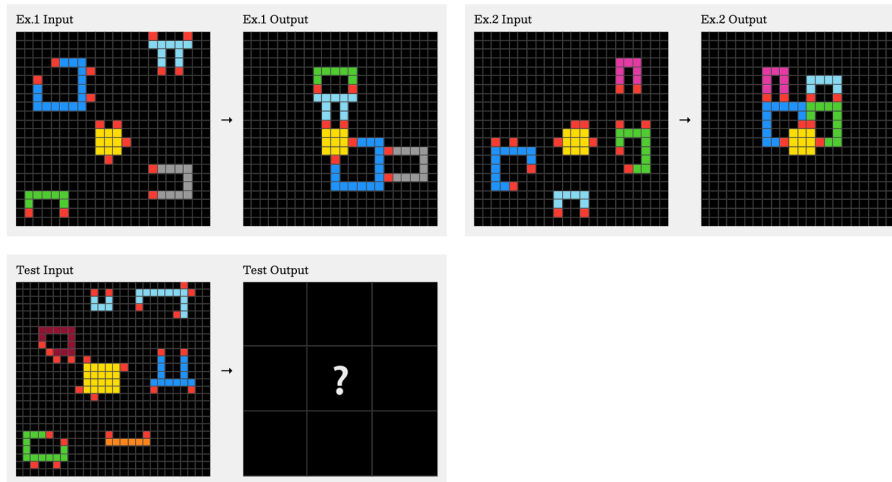


Figura 2.11: Tarea ARC-AGI 2: cbebaa4b.

Fuente: Adaptado de “ARC-AGI-2: A New Challenge for Frontier AI Reasoning Systems” (Chollet, Knoop, Kamradt, Landers, y Pinkard, 2025)

2.6.7. ARC-AGI-3

En marzo de 2026, la Fundación ARC Prize introdujo un cambio fundamental en el *benchmark* ARC-AGI-3, trasladando el paradigma de evaluación de *puzzles* estáticos a entornos interactivos. A diferencia de versiones anteriores, se coloca al agente en un juego por turnos sin instrucciones, sin objetivo explícito ni reglas definidas. El agente debe explorar el entorno, deducir las mecánicas del sistema y determinar por sí mismo la meta a alcanzar.

Capacidades evaluadas

Este enfoque interactivo evalúa habilidades de razonamiento distintas a las medidas en ARC-AGI-1 y ARC-AGI-2:

- **Exploración eficiente:** capacidad de interactuar con el entorno de forma estratégica en lugar de aleatoria.
- **Búsqueda de objetivos:** deducción de metas cuando no existen instrucciones explícitas.
- **Memoria y aprendizaje:** retención de experiencias previas para optimizar decisiones futuras.

- **Mejora iterativa:** progresión en el rendimiento a lo largo de interacciones sucesivas.

La evaluación se basa en la eficiencia de las acciones del agente comparada con la eficiencia humana, promediada sobre aproximadamente 135 juegos diseñados manualmente. Una puntuación del 100 % indica paridad con el desempeño humano.

Resultados preliminares

En el lanzamiento de marzo de 2026, todos los modelos de frontera obtuvieron puntuaciones inferiores al 1 %, incluyendo GPT-5.x, Gemini 3 y Claude Opus 4.x. En contraste, los humanos resolvieron prácticamente la totalidad de los juegos. Los enfoques más efectivos combinaron modelos basados en redes neuronales convolucionales entrenadas con aprendizaje por refuerzo para predecir cambios ambientales, junto con representaciones gráficas del espacio de estados. Los agentes basados únicamente en lenguaje mostraron un desempeño significativamente inferior, indicando que este dominio requiere técnicas de aprendizaje por refuerzo y modelado del mundo más que ingeniería de *prompts*.



Figura 2.12: Tarea ARC-AGI 3: ls20.

Fuente: Adaptado de “ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence” (Chollet y cols., 2025)

2.6.8. Comparativa entre sistemas ARC-AGI

Característica	ARC-AGI-1	ARC-AGI-2	ARC-AGI-3
Lanzado	2019	2025	2026
Formato	Cuadrícula estática	Cuadrícula estática (salida más difícil)	Juegos interactivos por turnos
Requerimiento	Generalización, abstracción básica	Razonamiento simbólico, composicional y contextual	Exploración, búsqueda de objetivos, aprendizaje dinámico
Fuerza bruta	Sí	No, por diseño	Nuevo paradigma
Base humana	Alta	~60–66 % individual / 100 % panel	Los humanos resuelven ~ todas
E. Resolución	~90–96 %	~85 % sin restricciones / ~24 % con límites de competencia	<1 %

Tabla 2.1: Comparación de las versiones de ARC-AGI.

2.7. Primeros enfoques para resolver ARC-AGI

2.7.1. Domain Specific Languages (DSLs) y síntesis de programas

En 2020, Johan Sokrates Wind publicó uno de los primeros intentos basados en un lenguaje específico de dominio o DSL. El uso de código estrictamente diseñado para resolver tareas específicas de ARC-AGI permitió alcanzar una puntuación de 20 % en el *benchmark*. La idea detrás de esto es codificar algoritmos de programación para crear un conjunto de programas primitivos capaces de modificar matrices bidimensionales.

Los algoritmos realizan búsquedas exhaustivas sobre el espacio de programas posibles que se puede construir cambiando núcleos de conocimiento. Esta mecánica se puede considerar casi fuerza bruta. Formalmente, el método evalúa de manera iterativa funciones unarias sobre fragmentos de las cuadrículas de entrada, formando un Grafo Acíclico Dirigido (DAG) que representa múltiples secuencias de transformaciones.

Según el propio autor, la búsqueda de nivel 4, una de las más altas, dura en torno a 9 horas usando solamente la CPU. Este enfoque no requiere ningún preentrenamiento y

sus conocimientos previos son inyectados mediante líneas de código. El espacio de búsqueda crece de manera exponencial a medida que aumenta el tamaño del DSL y, para los rompecabezas que requieren más pasos lógicos, la búsqueda simplemente agota el tiempo disponible. Otro aspecto negativo del enfoque es que, si un problema de transformación no se programó específicamente, el sistema no es capaz de resolverlo. Escalar este enfoque requiere capacidad humana constante para seguir añadiendo conocimientos primitivos y heurísticas de poda diseñadas manualmente para guiar la búsqueda (Wind, 2020).

2.7.2. Enfoque Neurosimbólico

El enfoque propuesto por Simon Alford (2021) aborda el reto de la inteligencia general artificial utilizando el *benchmark ARC-AGI*. Alford (2021) parte de la premisa de que los sistemas tradicionales de aprendizaje profundo fallan en la generalización porque tienden a memorizar datos en lugar de aprender procedimientos. Para superar esto, la tesis propone un enfoque neurosimbólico inspirado en la teoría de los sistemas de pensamiento, que combina la rapidez asociativa de las redes neuronales (Sistema 1) con la capacidad estructurada y lógica del razonamiento simbólico (Sistema 2). La solución se divide en dos grandes pilares dependientes, la abstracción y el razonamiento.

Para el primer pilar, la abstracción, el sistema emplea un motor de síntesis de programas llamado *DreamCoder*. La clave de este motor radica en su algoritmo de compresión. A medida que el agente resuelve problemas básicos combinando funciones sencillas, como rotar o espejar una matriz, analiza los programas exitosos para identificar patrones recurrentes. Luego, empaqueta estas secuencias comunes en nuevas funciones de alto nivel, ampliando así su vocabulario de comandos y logrando generalizar su conocimiento hacia tareas cada vez más complejas que el desarrollador original no había anticipado. No obstante, Alford (2021) advierte que la búsqueda guiada por redes neuronales estándar de DreamCoder es insuficiente para escalar y resolver programas largos o con muchas operaciones repetidas, algo muy habitual en los *puzzles* de ARC-AGI (Alford, 2021).

Para solventar esta limitación en la búsqueda, Alford (2021) desarrolla el segundo pilar, un modelo de razonamiento avanzado basado en la síntesis guiada por ejecución bidireccional. En lugar de intentar predecir el código completo a ciegas, el sistema ejecuta el programa paso a paso (hacia adelante o *bottom-up**), observando los resultados intermedios en la matriz para decidir la siguiente acción. Simultáneamente, el sistema emula la deducción humana trabajando hacia atrás (*top-down**) mediante el uso de la semántica

inversa y la inversa condicional. Esto significa que el agente analiza la imagen final deseada y deduce matemáticamente qué entradas intermedias habrían sido necesarias para producirla. Todo este proceso se modela matemáticamente como un Proceso de Decisión de Markov, donde se busca conectar los nodos de entrada iniciales con el nodo de salida objetivo.

En conclusión, la tesis sostiene que ninguno de los dos enfoques es completamente suficiente por sí solo para dominar el *benchmark ARC*. La arquitectura ideal requiere la unificación de ambos sistemas: se necesita la capacidad de compresión de DreamCoder para generar abstracciones que enriquezcan el lenguaje disponible, y se requiere la potencia de la búsqueda bidireccional para navegar el inmenso espacio de posibles programas combinando inteligentemente las deducciones lógicas inversas con la evaluación visual paso a paso.

2.8. Grandes Modelos de Lenguaje en ARC-AGI-1

Desde 2019 se han evaluado los grandes modelos de lenguaje (LLM) en el contexto de ARC-AGI-1. Gendron, Bao, Witbrock, y Dobbie (2024) analizan una de estas debilidades fundamentales de los LLM, el razonamiento abstracto. Aunque estos modelos destacan en tareas de procesamiento de lenguaje natural como la traducción o la generación de código, los mecanismos detrás de este éxito son opacos, lo que genera dudas sobre si realmente poseen capacidades cognitivas similares a las humanas o si sus resultados provienen de la memorización de datos. El razonamiento abstracto, el cual consiste en identificar patrones generales a partir de muy pocos ejemplos, es ideal para evaluar la capacidad de generalización profunda de un sistema. Se han podido probar una amplia gama de arquitecturas modernas, desde modelos de la familia GPT hasta modelos de código abierto como LLaMA, Alpaca y Zephyr.

Los resultados de los experimentos revelan de manera contundente que los LLMs actuales tienen un rendimiento muy deficiente en tareas de razonamiento abstracto. En las pruebas abiertas, modelos como GPT-4 y Text-Davinci-3 obtuvieron los mejores resultados, pero aun así fallaron en la gran mayoría de los rompecabezas. Además, Gendron y cols. (2024) advierten que el relativo “éxito” de GPT-4 en pruebas clásicas como las matrices de Raven podría deberse a que dichas pruebas ya estaban incluidas en sus datos de entrenamiento, más que a una verdadera capacidad de deducción. En los escenarios de opción múltiple, el rendimiento también se mantuvo cercano a la aleatoriedad, demostrando

que los modelos ni siquiera son capaces de reconocer la respuesta correcta cuando se les da a elegir, lo que los hace ineficaces para tareas de auto-evaluación o auto-refinamiento.

Las técnicas de *prompting*, que habitualmente mejoran el rendimiento en otras áreas del lenguaje natural, no funcionan en este dominio. Estrategias como la “Cadena de Pensamiento” (*Chain-of-Thought*) o pedirle al modelo que escriba código informático para resolver el problema no proporcionaron mejoras significativas. Por otro lado, al aplicar *fine-tuning* a modelos como LLaMA2, se logró una precisión casi perfecta en problemas similares a los del entrenamiento, pero fracasaron estrepitosamente al evaluar variaciones fuera de esa distribución, como agrupar múltiples objetos. Esto indica que el modelo memoriza reglas de sintaxis pero no adquiere una verdadera capacidad de abstracción. Curiosamente, los modelos muestran un mejor rendimiento cuando los problemas se representan mediante vectores numéricos en lugar de texto en lenguaje natural, ya que este último introduce sesgos y distracciones perjudiciales.

Los Grandes Modelos de Lenguaje actuales carecen de las propiedades fundamentales necesarias para lograr un razonamiento abstracto sólido (Gendron y cols., 2024). El verdadero cuello de botella de estos sistemas no es la falta de comprensión de las instrucciones del usuario, sino su incapacidad para reconocer y abstraer nuevos patrones. Para superar estas barreras y acercarse a una cognición similar a la humana, Gendron y cols. (2024) postulan que la inteligencia artificial requerirá de nuevos enfoques enfocados específicamente en la inducción de programas y el razonamiento causal.

Estas limitaciones fundamentales han motivado una línea de investigación paralela enfocada en arquitecturas alternativas que buscan superar las debilidades identificadas en los LLMs. Los siguientes enfoques exploran modelos más especializados y eficientes en términos computacionales, combinando técnicas neurosimbólicas, transformers compactos y síntesis de programas mejorada, con el objetivo de lograr un razonamiento abstracto más robusto y generalizable.

2.9. Modelos basados en la Neurodiversidad

Una línea de trabajo particularmente original dentro del ecosistema ARC-AGI es la propuesta por Ainooson, Sanyal, Michelson, Yang, y Kunda (2023), que toma como punto de partida hallazgos de la psicología cognitiva y de la investigación sobre neurodiversidad para replantear el modo en que un sistema artificial representa y manipula regularidades

visuales. Su tesis central es que muchas tareas de ARC-AGI no fallan por ausencia de potencia de cálculo en sentido estricto, sino porque los sistemas habituales carecen de una forma adecuada de representar conocimiento nuclear sobre objetos, relaciones espaciales, continuidad visual y transformaciones topológicas. Desde esta perspectiva, el problema no se reduce a encontrar una salida correcta, sino a disponer de una representación interna capaz de operar sobre configuraciones visuales de forma flexible, composicional y cercana a la manera en que ciertos sujetos humanos resuelven problemas abstractos mediante imaginación visual.

El núcleo de la propuesta se articula en torno a un lenguaje de razonamiento visual denominado VIMRL, concebido como un lenguaje especializado para tareas de ARC-AGI (Ainooson y cols., 2023). En lugar de depender exclusivamente de descripciones simbólicas de alto nivel, VIMRL trabaja con tipos como imagen, objeto, color, número y listas de objetos, y permite construir programas que encadenan operaciones de bajo y alto nivel. Las operaciones de bajo nivel realizan transformaciones explícitamente definidas sobre las rejillas, mientras que las de alto nivel analizan los ejemplos de entrenamiento para inducir reglas locales que luego pueden aplicarse al caso de prueba.

Sobre esa base representacional, el solucionador explora el espacio de programas VIMRL mediante técnicas clásicas de síntesis y poda, aceptando como candidatos aquellos programas que alcanzan suficiente precisión sobre los pares de entrenamiento y priorizando después soluciones pequeñas y salidas diversas (Ainooson y cols., 2023). El trabajo subraya que la dificultad principal no reside solo en definir operaciones útiles, sino en contener la explosión combinatoria del espacio de búsqueda. Para ello introduce restricciones sobre profundidad, reutilización de variables, eliminación de programas lógicamente equivalentes y supresión de referencias redundantes a la entrada original. Este punto es relevante porque muestra una tensión recurrente en ARC-AGI entre expresividad y eficiencia computacional. Cuanto más rico es el lenguaje de operaciones, mayor es la capacidad de representar conceptos abstractos, pero también crece el coste de explorar programas plausibles.

Los resultados obtenidos por Ainooson y cols. (2023) respaldan el interés de este enfoque, aunque también dejan ver sus límites. Su mejor configuración resolvió 111 tareas del conjunto público de entrenamiento y 45 del conjunto de evaluación. Estas cifras no suponen una resolución general del problema, pero sí aportan evidencia de que una arquitectura neuro-inspirada, combinada con síntesis de programas, puede capturar regularidades que escapan tanto a enfoques puramente neuronales como a formulaciones simbólicas demasia-

do rígidas. En el marco de este trabajo, esta aportación es especialmente valiosa porque refuerza la idea de que los avances en ARC no dependen únicamente de aumentar escala o parámetros, sino de diseñar mecanismos de inferencia mejor alineados con la estructura del razonamiento abstracto humano.

2.9.1. Inducción de Síntesis de programas mediante Redes Neuronales Eficientes

En 2024 comienzan a aparecer trabajos que combinan síntesis de programas con aprendizaje profundo para guiar la búsqueda de soluciones en ARC-AGI. Se estudia cómo extender la síntesis de programas para ARC-AGI mediante guiado neuronal.

La idea parte de una premisa metodológica importante. En ARC-AGI no basta con aprender a interpolar sobre una distribución cerrada de ejemplos, porque el conjunto de evaluación incluye estructuras distintas de las vistas en entrenamiento. Por ello, el problema se formula como una tarea de inducción de programas sobre un DSL dado, donde el sistema debe encontrar, dentro de un presupuesto limitado de tiempo y memoria, el programa capaz de transformar correctamente las entradas de entrenamiento en las salidas objetivo. Esta formulación sitúa el foco en la eficiencia de la búsqueda y en la capacidad de generalización estructural, dos ejes que también son centrales en la presente investigación.

“We place the additional constraint to our problem domain that the test set must be out-of-distribution with respect to the training set in terms of the compositional structures of P .”

(Ouellette, 2024, p. 2)

Ouellette (2024) ordena su propuesta en tres formas de abordar el problema. La primera, *Learning the Grid Space* (LGS), se centra en comparar la rejilla que el sistema va obteniendo con la rejilla final correcta, para decidir qué paso conviene dar a continuación. La segunda, *Learning the Program Space* (LPS), desplaza la atención desde la imagen hacia el propio programa y trata de predecir qué secuencia de operaciones tiene más posibilidades de resolver la tarea. La tercera, *Learning the Transformation Space* (LTS), intenta combinar las ventajas de las dos anteriores, porque no solo propone operaciones probables, sino que también tiene en cuenta cómo va cambiando el estado parcial de la solución a medida que se ejecuta (Ouellette, 2024).

La aportación técnica principal del artículo reside en GridCoder, una implementación del paradigma LPS. En este enfoque, una arquitectura de tipo *Vision-Language Model* recibe pares de rejillas de entrada y salida y produce, de forma autorregresiva, probabilidades sobre *tokens* que codifican primitivas del DSL. A diferencia de otros enfoques clásicos apoyados en n-gramas o en expansión puramente simbólica, GridCoder utiliza la secuencia completa de probabilidades emitidas por el decodificador para construir una enumeración probabilística del espacio de programas. El algoritmo calcula una distribución inicial, la refuerza mediante un pequeño proceso de *bootstrapping* con distintos ejemplos y prefijos, y después ordena los programas candidatos por probabilidad conjunta antes de evaluarlos ejecutivamente (Ouellette, 2024). Un rasgo especialmente interesante es que el autor introduce deliberadamente una hipótesis de independencia condicional aproximada entre posiciones de la secuencia durante la fase de búsqueda. Aunque esa simplificación no es teóricamente exacta, reduce de manera notable el número de consultas al modelo y mejora la eficiencia práctica del procedimiento.

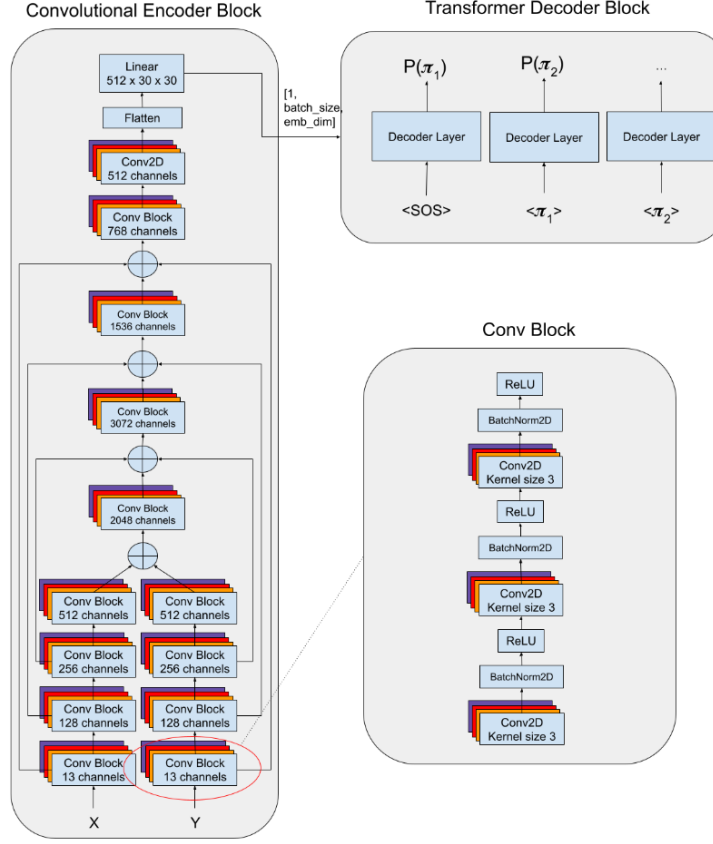


Figura 2.13: Arquitectura VLM utilizada en GridCoder.

Fuente: Adaptado de “Towards Efficient Neurally-Guided Program Induction for ARC-AGI” (Ouellette, 2024)

El estudio también examina cómo evoluciona el sistema al ampliar progresivamente el DSL. Se construyen varias versiones del lenguaje, comenzando por primitivas puramente rejilla-a-rejilla e incorporando después operaciones de detección, filtrado y manipulación de objetos. Los resultados muestran que el enfoque seleccionado supera tanto a la red neuronal sin búsqueda como a variantes guiadas por similitud aprendida entre rejillas, y además escala de manera razonablemente favorable al crecer el número de primitivas (Ouellette, 2024). En el conjunto de evaluación de ARC-AGI, GridCoder alcanza un 5,75 % con la primera versión del DSL y mejora hasta el 8,25 % con versiones más amplias, mientras que el tiempo medio normalizado por primitiva no empeora de forma proporcional. Este hallazgo es relevante porque sugiere que una mayor expresividad del DSL no implica necesariamente una penalización explosiva si la enumeración está bien guiada por el modelo neuronal.

Sin embargo, el artículo no se limita a presentar mejoras cuantitativas, sino que identifica con bastante precisión el principal cuello de botella del enfoque. Según Ouellette (2024), GridCoder generaliza razonablemente bien ante desplazamientos en la distribución visual de las rejillas, pero sigue mostrando debilidades claras cuando debe afrontar estructuras de programa no observadas durante el entrenamiento. En otras palabras, el modelo aprende con eficacia qué primitivas locales parecen plausibles, pero tiende a fijar con excesiva rigidez los esqueletos de los programas. Este diagnóstico es especialmente valioso para el presente trabajo, porque refuerza la idea de que la mera predicción autoregresiva de secuencias no basta para capturar razonamiento abstracto robusto si no se reintroduce algún mecanismo de comprobación o refinamiento sobre estados intermedios.

Un modelo verdaderamente útil para ARC debería recibir de forma explícita el estado parcial de ejecución y utilizarlo para decidir la siguiente transformación, en lugar de depender solo de la secuencia de *tokens* ya emitida y del emparejamiento global entre entrada y salida. Esa idea aproxima la síntesis de programas a un proceso iterativo de razonamiento, donde cada paso puede corregirse en función del resultado parcial obtenido. En términos conceptuales, esta hipótesis conecta de forma directa con la lógica de los modelos compactos con refinamiento iterativo, con los esquemas de *test-time compute* y con la necesidad de invertir cómputo de forma adaptativa solo cuando la tarea lo exige.

Por un lado, Ouellette (2024) ofrece evidencia empírica de que la búsqueda guiada por redes neuronales puede hacer viable una síntesis de programas más eficiente que la enumeración ciega, incluso cuando el DSL crece y las tareas exigen combinar múltiples primitivas. Por otro, delimita con precisión por qué ese mismo enfoque sigue siendo insuficiente para una generalización fuerte en ARC-AGI, lo que justifica explorar arquitecturas compactas capaces de mantener y refinar estados latentes intermedios durante la inferencia. Así, el trabajo no solo aporta un antecedente técnico sólido, sino también una justificación directa para investigar modelos pequeños que razonen en bucle, ajusten dinámicamente su computación y combinen inducción estructurada con evaluación progresiva del estado interno.

2.10. Afinamiento eficiente de modelos para ARC-AGI

En 2024, un grupo de investigadores realizó un hito histórico alcanzando una puntuación de 56 % en el concurso público de ARC Prize. El éxito de este enfoque no radica en el

uso de redes neuronales de gran tamaño ni en la síntesis de programas propiamente dicha, sino más bien en una estrategia de optimización extrema de modelos de lenguaje pequeños (SLM). Estos modelos no son especialmente compactos, pero con técnicas de optimización pueden llegar a ser lo suficientemente eficientes como para poder ser ejecutados en entornos limitados.

Los concursos de ARC Prize exigen que los modelos sean desplegados y ejecutados en entornos de computación especialmente limitados, con una capacidad de computación equivalente a dos GPU Nvidia T4 o una Nvidia P100, todas ellas con 16 GB de memoria. Franzen, Disselhoff, y Hartmann (2024) consiguen un ajuste fino eficiente, una tokenización específica y una adaptación en tiempo de prueba mediante *Test-Time Training*.

El equipo realiza un proceso de pre-entrenamiento masivo a partir de muchos datos aumentados sintéticamente. Partiendo del conjunto de datos original, se generaron casi 400.000 tareas adicionales con el repositorio público de Re-ARC. Además, se añadieron conjuntos de datos de entrenamiento proporcionados por la comunidad, como Concept-ARC y ARC-Heavy. Tras realizar una mezcla estructural de más de medio millón de tareas, se aplicó otro proceso de aumento de datos hasta alcanzar 290 millones de tareas sintéticas. Con tal magnitud de datos, el preentrenamiento del modelo debe hacerse mediante computación en la nube acelerada de considerable potencia.

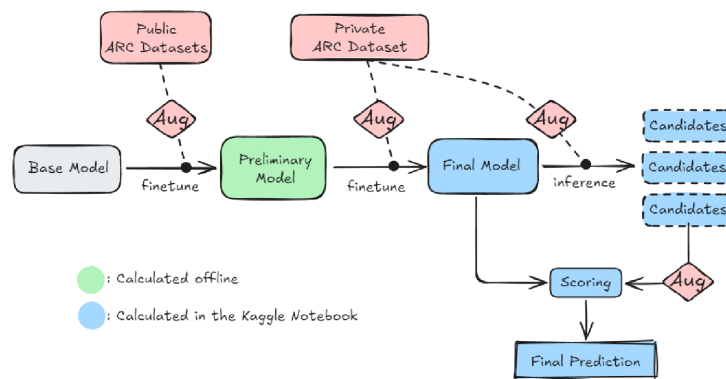


Figura 2.14: Flujo de proceso de pre-entrenamiento e inferencia.

Fuente: Adaptado de “The LLM ARChitect: Solving ARC-AGI Is A Matter of Perspective” (Franzen y cols., 2024)

Se hace uso de un modelo de lenguaje de Nvidia Mistral-NeMo-Minitron-8B-Base, de 8.000 millones de parámetros. Para que el modelo de lenguaje pueda entender el lenguaje de matrices 2D de ARC-AGI, el equipo desarrolló un *tokenizador* personalizado que reducía

el vocabulario a 64 *tokens*. Este sistema reduce significativamente el tamaño de la capa de *embedding* para abaratar el coste de inferencia. También ayuda a evitar alucinaciones, pero es necesario dominar el arte de usar estructuras y marcadores concretos. Este sistema es muy sensible al uso de delimitación de regiones para que el modelo sepa cuándo empieza una tarea y cuándo empieza otra.

Para que el modelo pueda ser inferenciado en GPUs de 16 GB, el equipo realiza una cuantización del modelo de 4 bits mediante el paquete *unsloth-4bit*, el más bajo posible. También hace uso de LoRA o *Low-Rank Adaptation*, una técnica de ajuste fino eficiente para hacer viable esta arquitectura. Sin LoRA, ajustar el modelo de 8 millones de parámetros sería imposible. Esta limitación condiciona diversas decisiones del flujo de diseño. LoRA puede conseguir la reducción de hasta 32 veces el número de parámetros y reducir solamente un pequeño subconjunto de parámetros entrenables del modelo (Franzen y cols., 2024).

Para la inferencia se usó el algoritmo DFS con un umbral probabilístico y poda. Otros algoritmos, como GBFS o *Beam Search*, provocaban una baja diversidad de soluciones o resultaban demasiado costosos en términos de memoria. Tras la búsqueda, se realiza una selección de candidatos.

El proceso de preentrenamiento se realiza usando una GPU Nvidia H100 con 1.979 TFLOPs en precisión FP16 y 80 GB de VRAM HBM. Esta capacidad de computación solo está disponible en servicios de computación en la nube y resulta muy costosa incluso en despliegues de *clústeres* optimizados. La adaptación en tiempo de prueba y la inferencia se realizan con dos GPU Nvidia T4 de 16 GB de VRAM durante casi 12 horas, el límite permitido por el concurso.

2.11. Enfoques de modelos sin preentrenamiento

Durante años, la creencia de que es necesario realizar un preentrenamiento exhaustivo de los modelos para poder desarrollar la capacidad de razonamiento abstracto ha sido dominante. En el trabajo de Liao y Gu (2025) se desarrollan diferentes enfoques, entre los que se encuentra CompressARC, un decodificador VAE o *Variational Autoencoder* encargado de reconstruir una serie de datos a partir de una representación comprimida. El modelo realiza su aprendizaje exclusivamente en el tiempo de inferencia, no cuenta con bases de datos externas, su único conjunto de datos son las matrices del problema exacto

que se intenta resolver en un momento determinado.

“The ability to compress information is equivalent to intelligence.”

(Ouellette, 2024, p. 15)

El modelo es realmente compacto, con tan solo 76 mil parámetros. Este sistema se rige por el principio de longitud de descripción mínima y la comprensión de información. La optimización se realiza mediante redes neuronales en lugar de algoritmos de búsqueda ciegos. Las redes neuronales pueden restringir el espacio de búsqueda (Liao y Gu, 2025). La red neuronal incluye operaciones diseñadas a mano (sesgos inductivos) que son muy útiles para transformaciones geométricas como medias, expansiones de tensores, funciones de activación SiLU, máximos acumulativos y desplazamientos directos de píxeles.

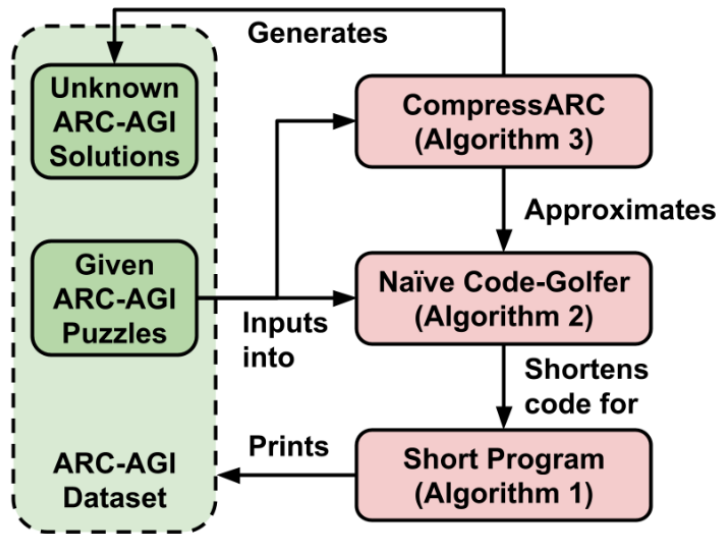


Figura 2.15: Algoritmo de compresión de CompressARC.

Fuente: Adaptado de “ARC-AGI WITHOUT PRETRAINING” (Franzen y cols., 2024)

El modelo logra generalizar y resolver el 20 % de las tareas de ARC-AGI-1. Lograr este resultado con apenas 76 mil parámetros y una GPU RTX 4070 con una precisión FP16 de 116,6 TFLOPs es realmente sorprendente.

2.12. Modelos basados en Transformers compactos

Li, Xu, Sanner, y Khalil (2024) estudian la viabilidad de aplicar un *Vision Transformer* compacto al *benchmark* ARC mediante entrenamiento supervisado específico por tarea con

grandes volúmenes de ejemplos sintéticos. El hallazgo central es que un *ViT* convencional sigue rindiendo de forma muy pobre incluso con abundantes datos, lo que apunta a una limitación de representación más que a un simple problema de escala. Para corregir esta debilidad, el trabajo propone *ViTARC*, una arquitectura que sustituye la representación por parches por una codificación a nivel de píxel e incorpora *tokens* de frontera, codificación posicional bidimensional y señales posicionales basadas en objetos. Estas modificaciones elevan de forma notable el rendimiento y permiten alcanzar en torno al 75 % de acierto sobre las 400 tareas públicas consideradas. El artículo resulta relevante porque refuerza una idea clave: en transformadores pequeños, la mejora no depende solo de aumentar parámetros o datos, sino de introducir sesgos inductivos que preserven explícitamente la estructura espacial y las relaciones entre objetos.

2.13. Modelos compactos basados en bucles de refinamiento iterativo

Una de las líneas más prometedoras recientes en ARC-AGI consiste en sustituir la profundidad estática de una red por cómputo iterativo aplicado sobre estados latentes. En lugar de forzar al modelo a producir la respuesta final en una única pasada, estos enfoques refinan progresivamente una hipótesis interna y adaptan el número de pasos de inferencia a la dificultad efectiva de cada tarea. Esta idea resulta especialmente valiosa en ARC-AGI, donde la dificultad no reside solo en reconocer patrones visuales, sino en encadenar transformaciones simbólicas a partir de muy pocos ejemplos.

2.13.1. Modelos de razonamiento jerárquico

El *Hierarchical Reasoning Model* o HRM de Wang y cols. (2025) representa uno de los primeros resultados convincentes dentro de esta familia. Su punto de partida es una crítica directa a los modelos de profundidad fija. Aunque un transformador convencional pueda aumentar su contexto o su número de parámetros, sigue realizando esencialmente un número fijo de operaciones para entradas fáciles y difíciles. HRM intenta corregir esta limitación introduciendo dos módulos recurrentes acoplados que operan a escalas temporales distintas. Un módulo de alto nivel mantiene una representación lenta y abstracta del problema, mientras que un módulo de bajo nivel realiza actualizaciones rápidas y detalladas sobre una solución parcial. La separación entre ambos niveles permite alternar

entre planificación y refinamiento, reduciendo el estancamiento que suele aparecer en redes recurrentes profundas cuando el estado oculto converge demasiado pronto.

A nivel de entrenamiento, la propuesta incorpora un esquema de gradiente de un solo paso inspirado en los *Deep Equilibrium Models*. En la práctica, la mayor parte de las iteraciones recurrentes se ejecutan sin almacenar todo el grafo completo, y únicamente el último tramo se utiliza para retro-propagación. Esta decisión reduce de forma drástica el coste de memoria y hace viable incrementar la profundidad efectiva del razonamiento sin multiplicar el consumo de recursos. Sobre este núcleo se añade un mecanismo de *Adaptive Computational Time*, de manera que el modelo aprende cuándo debe detener su refinamiento interno y cuándo conviene seguir computando. Con aproximadamente 27 millones de parámetros y sin depender de preentrenamiento masivo ni de *Chain-of-Thought* textual, HRM alcanza en torno al 40 % en ARC-AGI-1 y muestra que es posible competir en razonamiento abstracto aumentando la profundidad de cómputo antes que la escala bruta del modelo (Wang y cols., 2025).

2.13.2. Modelos de razonamiento recursivo con redes neuronales compactas

Jolicoeur-Martineau (2025) llevan esta intuición un paso más lejos con el *Tiny Recursive Model* o TRM. La aportación central de este trabajo consiste en mostrar que una parte importante de la mejora no proviene necesariamente de mantener dos módulos distintos, sino del propio mecanismo de refinamiento recursivo. TRM simplifica el planteamiento jerárquico y reutiliza una única red extremadamente pequeña para actualizar de forma iterativa sus estados latentes y su respuesta candidata. El modelo conserva dos espacios internos diferenciados, uno más próximo a la respuesta decodificable y otro dedicado al razonamiento latente, pero elimina buena parte de la complejidad estructural de HRM. De este modo, desplaza el foco desde la sofisticación del bloque arquitectónico hacia la reutilización intensiva de un mismo bloque diminuto.

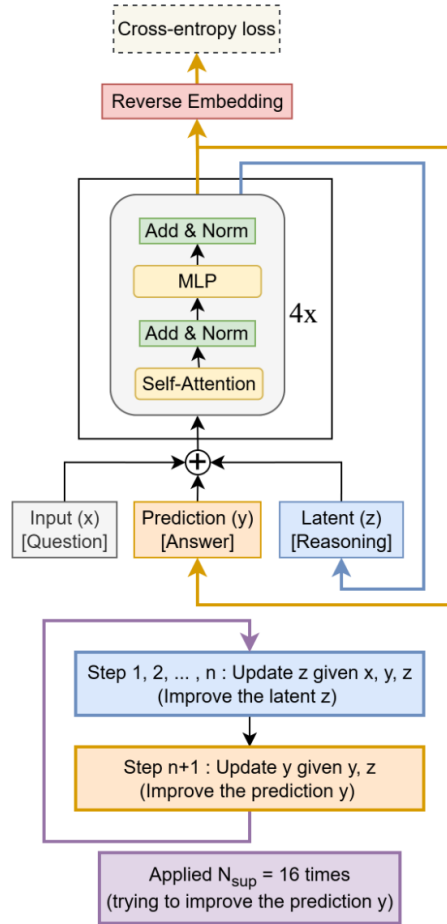


Figura 2.16: Arquitectura TRM con proceso recursivo y eficiente en cuanto a parámetros.

Fuente: Adaptado de “Less is More: Recursive Reasoning with Tiny Networks” (Jolicoeur-Martineau, 2025)

Esta simplificación es relevante por dos motivos. En primer lugar, mantiene las ventajas del refinamiento iterativo. El sistema no necesita resolver la tarea de una vez, sino que puede corregir sucesivamente su hipótesis interna a lo largo de varios ciclos. En segundo lugar, mejora de forma notable la eficiencia paramétrica. TRM funciona con una red de solo dos capas y alrededor de 7 millones de parámetros, muy por debajo de la escala habitual de los modelos de frontera. Aun así, obtiene un 45 % en ARC-AGI-1 y un 8 % en ARC-AGI-2, lo que sitúa a un modelo extremadamente pequeño en una zona competitiva frente a sistemas mucho mayores (Jolicoeur-Martineau, 2025). Desde la perspectiva de este trabajo, TRM refuerza una conclusión especialmente importante. En problemas de razonamiento abstracto como ARC-AGI, puede resultar más rentable invertir cómputo adaptativo sobre estados latentes bien diseñados que aumentar indiscriminadamente el

número de parámetros.

La autora del trabajo ha realizado el pre-entrenamiento del modelo en un clúster de 8 GPUs Nvidia A100 de 80 GB VRAM, con una precisión total de 7.916 PFLOPs y 320 GB de VRAM. La inferencia se ha realizado en una sola GPU Nvidia RTX 4090 con 24 GB de VRAM.

2.14. Estado actual con perspectiva de eficiencia computacional

Desde una perspectiva estrictamente computacional, el estado actual de ARC-AGI muestra un patrón muy claro. Cada nueva generación del *benchmark* ha introducido un precipicio de rendimiento que reduce de forma drástica la precisión de los sistemas previos. En términos agregados, el paso de ARC-AGI-1 a ARC-AGI-2 y posteriormente a ARC-AGI-3 implica caídas del orden de 2,5 a 3 veces, lo que sugiere que buena parte de los avances recientes siguen dependiendo de regularidades explotables en una versión concreta del problema más que de una generalización robusta. Este comportamiento refuerza la idea de que el progreso en ARC no puede evaluarse solo por la puntuación absoluta, sino por la estabilidad del rendimiento cuando aumenta la profundidad conceptual de las tareas.

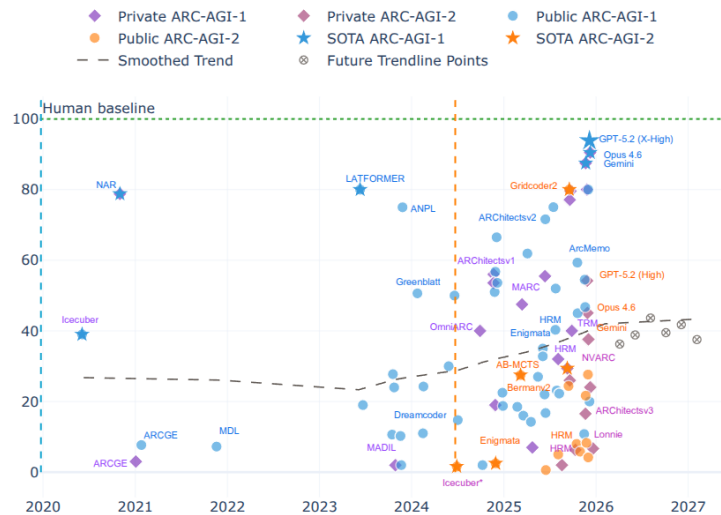


Figura 2.17: Evolución de la puntuación de ARC-AGI desde 2020.

Fuente: Adaptado de “The ARC of Progress towards AGI: A Living Survey of Abstraction and Reasoning” (Vahdati, Aioanei, Suresh, y Lehmann, 2026)

Este diagnóstico abre además una tensión económica y teórica. Los modelos de frontera

han logrado mejoras relevantes, pero a menudo a costa de presupuestos de entrenamiento e inferencia muy elevados, mientras que la cognición humana resuelve estas tareas con un consumo energético ínfimo en comparación. La diferencia no es solo de escala, sino de mecanismo. En el ser humano, la resolución parece apoyarse en una forma de comprensión conceptual que abstrae reglas y encuentra trayectorias de solución eficientes. En cambio, muchos sistemas actuales aproximan ese comportamiento mediante búsqueda, paralelismo y cómputo masivo. Desde este punto de vista, la eficiencia no debe entenderse únicamente como reducción de coste monetario, sino como capacidad de transformar energía y cómputo en abstracciones reutilizables.

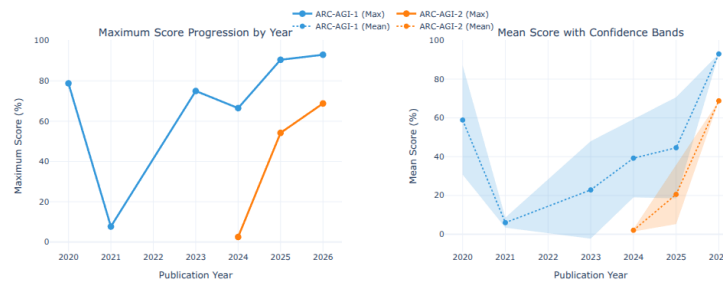


Figura 2.18: Comparativa de la progresión entre ARC-AGI-1 y ARC-AGI-2.

Fuente: Adaptado de “The ARC of Progress towards AGI: A Living Survey of Abstraction and Reasoning” (Vahdati y cols., 2026)

En ese contexto, el cambio más relevante observado en 2025 no fue simplemente el escalado de parámetros, sino la consolidación de los bucles de refinamiento en tiempo de inferencia. Los enfoques más competitivos ya no se limitan a producir una respuesta directa, sino que encadenan ciclos de generación, verificación y corrección. Modelos como TRM y propuestas como *CompressARC* muestran que este principio puede ofrecer resultados competitivos incluso con redes muy pequeñas o sin pre-entrenamiento masivo, siempre que el sistema disponga de mecanismos para revisar de manera iterativa su hipótesis interna. La eficiencia computacional no reside en el tamaño bruto del modelo, sino en la calidad del proceso de refinamiento que es capaz de ejecutar durante la inferencia.



Figura 2.19: Comparativa del coste por tarea \$ entre diferentes soluciones. El tamaño de los círculos representa el tamaño en parámetros de cada modelo.

Fuente: Adaptado de “The ARC of Progress towards AGI: A Living Survey of Abstraction and Reasoning” (Vahdati y cols., 2026)

El coste por tarea es un indicador especialmente relevante para medir la eficiencia de los diferentes modelos. También existen diferentes sistemas como TRM que suponen un coste especialmente elevado teniendo en cuenta el reducido tamaño del modelo. El tamaño del modelo no siempre es un indicador de coste equivalente. Este sistema tiene pocos parámetros, pero su proceso de entrenamiento requiere una desmesurada capacidad de cómputo, como se ha podido ver en la sección anterior. Esta capacidad de computación no está al alcance de la mayoría de investigadores y es un factor limitante para la reproducibilidad de los resultados. La figura 2.20 muestra una comparativa de coste por tarea entre diferentes modelos, donde el tamaño de los círculos representa la capacidad computacional en TFLOPs FP16 usada en cada modelo. Esta representación es más fiel a la realidad que el tamaño del modelo, ya que un modelo pequeño puede requerir una gran capacidad de cómputo para su entrenamiento y ajuste fino.

3. Metodología

3.1. Enfoque Científico y Método de Investigación

El presente trabajo se enmarca dentro del método científico empírico-analítico. Dado que el objetivo principal es evaluar la eficiencia y la capacidad de un modelo de lenguaje compacto en la resolución del *benchmark* ARC-AGI-1, el enfoque adoptado es de carácter cuantitativo. La investigación sigue un proceso hipotético-deductivo articulado en etapas: planteamiento del problema ante los límites de la hipótesis de escala en modelos LLM, formulación de la hipótesis basada en modelos compactos, diseño experimental, ejecución y análisis métrico de los resultados.

Desde la perspectiva investigadora, se aplica un método empírico experimental. Este método es el más idóneo ya que permite establecer relaciones causales bajo condiciones de laboratorio estrictamente controladas. Se manipularán de forma deliberada variables arquitectónicas independientes para medir su impacto sobre variables dependientes clave del rendimiento: la tasa de precisión en la evaluación de ARC-AGI y el coste computacional asociado.

3.2. Metodología de Desarrollo y Gestión del Proyecto

Debido a la naturaleza incremental de la síntesis de programas y la necesidad de adaptarse a los resultados de las pruebas continuas, el desarrollo del componente software se rige por metodologías ágiles, combinando prácticas de Scrum y Kanban. El trabajo se ha estructurado en torno a una épica principal desglosada en cuatro grandes hitos o bloques de tareas o *milestones*:

- **Milestone 1:** Investigación y análisis.
- **Milestone 2:** Escritura y documentación del trabajo.
- **Milestone 3:** Diseño y desarrollo del modelo.
- **Milestone 4:** Validación y experimentación del modelo.

El ciclo de desarrollo se ejecuta en iteraciones cortas o *Sprints* de 14 días de duración. Esta cadencia temporal permite realizar inspecciones regulares del código, facilitando la

adaptación temprana de hiperparámetros o la corrección de derivas en el entrenamiento. El control de versiones, el seguimiento de tareas y la representación visual de los tableros Kanban se centralizan mediante un repositorio de código alojado en GitHub.

3.3. Ciclo de Vida del Modelo basado en MLOps

En consonancia con las prácticas de desarrollo ágil para proyectos de inteligencia artificial, se ha adoptado una versión adaptada del ciclo de vida MLOps orientada a la experimentación en entornos restringidos. Este ciclo se compone de las siguientes fases iterativas:

- **Preparación de datos:** Tratamiento del corpus ARC-AGI, definiendo las rutinas de visualización y codificación espacial necesarias.
- **Desarrollo del modelo y selección algorítmica:** Diseño de las arquitecturas compactas y las rutinas de inferencia iterativa.
- **Entrenamiento y ajuste:** Ejecución de pruebas de *Test-Time Training* y refinamiento de hiperparámetros bajo las restricciones de *hardware*.
- **Evaluación:** Medición cuantitativa del rendimiento del modelo utilizando conjuntos de evaluación de ARC-AGI-1.

Esta estructura garantiza que tanto el marco teórico como el desarrollo de la ingeniería se ejecuten de manera rigurosa, permitiendo que las conclusiones extraídas al final del proyecto sean sólidas y reproducibles.

4. Infraestructura y herramientas

4.1. Infraestructura de computación

Para el desarrollo y experimentación de este proyecto, se ha utilizado una estación de trabajo local equipada con hardware de alto rendimiento. Las especificaciones técnicas del equipo base son las siguientes:

- Sistema operativo y entorno: El sistema opera bajo la distribución Linux Ubuntu 24.04.4 LTS con el *kernel* 6.14.0-34-generic. Para la aceleración de cargas de trabajo de IA en la GPU, se emplea la plataforma abierta de AMD, ROCm en su versión 7.2.3.
- Procesador (CPU): Intel Core i9-12900K de 12^a generación.
- Memoria principal (RAM): El sistema dispone de aproximadamente 96 GB de memoria RAM disponible.

El componente central para el entrenamiento de este proyecto es la GPU AMD Radeon RX 9070 XT. Esta GPU de arquitectura avanzada está equipada con 64 unidades de cómputo, 4096 *stream processors* y cuenta con 128 aceleradores de IA dedicados específicamente a optimizar las cargas de trabajo de aprendizaje profundo. Su consumo típico (Typical Board Power) es de 304 W.

Para las tareas de Inteligencia Artificial, las dos métricas más críticas son la memoria de vídeo (VRAM) y el rendimiento en diferentes precisiones matemáticas. La tarjeta cuenta con 16 GB de memoria GDDR6 operando sobre una interfaz de 256 bits. Esta memoria proporciona un ancho de banda de hasta 640 GB/s.

La GPU ofrece un rendimiento de hasta 48,7 TFLOPs en precisión de punto flotante de 32 bits (FP32) para operaciones vectoriales y hasta 195,0 TFLOPs en precisión de punto flotante de 16 bits (FP16) para operaciones matriciales. En la siguiente tabla 4.1 se muestra el rendimiento de la GPU en diferentes niveles de precisión y formatos, incluyendo el impacto de la dispersión estructurada en el rendimiento.

Nivel de Precisión	Formato	Rendimiento Base	Rendimiento con Dispersión Estructurada
Precisión Simple (Vector)	FP32	48.7 TFLOPs	-
Media Precisión (Matriz)	FP16	195.0 TFLOPs	389.0 TFLOPs
Precisión de 8 bits (Matriz)	FP8 / INT8	389.0 TFLOPs / TOPs	779.0 TFLOPs / TOPs
Precisión de 4 bits (Matriz)	INT4	779.0 TOPs	1557.0 TOPs

Tabla 4.1: Rendimiento por nivel de precisión y formato.

4.2. Entorno de desarrollo

Se ha utilizado un amplio ecosistema de paquetes y librerías para el desarrollo y la experimentación. En la tabla 4.2 se muestra una lista de las principales librerías utilizadas, junto con sus versiones y abreviaciones para su referencia en el resto del documento.

Nombre	Versión	Abreviación
PyTorch	2.12.0+rocm7.2	torch
Torchaudio	2.11.0+rocm7.2	torchaudio
Torchvision	0.27.0+rocm7.2	torchvision
Huggingface_hub	0.34.4	huggingface_hub
Packaging	24.1	packaging
Ninja	1.13.0	ninja
Wheel	0.45.1	wheel
Setuptools	80.9.0	setuptools
Setuptools-scm	9.2.0	setuptools-scm
Pydantic-core	2.33.2	pydantic-core
Triton	3.3.0	triton
Matplotlib	3.10.0	matplotlib
Numpy	2.2.2	numpy
Jupyter-events	0.12.0	jupyter-events
Jupyter-lsp	2.2.5	jupyter-lsp
Jupyter_client	8.6.3	jupyter_client
Jupyter_core	5.7.2	jupyter_core
Jupyter_server	2.16.0	jupyter_server
Jupyter_server_terminals	0.5.3	jupyter_server_terminals
Jupyterlab	4.4.2	jupyterlab
Jupyterlab_pygments	0.3.0	jupyterlab_pygments
Jupyterlab_server	2.27.3	jupyterlab_server
Keras	3.9.2	keras
Notebook	7.4.2	notebook
Tensorboard	2.19.0	tensorboard
Tensorboard-data-server	0.7.2	tensorboard-data-server
Tensorflow	2.19.1	tensorflow_rocm

Tabla 4.2: Librerías de entorno en Python, versión y abreviación.

4.3. Herramientas externas

En el desarrollo del proyecto se han usado herramientas externas de control de versiones y plataformas de colaboración y desarrollo, como GitHub Copilot. Además, para potenciar la productividad se han utilizado diversas herramientas, como los servidores MCP.

Cabe mencionar el uso de la herramienta Codebase Memory como *indexer* de código para realizar búsquedas e implementar soluciones. La herramienta ayuda a agentes de inteligencia artificial a organizar y localizar el código de manera estructural en lugar de recorrer archivos constantemente.

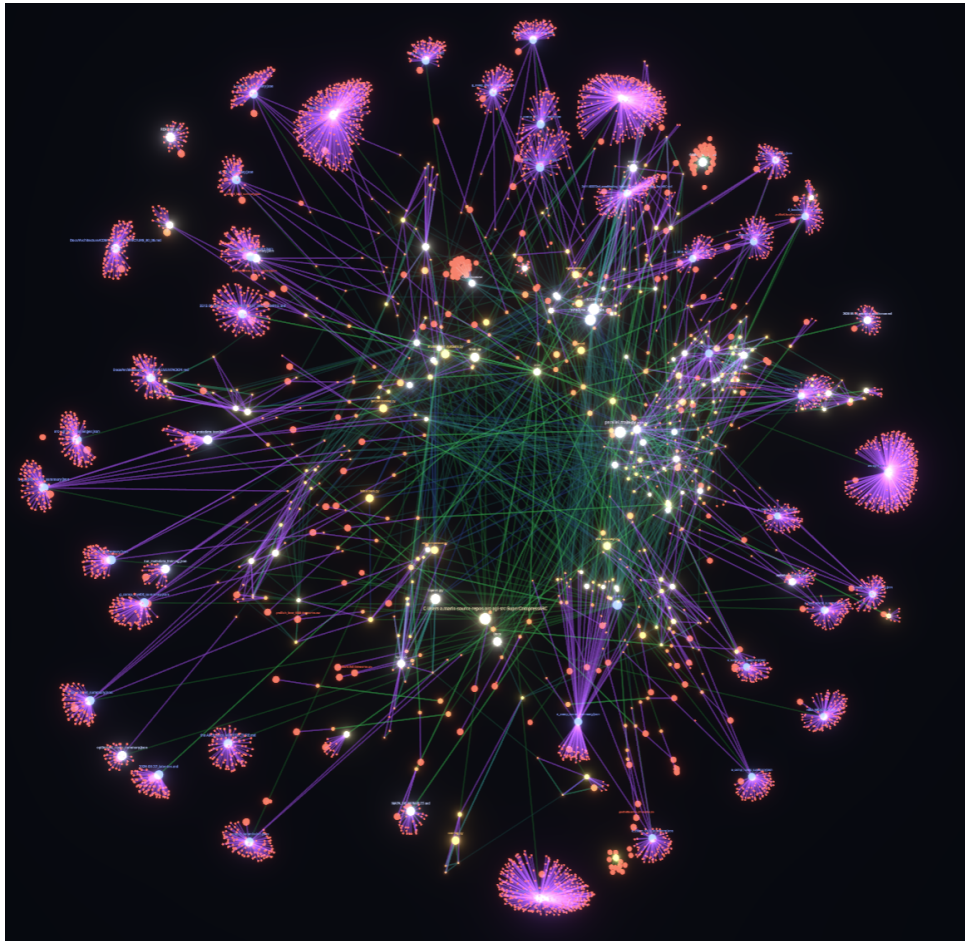


Figura 4.1: Representación semántica y relacional del uso de la herramienta Codebase Memory como *indexer* de código.

Fuente: Elaboración propia.

5. Desarrollo

ARC-AGI es algo más que un *benchmark*; es un riguroso manifiesto sobre la naturaleza de la propia inteligencia. Su diseño permite a la comunidad realizar mediciones objetivas y evaluar progresivamente el avance de los sistemas. Esta mecánica de medición es relativamente sencilla para humanos pero extremadamente compleja para sistemas de inteligencia artificial. El diseño y desarrollo de modelos de lenguaje para resolver las diferentes tareas de ARC-AGI requiere de un enfoque meticuloso y una comprensión profunda de los principios subyacentes que rigen la inteligencia general. Por ello, se convierte en un gran desafío para el autor de este proyecto y para toda la comunidad de investigación en inteligencia artificial.

La idea central es que la inteligencia no se demuestra mediante la posesión de una habilidad específica, sino mediante la eficiencia en la adquisición de nuevas habilidades al enfrentarse a un problema nuevo. Esto es casi lo opuesto de lo que premian la mayoría de los indicadores de rendimiento clásicos. Estos indicadores miden el desempeño en tareas que se pueden dominar mediante grandes cantidades de datos, en lugar de medir la capacidad de adquirir nuevas habilidades.

Para que exista una comparación justa entre humanos y modelos de inteligencia artificial, la tarea debe basarse en inteligencia fluida, con capacidad de razonar, resolver problemas nuevos y adaptarse a nuevas situaciones y no en la inteligencia cristalizada que se basa en conocimiento y las habilidades acumuladas. ARC-AGI sortea este problema basándose únicamente en un pequeño conjunto de conocimientos previos básicos, bloques cognitivos fundamentales que están presentes al nacer o que se adquieren muy temprano en la vida, con una instrucción explícita mínima.

La síntesis de programas es uno de los enfoques clásicos para ARC-AGI y aborda directamente el fondo del problema, que es deducir la regla a partir de los ejemplos. La idea, también llamada programación inductiva, es escribir automáticamente un programa que satisfaga una especificación. En este caso, la especificación está formada por los pares de matrices. Se busca un programa que convierta la entrada del entrenamiento en la salida correspondiente, nada más. Este enfoque es inductivo, parte de una regla general y busca deducir la regla a partir de los ejemplos. El otro enfoque es transductivo, que es evitar generar un programa general e intentar predecir la salida directamente a partir de los ejemplos de entrada. En las soluciones basadas en *Domain Specific Languages* (DSL),

como se ha visto en la sección 2.7.1, el espacio de posibles programas que se pueden generar es inmenso, computacionalmente costoso para la CPU y poco práctico.

Resolver el *benchmark* ARC-AGI requiere de un enfoque meticuloso y una comprensión profunda de los principios subyacentes que rigen la inteligencia general. Es necesario establecer una serie de componentes básicos previos:

1. **Preparación de datos:** Tratamiento del corpus ARC-AGI, definiendo las rutinas de visualización y codificación espacial necesarias. Existen ya herramientas externas que permiten realizar esta tarea de forma más eficiente y rápida.
2. **DSL básico:** Un lenguaje de programación específico del dominio (DSL) básico para empezar a realizar transformaciones.
3. **Pruebas y elección de algoritmos de búsqueda:** Evaluación de diferentes estrategias de búsqueda.
4. **Herramientas de visualización tras pruebas:** Herramientas que permitan de forma rápida analizar los resultados de las pruebas y la evolución de los modelos.
5. **Bucles de retroalimentación:** Preparar un sistema de retroalimentación que permita mejorar el modelo de forma iterativa.

Como se ha visto en la sección 2.6, el espíritu de ARC-AGI es encontrar soluciones que sean lo más eficientes posible, no solo en términos de precisión, sino también en términos de coste computacional. La inteligencia no se debe comprar con datos ni con cómputo masivo. Este trabajo se construye a partir de este espíritu de la norma. Se han estudiado numerosos modelos y soluciones para buscar un enfoque acorde con este principio.

Dejando al margen los grandes modelos de lenguaje y los modelos de frontera, se han estudiado diferentes enfoques de modelos compactos y eficientes. La figura 5.1 muestra una comparativa del coste por tarea entre diferentes modelos compactos, donde el tamaño de los círculos representa la capacidad computacional en TFLOPs FP16 usada en cada modelo. Esta representación es más fiel a la realidad que el propio tamaño del modelo, ya que un modelo pequeño puede requerir una gran capacidad de cómputo para su entrenamiento y ajuste fino. Los datos obtenidos son datos oficiales de la página web oficial de ARC-AGI.

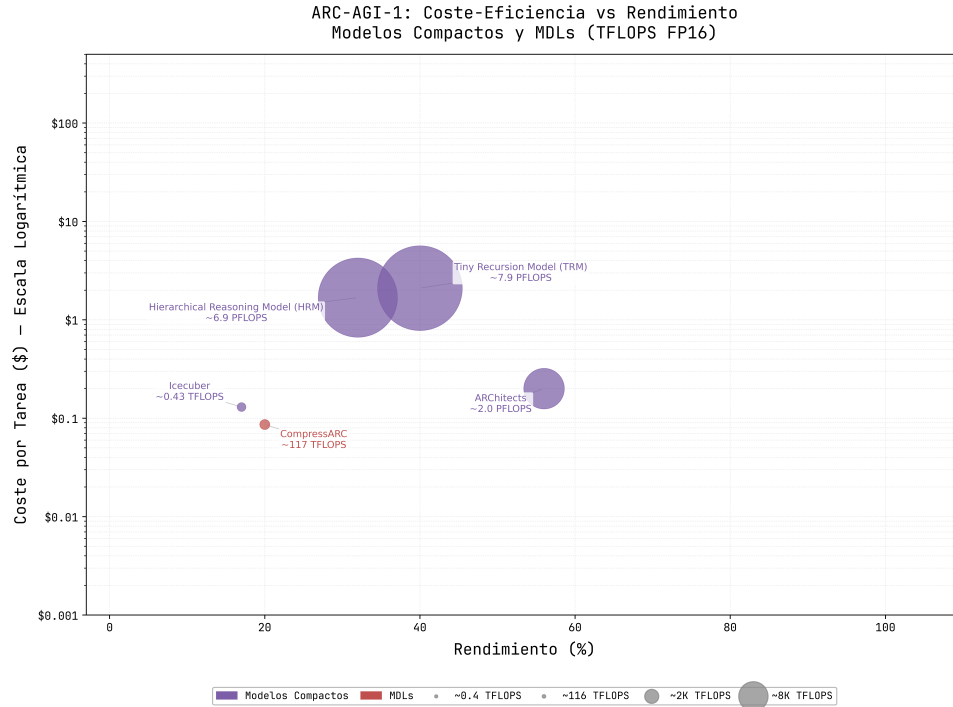


Figura 5.1: Comparativa del coste por tarea entre modelos compactos. El tamaño de los círculos representa la capacidad computacional en TFLOPS FP16 usada en cada modelo.

Fuente: Elaboración propia. Datos obtenidos de la página web oficial de ARC-AGI. Imagen completa:

https://github.com/UNIR-TUC/arc-agi/blob/main/src/arc_agi_data_scraper/images/compact_tflops_cost_light.png

Algunas soluciones, como TRM, requieren de una capacidad de cómputo muy elevada, que no está al alcance de cualquier investigador. Por otra parte, este enfoque también se encuentra en el filo del espíritu de ARC-AGI, ya que es necesario realizar preentrenamientos sin ningún límite de capacidad de computación para posteriormente realizar ajustes finos e inferencia en las GPU exigidas por los órganos de ARC Prize. La diferencia entre la capacidad de cómputo usada para el modelo TRM y la del modelo *CompressARC* es de 7.916 PFLOPs frente a 116,6 TFLOPs, lo que supone una diferencia de más de 67 veces. En cuestión de eficiencia computacional es realmente destacable el modelo de *CompressARC*.

La siguiente figura 5.2 muestra otra comparativa de diferentes modelos compactos desde la perspectiva de coste por tarea.

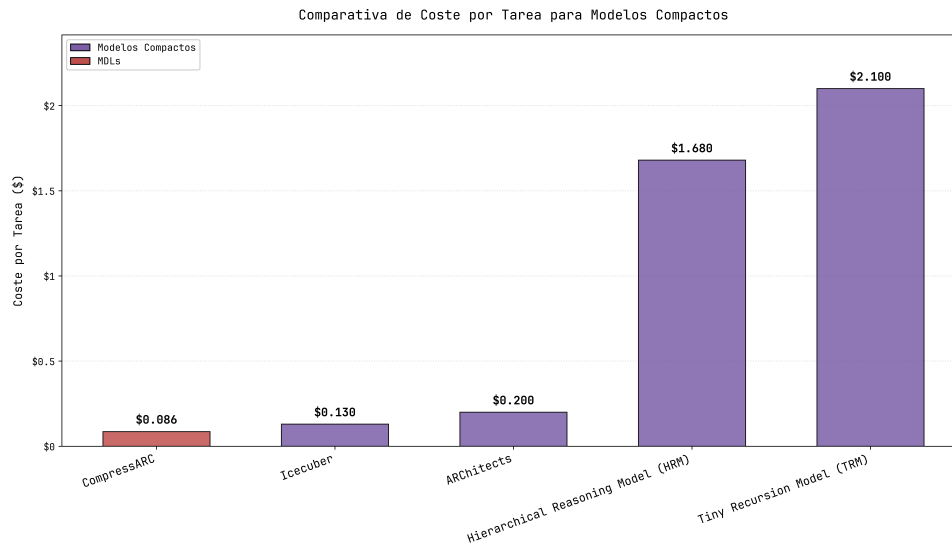


Figura 5.2: Comparativa del coste por tarea entre modelos compactos.

Fuente: Elaboración propia. Datos obtenidos de la página web oficial de ARC-AGI. Imagen completa:

https://github.com/UNIR-TUC/arc-agi/blob/main/src/arc_agi_data_scraper/images/cost_models_cost_light.png

Dado el análisis y las evidencias presentadas, se ha decidido optar por el enfoque presentado por Liao y Gu, CompressARC. En las siguientes secciones se analizará con detalle este enfoque y se presentará una variante propia que se ajusta a las limitaciones de hardware y a los objetivos de eficiencia computacional del proyecto.

5.0.1. Análisis exploratorio de datos

Los datos de entrada del *benchmark* ARC-AGI se presentan en forma de pares de matrices, donde cada par consiste en una matriz de entrada y su correspondiente matriz de salida. El análisis exploratorio de datos (EDA) es un paso crucial para comprender la naturaleza de estos datos y para identificar patrones, tendencias y posibles anomalías que puedan influir en el diseño del modelo. Los datos se presentan habitualmente en archivos JSON de texto simple, que contienen representaciones de las matrices en forma de listas anidadas. Cada matriz puede variar en tamaño y contenido, lo que introduce un nivel adicional de complejidad en el análisis.

El conjunto de datos se compone de cinco archivos:

- **train_challenges:** Tareas para el entrenamiento. 400 tareas de entrenamiento, cada una con 3 ejemplos de entrada (o más) y salida. Estas tareas son públicas y se pueden

usar para entrenar modelos de lenguaje.

- **train_solutions:** Soluciones correspondientes a las tareas de entrenamiento. 400 soluciones de entrenamiento, también son públicas.
- **evaluation_challenges:** Tareas para la evaluación. 400 tareas de evaluación, cada una con 3 ejemplos de entrada (o más) y salida. Estas tareas son públicas y se pueden usar para evaluar modelos de lenguaje.
- **evaluation_solutions:** Soluciones correspondientes a las tareas de evaluación. 400 soluciones de evaluación, también son públicas.
- **test_challenges:** Tareas adicionales para la prueba privada final de ARC Prize. Estas tareas suelen ser nuevas y totalmente privadas hasta que se evalúan en una máquina oficial. La idea es que los modelos no puedan sobreajustarse a estas tareas y que la evaluación final sea lo más objetiva posible. No se dispone de sus soluciones, ya que son privadas y solo se pueden evaluar en la máquina oficial de ARC Prize.

```
display(task['train'][0]['input'])
display(task['train'][0]['output'])
[12]
... [[0, 7, 7], [7, 7, 7], [0, 7, 7]]
... [[0, 0, 0, 0, 7, 7, 0, 7, 7],
      [0, 0, 0, 7, 7, 7, 7, 7, 7],
      [0, 0, 0, 0, 7, 7, 0, 7, 7],
      [0, 7, 7, 0, 7, 7, 0, 7, 7],
      [7, 7, 7, 7, 7, 7, 7, 7, 7],
      [0, 7, 7, 0, 7, 7, 0, 7, 7],
      [0, 0, 0, 0, 7, 7, 0, 7, 7],
      [0, 0, 0, 7, 7, 7, 7, 7, 7],
      [0, 0, 0, 0, 7, 7, 0, 7, 7]]
```

Figura 5.3: Ejemplo de tarea desde el editor de código.

Fuente: Elaboración propia.

Los datos se presentan habitualmente en pares de matrices y están compuestos por valores enteros que representan una paleta de 10 colores.



Figura 5.4: Valor asignado a cada color en la paleta de 10 colores.

Fuente: Elaboración propia.

Cada tarea está identificada por un código único de 8 caracteres alfanuméricos. En los conjuntos de datos se pueden encontrar tareas muy diferentes entre sí.

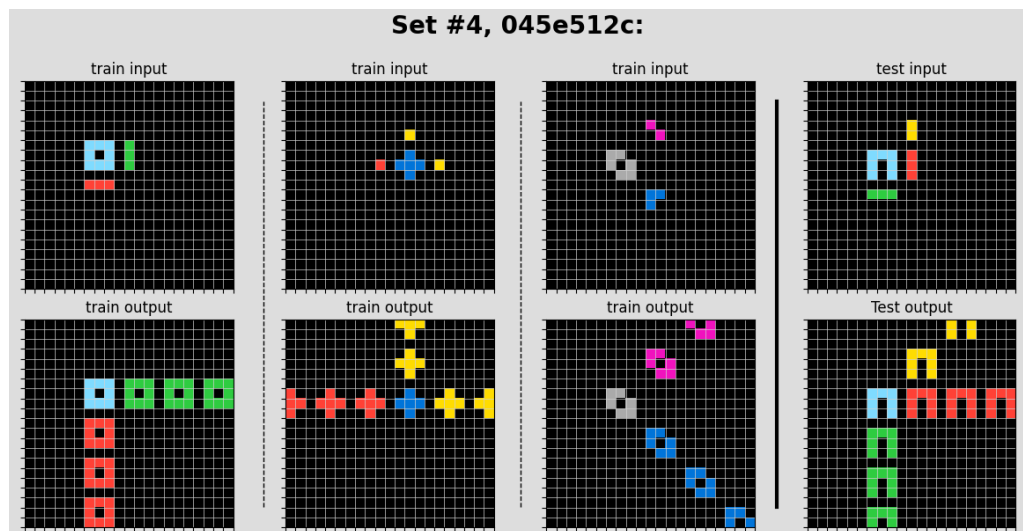


Figura 5.5: Tarea con identificador único 045e512c.

Fuente: Elaboración propia.

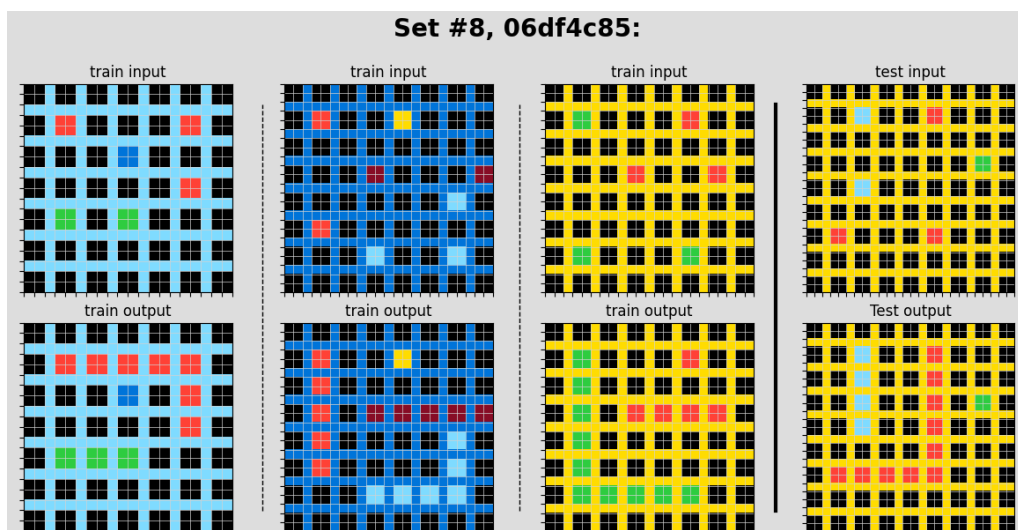


Figura 5.6: Tarea con identificador único 06df4c85.

Fuente: Elaboración propia.

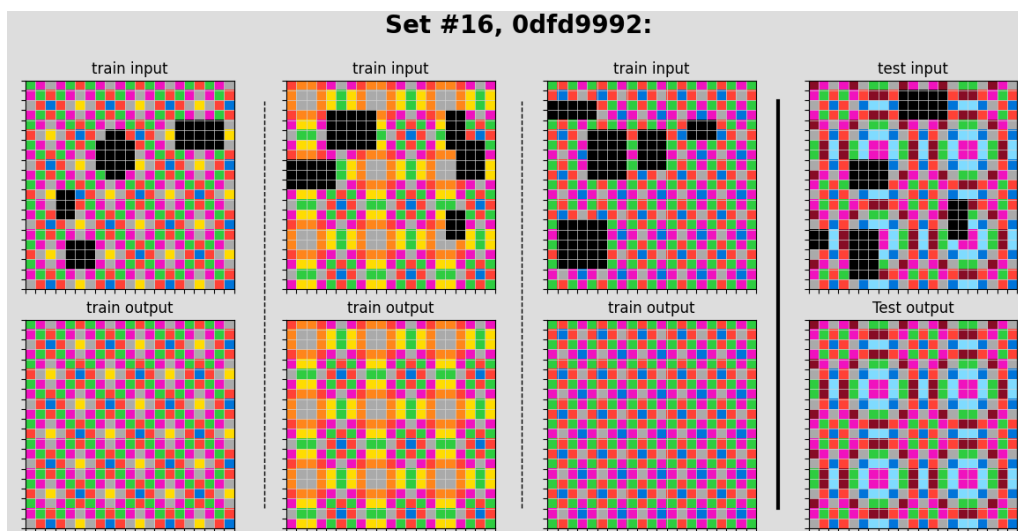


Figura 5.7: Tarea con identificador único 0dfd9992.

Fuente: Elaboración propia.

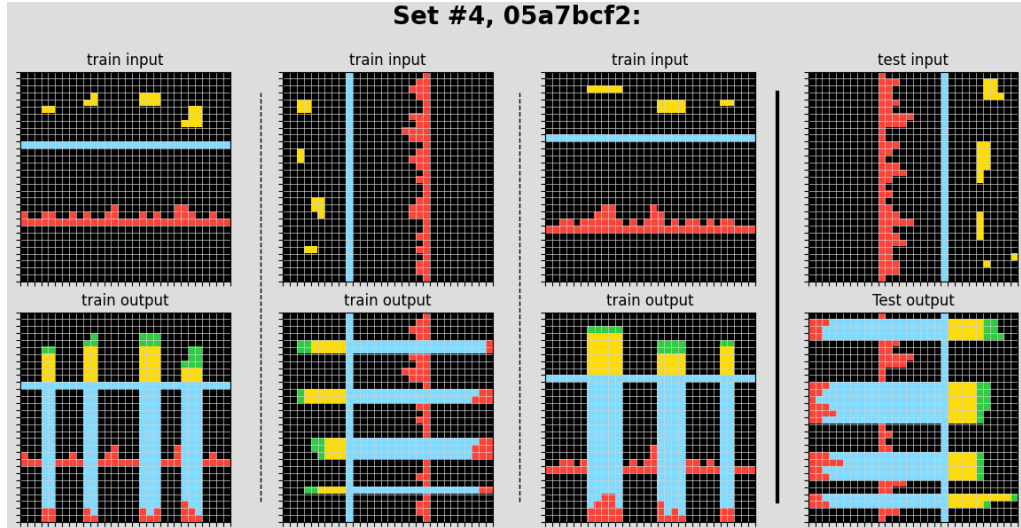


Figura 5.8: Tarea con identificador único 05a7bcf2.

Fuente: Elaboración propia.

5.1. Sistema de referencia

Tal como se ha adelantado en capítulos anteriores, el diseño de la arquitectura propuesta en este trabajo se fundamenta en los principios de *CompressARC* (Liao y Gu, 2025). Esta elección responde a la necesidad de encontrar un equilibrio entre la expresividad del modelo y la eficiencia computacional, evitando el pre-entrenamiento masivo en favor de una optimización centrada en el tiempo de inferencia (*Test-Time Training*).

Un autocodificador es un modelo de aprendizaje profundo que aprende a generar variables latentes que representen características ocultas de los datos, para posteriormente, decodificarlas y reconstruir los datos de entrada. Estos modelos son especialmente útiles para tareas de reducción de dimensionalidad, compresión de datos y generación de nuevas muestras a partir de las características aprendidas. El problema de estos modelos es que aprenden los datos hasta el punto de memorizar en lugar de generalizar, haciendo que sean especialmente buenos para reproducir comportamientos concretos. Los autocodificadores, en concreto, son capaces de generar representaciones discretas, como un vector simple de variables binarias.

Los autocodificadores variacionales o VAE son un tipo de autocodificador con ciertas particularidades. A diferencia de los autocodificadores tradicionales, codifican representaciones continuas y probabilísticas en un espacio latente. En vez de codificar un vector de valores determinísticos, típicamente se genera una media y una desviación estándar por

cada variable latente. Este mecanismo presenta una ventaja: lo convierte en un modelo de aprendizaje profundo generativo capaz de reconstruir con precisión la entrada general exacta y además variaciones que se parecen a los datos originales.

Cabe destacar que los autocodificadores variacionales se componen habitualmente de dos elementos principales: un codificador y un decodificador. Ambos son redes neuronales densas. El codificador transforma la entrada en variables de medias y desviaciones estándar, mientras que el decodificador recoge estas variables para reconstruir la entrada original. Este proceso permite que el modelo capture la incertidumbre inherente en los datos y genere nuevas muestras que sean coherentes con la distribución aprendida. El espacio latente es un conjunto de variables latentes. Estas variables latentes son características ocultas de los datos, representaciones que no son directamente observables en los datos de entrada, pero que capturan información relevante para la generación y reconstrucción de los mismos.

CompressARC es, hasta cierto punto, un sistema basado en autocodificadores variacionales. Este sistema es más profundo y complejo, y contiene un sistema de estructuras con diversas variaciones. Las tareas de ARC-AGI son complejas y variadas. Cada tarea puede contener diversos ejemplos, colores, tamaños y patrones diferentes. Todas estas variaciones en los datos deben ser capturadas y representadas en variables latentes. Este proceso es mucho más complejo a simple vista y requiere de un diseño sofisticado. Para poder realizar este cometido, *CompressARC* hace uso de un sistema denominado Multitensor, y posteriormente se descodifica mediante una compleja arquitectura de redes neuronales densas.

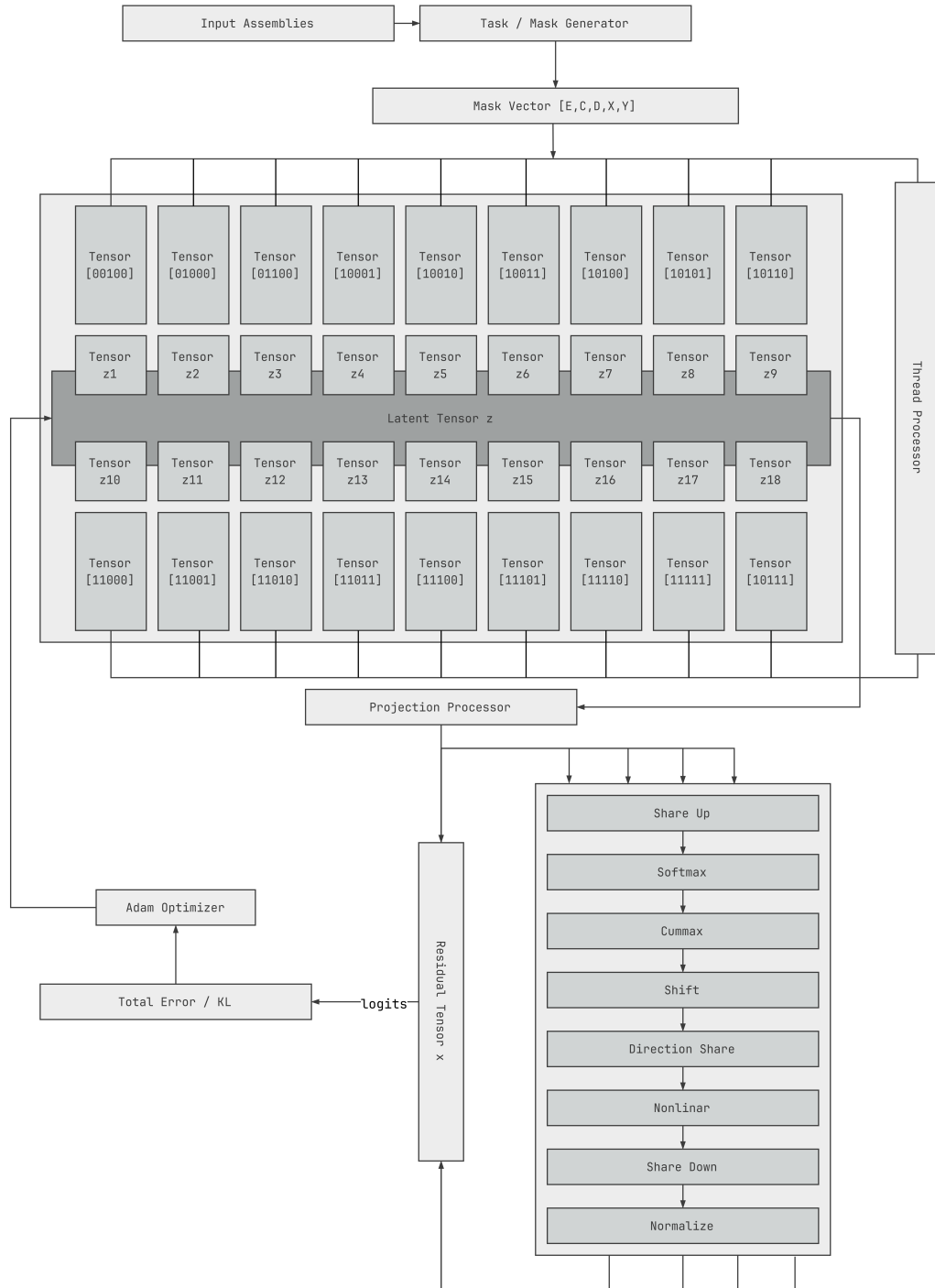


Figura 5.9: Arquitectura *CompressARC*.

Fuente: Elaboración propia.

El objetivo del modelo es el mismo que el de cualquier autocodificador variacional: codificar un espacio latente y posteriormente decodificarlo para reconstruirlo mediante los denominados *logits*. En los siguientes apartados se explicará con todo tipo de detalles el funcionamiento de cada sistema.

El flujo completo de entrenamiento en tiempo de inferencia de manera simple podría ser el siguiente:

1. Se recibe una tarea con sus ejemplos y se extraen las características relevantes: número de ejemplos, número de colores, direcciones, número de filas y número de columnas.
2. Se construye el Multitensor a partir de estas características, aplicando la máscara binaria para identificar los tensores válidos.
3. Cada tensor válido se recorre y se representa en el espacio latente mediante un vector de 4 dimensiones.
4. Se combina toda la información del espacio latente en un vector único denominado z .
5. El vector z se pasa a través de la arquitectura de decodificación para reconstruir los *logits*.
6. Se calcula la pérdida comparando los *logits* reconstruidos con los datos originales.
7. Se actualizan los parámetros del modelo mediante retropropagación.

El sistema está basado en el principio de Mínima Longitud de Descripción (MDL). Si el modelo logra describir los ejemplos de entrenamiento de una tarea con un código más corto que el código trivial de enumerar píxeles, significa que ha descubierto la estructura de la tarea y esa estructura debería extrapolar al ejemplo de pruebas. Entre dos explicaciones que describen correctamente los datos conocidos, se prefiere la que necesita menos información para expresarse. Formalmente, para un conjunto de datos D y un modelo M , esta longitud puede expresarse como:

$$L(D, M) = L(M) + L(D | M), \quad (5.1)$$

donde $L(M)$ representa la información necesaria para describir el modelo y $L(D | M)$ representa la información adicional necesaria para describir los datos una vez conocido

dicho modelo. En el caso de *CompressARC*, esta descomposición se implementa mediante una aproximación diferenciable de la pérdida del VAE, que combina el coste de codificar las variables latentes y el coste de reconstruir las cuadrículas observadas:

$$\mathcal{L}_{\text{MDL}} = D_{\text{KL}}(q(z) \parallel \mathcal{N}(0, I)) + 10 \cdot H(\text{logits}, \text{cuadrículas observadas}), \quad (5.2)$$

El primer término mide la información específica que debe transportar la distribución latente $q(z)$ con respecto a la distribución previa normal estándar, mientras que el segundo penaliza los errores de reconstrucción. El factor 10 es un coeficiente empírico que equilibra ambos términos. Esta expresión es una aproximación diferenciable del principio MDL, restringida a las variables y operaciones de esta arquitectura, por lo que no constituye una medida exacta de la longitud mínima de todos los programas capaces de resolver una tarea. Por tanto, una explicación que comprime los ejemplos conocidos con menor coste tiene más posibilidades de capturar la estructura de la tarea y extrapolarla al ejemplo de prueba. En esta formulación, los píxeles observados incluyen las entradas de prueba, pero no sus salidas, que permanecen ocultas.

En cada tarea se realiza un bucle de entrenamiento completo. Esto lo convierte en un sistema esencialmente transductivo. Se realiza una optimización específica por tarea en vez de un entrenamiento global. No es un modelo inductivo convencional entrenado globalmente para una función reutilizable entre tareas. En términos más precisos, *CompressARC* es un modelo transductivo por tarea, con aprendizaje durante la inferencia y con sesgos inductivos arquitectónicos derivados de características específicas.

5.1.1. Pruebas iniciales

El autor del modelo Liao y Gu (2025) muestra una serie de métricas de las pruebas realizadas en documentos explicativos exhaustivos. El entrenamiento del modelo para todas las tareas se realiza de manera secuencial: se entrena una tarea, se compara el resultado y se continúa con la siguiente. Este procedimiento se realiza por completo en ambos conjuntos de datos de entrenamiento y evaluación. El autor reporta que el tiempo total de ejecución de todas las tareas de entrenamiento es de 130 horas realizando 2000 iteraciones por tarea. El proceso de ejecución de las tareas de evaluación es algo más largo: 138 horas.

De manera experimental también incluye una opción de lanzar diversos procesos en paralelo, lo que puede reducir el tiempo total de ejecución. En las pruebas iniciales se usaron ambas técnicas para comprobar si realmente existía una mejora significativa con

un subconjunto pequeño de tareas. Ambas pruebas se realizaron y tardaron alrededor de 17,3 horas, con lo cual, el uso de ejecuciones en paralelo tal y como está diseñado el proceso de entrenamiento no proporciona una mejora significativa en términos de tiempo de ejecución.

Al usar el entrenamiento en paralelo también se pudo observar un uso muy intensivo de la CPU, por lo tanto, las sospechas iniciales apuntaban a que podría ocurrir alguna limitación de los procesos relacionados entre la CPU y GPU.

Por otro lado, Liao y Gu (2025) expresó que en sus pruebas realizadas, se usó una NVIDIA RTX 4070, una GPU a nivel computacional teórico inferior a la utilizada para este trabajo, la RX 9070 XT. Esto induce a pensar que no está habiendo ningún aprovechamiento computacional debido a algún problema en el proceso de entrenamiento. Esta circunstancia ocurre tanto en entrenamiento secuencial como en el entrenamiento en paralelo y por lo tanto, no parece haber relación con el posible uso de la CPU, más bien es de diseño.

5.1.2. Codificador Multitensor

Las tareas de ARC-AGI contienen habitualmente entre 2 y 7 ejemplos en los datos de entrenamiento. También pueden contener entre 2 y 5 colores, además de cambios espaciales. En ARC-AGI, las tareas que presentan una variación en el tamaño de la matriz pueden verse reflejadas en las salidas de los propios ejemplos. Si tomamos todas estas características en conjunto podemos tener una representación compleja de 5 dimensiones: [número de ejemplos, número de colores, direcciones, número de filas, número de columnas]. El sistema al principio almacena toda esta información de un vector de estas características. Debido a las grandes variaciones de las 800 tareas que representa ARC-AGI, se recurre a la creación de una estructura denominada Multitensor.

El Multitensor es un conjunto de tensores, cada tensor guarda combinaciones dimensionales. No todas las combinaciones posibles pueden contener información relevante. Las combinaciones que no representan información relevante se consideran inválidas. Una máscara binaria de 5 dimensiones da como resultado 32 posibles combinaciones binarias. De estas 32 posibles combinaciones, se consideran sólo 18 como tensores válidos.

En la tabla 5.1, E representa los ejemplos, C los colores, D las direcciones y X e Y las dimensiones espaciales. El último eje corresponde siempre a los canales de cada representación.

Máscara (ECDXY)	Estado	Dimensiones generales o motivo
00000	Inválido	No contiene ningún eje informativo.
00001	Inválido	El eje Y requiere el eje E .
00010	Inválido	El eje X requiere el eje E .
00011	Inválido	Los ejes X e Y requieren el eje E .
00100	Válido	$[D, \text{canal}]$
00101	Inválido	El eje Y requiere el eje E .
00110	Inválido	El eje X requiere el eje E .
00111	Inválido	Los ejes X e Y requieren el eje E .
01000	Válido	$[C, \text{canal}]$
01001	Inválido	El eje Y requiere el eje E .
01010	Inválido	El eje X requiere el eje E .
01011	Inválido	Los ejes X e Y requieren el eje E .
01100	Válido	$[C, D, \text{canal}]$
01101	Inválido	El eje Y requiere el eje E .
01110	Inválido	El eje X requiere el eje E .
01111	Inválido	Los ejes X e Y requieren el eje E .
10000	Inválido	Sólo contiene E ; falta al menos uno de los ejes C , D , X o Y .
10001	Válido	$[E, Y, \text{canal}]$
10010	Válido	$[E, X, \text{canal}]$
10011	Válido	$[E, X, Y, \text{canal}]$
10100	Válido	$[E, D, \text{canal}]$
10101	Válido	$[E, D, Y, \text{canal}]$
10110	Válido	$[E, D, X, \text{canal}]$
10111	Válido	$[E, D, X, Y, \text{canal}]$
11000	Válido	$[E, C, \text{canal}]$
11001	Válido	$[E, C, Y, \text{canal}]$
11010	Válido	$[E, C, X, \text{canal}]$
11011	Válido	$[E, C, X, Y, \text{canal}]$
11100	Válido	$[E, C, D, \text{canal}]$
11101	Válido	$[E, C, D, Y, \text{canal}]$
11110	Válido	$[E, C, D, X, \text{canal}]$
11111	Válido	$[E, C, D, X, Y, \text{canal}]$

Tabla 5.1: Combinaciones dimensionales del sistema Multitensor.

A estos tensores se les añade una dimensión adicional denominada canal o, más correctamente, espacio latente. Cada posible valor de las hojas contendrá un vector de 4 valores que representará este espacio. Para recorrer todas las posibilidades de los diferentes tensores en los que haya datos representados, se hará uso de una máscara binaria de 5 dimensiones. Posteriormente se representará todo el estado latente mediante un tensor denominado z .

Las tareas de ARC-AGI mezclan información de naturalezas muy distintas. Tratar todo en un único tensor obligaría a la red a redescubrir que estos ejes son cualitativamente distintos. En cambio, un Multitensor mantiene 18 tensores distintos y permite que cada uno se procese localmente y se comunique con los demás de forma controlada.

Direcciones

El eje de direcciones del Multitensor tiene 8 dimensiones fijas. Estas direcciones cubren un grupo diédrico con el conjunto de simetrías de un cuadrado, es decir, todas las formas de moverlo. Las transformaciones más habituales son las rotaciones o reflexiones, mediante las cuales que vuelve a ocupar exactamente la misma posición. La operación del grupo es componer simetrías, es decir, aplicar una tras otra. Está formada por 8 elementos, correspondientes a las 4 rotaciones y 4 reflexiones posibles de un cuadrado. Básicamente describe todas las maneras de mover un cuadrado sin cambiar su apariencia geométrica.

5.1.3. Arquitectura de decodificación

El objetivo de la capa de decodificación es decodificar el espacio latente z . Este proceso se realiza mediante un método interno que se encarga de decodificar de manera independiente cada uno de los 18 tensores del Multitensor. Utiliza una tabla de parámetros ajustables con la forma de la vista correspondiente, que indica la dirección en la que deben desviarse los valores latentes respecto al ruido inicial genérico. Durante todo el proceso de entrenamiento se crean valores aleatorios basados en una distribución normal estándar. Mantener una parte de ruido evita que el espacio latente se convierta en una tabla de memoria sin coste.

También se hace uso de divergencia de Kullback-Leibler, una medida que compara distribuciones de probabilidad y cuantifica cuánta información adicional hace falta para describir la distribución real a partir de la aproximación. En el modelo el tensor latente z funciona como un código interno de la tarea ARC. Ese código debe contener la información necesaria para reconstruir los ejemplos y descubrir la regla.

El proceso de decodificación consta de 4 bloques idénticos que se ejecutan de manera secuencial, como una especie de bucle. Estos bloques están formados por una serie de operaciones:

- **Comunicación ascendente:** difunde información desde las hojas compactas hacia representaciones con más ejes.
- **Selección multi-eje:** obtiene distribuciones relativas sobre colores, direcciones y posiciones mediante *softmax*.
- **Máximo acumulado direccional:** propaga a cada posición el mayor valor obser-

vado durante un recorrido espacial.

- **Desplazamiento direccional:** traslada características entre píxeles vecinos en las ocho orientaciones disponibles.
- **Comunicación entre direcciones:** combina las representaciones asociadas a orientaciones distintas.
- **Transformación no lineal:** aplica la función SiLU para componer relaciones que no pueden expresarse linealmente.
- **Comunicación descendente:** resume representaciones detalladas en hojas con menos ejes.
- **Normalización final:** estabiliza la escala del estado antes de comenzar el bloque siguiente.

Para describir matemáticamente estas operaciones, se denota por h_s el estado de la hoja del Multitensor cuya máscara dimensional es s . Las matrices P_s y Q_s representan, respectivamente, las proyecciones lineales aprendidas de entrada y de retorno al flujo residual. Ambas actúan únicamente sobre el último eje, correspondiente a los canales. La función $\mathcal{N}$ representa la normalización propia del modelo. La relación $r \leq s$ indica que todos los ejes activos de r también están presentes en s . Con esta notación, la estructura general de una transformación residual se expresa en la Ecuación (5.3):

$$h_s^+ = h_s + Q_s \Phi_s(P_s h_s), \quad (5.3)$$

donde Φ_s es la operación característica de la capa. En las capas de comunicación intervienen varias hojas y esta expresión se amplía mediante sumas. Los pesos de P_s y Q_s se aprenden para cada tarea, por lo que no existe un valor numérico universal que pueda emplearse en los ejemplos. Para que los cálculos sean reproducibles, los ejemplos siguientes parten de la activación ya proyectada, consideran válidas todas las celdas y aíslan la operación característica. Cuando se muestra una suma residual, la proyección de retorno se considera identidad sobre los valores representados. Esta simplificación es únicamente pedagógica; las ecuaciones anteriores y las de cada apartado conservan el flujo completo de la implementación.

Comunicación ascendente del Multitensor

La primera operación del bloque corresponde a *share up*. Su objetivo es trasladar regularidades expresadas en hojas compactas hacia hojas más específicas. Por ejemplo, una señal que depende solo del color puede resultar relevante para una representación que, además del color, distingue cada ejemplo y cada posición de la cuadrícula. Sin esta comunicación, las hojas del Multitensor evolucionarían como representaciones independientes y la información descubierta en una de ellas no podría utilizarse en las demás.

Para cada hoja receptora s , la capa selecciona todas las hojas r que cumplen $r \leq s$. Primero proyecta sus canales mediante P_r . Después, el operador $\mathcal{B}_{r \rightarrow s}$ inserta los ejes ausentes y expande los valores hasta hacerlos compatibles con la forma de s . Las contribuciones se suman, se normalizan, se proyectan al número de canales original y se añaden al estado receptor, tal como se muestra en la Ecuación (5.4):

$$\begin{aligned} u_s^\uparrow &= \sum_{r \leq s} \mathcal{B}_{r \rightarrow s}(P_r h_r), \\ h_s^+ &= h_s + Q_s \mathcal{N}(u_s^\uparrow). \end{aligned} \tag{5.4}$$

Ejemplo simplificado. Suponiendo una hoja por color h_C y otra por color y posición h_{CX} . Las filas representan dos colores y las columnas, dos posiciones. Si las demás contribuciones son nulas, la hoja por color se replica sobre el eje espacial, como se detalla en la Ecuación (5.5):

$$\begin{aligned} h_C &= \begin{pmatrix} 2 \\ 3 \end{pmatrix}, & h_{CX} &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \\ \mathcal{B}_{C \rightarrow CX}(h_C) &= \begin{pmatrix} 2 & 2 \\ 3 & 3 \end{pmatrix}, & u_{CX}^\uparrow &= h_{CX} + \mathcal{B}_{C \rightarrow CX}(h_C) = \begin{pmatrix} 3 & 2 \\ 3 & 4 \end{pmatrix}. \end{aligned} \tag{5.5}$$

El valor asociado a cada color queda disponible en todas sus posiciones. En la capa real, esta suma todavía atraviesa la normalización y la proyección de retorno antes de incorporarse mediante la conexión residual.

Selección multi-eje

La segunda operación corresponde a *softmax*. Su función es convertir puntuaciones internas en comparaciones relativas. En lugar de normalizar siempre sobre un único eje, la capa considera todos los subconjuntos no vacíos de los ejes informativos presentes en la

hoja. Estos ejes pueden ser el color, la dirección, las filas y las columnas; los ejemplos y los canales nunca forman parte de la normalización. Si una hoja contiene k ejes elegibles, se calculan $2^k - 1$ distribuciones diferentes y se concatenan sobre el eje de canales antes de aplicar Q_s .

Sea A_s el conjunto de ejes elegibles de la hoja s y sea J uno de sus subconjuntos no vacíos. Para cada canal interno q , los índices pertenecientes a los ejes que no están en J permanecen fijos. La operación se define mediante la Ecuación (5.6):

$$\begin{aligned} m_{J, \mathbf{i}_J, q} &= \max_{\mathbf{j}_J} u_s(\mathbf{j}_J, \mathbf{i}_J, q), \\ p_J(\mathbf{i}, q) &= \frac{\exp(u_s(\mathbf{i}, q) - m_{J, \mathbf{i}_J, q})}{\sum_{\mathbf{j}_J} \exp(u_s(\mathbf{j}_J, \mathbf{i}_J, q) - m_{J, \mathbf{i}_J, q})}, \\ \Phi_s^{\text{sm}}(u_s) &= \text{concat}_{\emptyset \neq J \subseteq A_s} p_J(u_s), \\ h_s^+ &= h_s + Q_s \Phi_s^{\text{sm}}(P_s h_s). \end{aligned} \tag{5.6}$$

La resta del máximo no cambia las probabilidades, pero evita desbordamientos al calcular la exponencial.

Ejemplo simplificado. Considérese una activación proyectada con dos colores por filas y dos posiciones por columnas. Si se elige únicamente el subconjunto formado por el eje de color, cada columna se normaliza por separado, como se observa en la Ecuación (5.7):

$$U = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \implies \text{softmax}_C(U) = \begin{pmatrix} 0,731 & 0,119 \\ 0,269 & 0,881 \end{pmatrix}. \tag{5.7}$$

En la primera posición domina el primer color, mientras que en la segunda domina el segundo. Esta matriz es solo una de las distribuciones generadas; la implementación calcula también las correspondientes a los demás subconjuntos de ejes. Aunque la llamada a la capa solicita una pre-normalización, el envoltorio residual de la versión analizada sobrescribe inmediatamente ese valor con la proyección de h_s , por lo que la pre-normalización no interviene efectivamente en este cálculo.

Máximo acumulado direccional

La operación *cummax* proporciona un mecanismo de propagación espacial de largo alcance. Para cada posición conserva el mayor valor encontrado desde el inicio del recorrido

hasta la celda actual. El sentido del recorrido depende de la orientación y de la paridad del canal proyectado, ya que la implementación divide los canales en dos mitades intercaladas que se procesan en sentidos opuestos. Esto permite representar condiciones como que una señal ya ha aparecido en una fila, que una línea debe prolongarse desde un punto o que se ha alcanzado un límite durante una exploración. La capa solo modifica las hojas 11111 y 10111, que contienen ejemplos, direcciones, filas y columnas, con o sin color. En las demás hojas se comporta como la identidad.

Sea $j \leq_{d,p} i$ la relación que indica que j precede a i para la orientación d y la paridad de canal $p \in \{0, 1\}$, y sea m_j la máscara que determina si la posición es válida. El máximo acumulado y su reescalado al intervalo $[-1, 1]$ se expresan en la Ecuación (5.8):

$$\begin{aligned} c_i^{(d,p)} &= \text{máx}\{u_j : j \leq_{d,p} i, m_j = 1\}, \\ \hat{c}_i^{(d,p)} &= 2 \frac{c_i^{(d,p)} - a_{d,p}}{b_{d,p} - a_{d,p}} - 1, \\ h_s^+ &= h_s + Q_s \mathcal{N}(\hat{c}^{(d,p)}), \end{aligned} \tag{5.8}$$

mediante la notación simplificada de una sección direccional y una paridad de canal. Los valores $a_{d,p}$ y $b_{d,p}$ son, respectivamente, el mínimo y el máximo válidos del recorrido. El código añade pequeñas constantes al denominador y a los extremos para asegurar la estabilidad numérica. En las direcciones cardinales se usa un máximo acumulado directo; en las diagonales se realiza un escaneo asociativo con desplazamientos de tamaño 1, 2, 4, ...

Ejemplo simplificado. Para una paridad de canal cuyo recorrido avanza de izquierda a derecha, el tercer valor deja de ser 1 porque antes se ha observado un 2. Tomando $a = 0$ y $b = 3$, se obtiene la transformación de la Ecuación (5.9):

$$\begin{aligned} U &= \begin{pmatrix} 0 & 2 & 1 & 3 \end{pmatrix}, \\ \text{cummax}(U) &= \begin{pmatrix} 0 & 2 & 2 & 3 \end{pmatrix}, \\ \hat{U} &= \begin{pmatrix} -1 & \frac{1}{3} & \frac{1}{3} & 1 \end{pmatrix}. \end{aligned} \tag{5.9}$$

La señal de valor 2 permanece activa hasta que aparece un valor mayor. Después, la rama se normaliza, se proyecta y se suma al estado anterior.

Desplazamiento direccional

La capa *shift* introduce comunicación local entre celdas vecinas. Cada sección del eje de dirección se divide en canales pares e impares, que se desplazan una posición en sen-

tidos opuestos. Las cuatro orientaciones cardinales modifican una sola coordenada y las cuatro diagonales modifican simultáneamente fila y columna. Los valores que salen de la cuadrícula se descartan y las nuevas posiciones se rellenan con cero. Como *cummax*, esta capa solo actúa sobre las hojas 11111 y 10111 y utiliza las máscaras para anular los valores de origen que pertenecen al relleno.

Para una orientación d y una paridad de canal p , sea $\Delta_{d,p} = (\Delta_{i,d,p}, \Delta_{j,d,p})$ el desplazamiento correspondiente. La transformación se expresa en la Ecuación (5.10):

$$\begin{aligned} i' &= i - \Delta_{i,d,p}, & j' &= j - \Delta_{j,d,p}, \\ [S_{d,p}(U)]_{i,j,q} &= \mathbb{K}_{\Omega}(i', j') m_{i',j'} U_{i',j',q}, \end{aligned} \quad (5.10)$$

donde Ω es el dominio de la cuadrícula y $\mathbb{K}_{\Omega}(i', j')$ vale uno cuando la posición de origen pertenece a dicho dominio y cero en caso contrario. Esta función indicadora representa el relleno con ceros aplicado en los límites.

Si $S(U)$ representa la colección de los desplazamientos conservados por dirección y paridad de canal, la capa completa queda recogida en la Ecuación (5.11):

$$h_s^+ = h_s + Q_s \mathcal{N}(S(P_s h_s)). \quad (5.11)$$

Ejemplo simplificado. Para una de las dos paridades de canal, al desplazar una columna hacia la derecha una activación situada en el centro de una cuadrícula 3×3 , se obtiene el resultado de la Ecuación (5.12):

$$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} \xrightarrow{\Delta=(0,1)} \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}. \quad (5.12)$$

La salida no se suma a las otras secciones direccionales en este punto. Cada orientación conserva su propia sección y sus dos paridades de canal representan sentidos opuestos. Las capas posteriores aprenden cómo combinar esas vistas. La repetición de *shift* en cuatro bloques permite componer desplazamientos que abarcan más de una celda.

Comunicación entre direcciones

Después de procesar cada orientación por separado, *direction_share* intercambia información entre las ocho direcciones. La capa se aplica a todas las hojas que contienen el eje direccional, incluso si no contienen filas o columnas. A diferencia del envoltorio residual

genérico, esta función implementa directamente una pre-normalización efectiva y acumula cada contribución sobre el estado original.

Sea $z_s = \mathcal{N}(h_s)$ y sea A_{s,d_1,d_2} el mapa lineal aprendido que lleva información desde la dirección d_2 hasta la dirección receptora d_1 . Los coeficientes c no se aprenden, sino que dependen de la separación angular, según la Ecuación (5.13):

$$z_s = \mathcal{N}(h_s),$$

$$h_{s,d_1}^+ = h_{s,d_1} + \sum_{d_2=0}^7 c_{(d_2-d_1) \bmod 8} A_{s,d_1,d_2} z_{s,d_2}, \quad (5.13)$$

$$(c_0, c_1, \dots, c_7) = (1, 0, 2, 0, 4, 0, 2, 1, 0, 2, 0, 4, 0, 2).$$

Las direcciones iguales u opuestas reciben peso 1, las ortogonales peso 0,4 y las separadas por un número impar de octavos de vuelta peso 0,2. Los mapas A_{s,d_1,d_2} , en cambio, sí se optimizan durante el entrenamiento.

Ejemplo simplificado. Considérese una posición local de un canal perteneciente al tensor ya normalizado, mapas lineales identidad y la dirección $d_1 = 0$ como receptora. Los valores de las restantes posiciones del tensor, que no se muestran, son los que completan los estadísticos globales de la normalización. Si en la posición observada solo las direcciones 0, 2 y 4 contienen información, se obtiene el cálculo de la Ecuación (5.14):

$$z_s = \begin{pmatrix} 1 & 0 & 2 & 0 & 3 & 0 & 0 & 0 \end{pmatrix}, \quad h_{s,0} = 1, \quad (5.14)$$

$$h_{s,0}^+ = 1 + 1 \cdot 1 + 0,4 \cdot 2 + 1 \cdot 3 = 5,8.$$

El resultado reúne en una sola orientación evidencia procedente de la propia dirección, de una dirección ortogonal y de la dirección opuesta, respetando la geometría impuesta por los coeficientes.

Transformación no lineal

Las proyecciones y sumas anteriores son operaciones lineales por partes. Si todo el bloque estuviera formado únicamente por ellas, varias capas consecutivas seguirían teniendo una capacidad expresiva limitada. La función *nonlinear* proyecta cada hoja a un espacio intermedio de 16 canales, aplica SiLU elemento a elemento, devuelve el resultado al número de canales original y lo añade al estado de entrada.

La función sigmoide σ actúa como una compuerta suave sobre cada activación y se incorpora al flujo residual mediante la Ecuación (5.15):

$$\begin{aligned}\text{SiLU}(u) &= u \sigma(u) = \frac{u}{1 + e^{-u}}, \\ h_s^+ &= h_s + Q_s \text{SiLU}(P_s h_s).\end{aligned}\tag{5.15}$$

Los valores positivos grandes se conservan casi por completo, los negativos se atenúan y el cero permanece en cero. La transición es continua y diferenciable, lo que facilita la optimización por gradiente.

Ejemplo simplificado. Para reproducir la identidad efectiva alrededor de SiLU, puede imaginarse que P_s coloca cada valor en el primer canal de los 16 disponibles y que Q_s recupera ese mismo canal. La operación central y su suma residual se muestran en la Ecuación (5.16):

$$\begin{aligned}H &= \begin{pmatrix} -2 & 0 \\ 1 & 2 \end{pmatrix}, \\ \text{SiLU}(H) &\simeq \begin{pmatrix} -0,238 & 0 \\ 0,731 & 1,762 \end{pmatrix}, \\ H^+ = H + \text{SiLU}(H) &\simeq \begin{pmatrix} -2,238 & 0 \\ 1,731 & 3,762 \end{pmatrix}.\end{aligned}\tag{5.16}$$

La salida muestra que SiLU modifica la intensidad sin convertir la activación en una decisión binaria. Como ocurre en *softmax*, la pre-normalización solicitada en la llamada queda anulada por la sobrescritura existente en el envoltorio residual de esta versión.

Comunicación descendente del Multitensor

La función *share_down* completa el intercambio iniciado por *share_up*. En este caso, una hoja compacta recibe información de las hojas que contienen sus mismos ejes y otros adicionales. Las dimensiones ausentes en el receptor se eliminan mediante promedios. Para determinados ejes espaciales, cuando la relación entre los tamaños de entrada y salida lo permite, el promedio se pondera con las máscaras para excluir las celdas de relleno.

Sea $\mathcal{R}_{t \rightarrow s}$ el operador que reduce los ejes presentes en una hoja superior t pero ausentes en la hoja receptora s . La comunicación descendente se formula en la Ecuación (5.17) (*share_down*):

$$\begin{aligned} u_s^\downarrow &= \sum_{t: s \leq t} \mathcal{R}_{t \rightarrow s}(P_t h_t), \\ h_s^+ &= h_s + Q_s \mathcal{N}(u_s^\downarrow). \end{aligned} \quad (5.17)$$

Ejemplo simplificado. En una hoja CX , las filas representan dos colores y las columnas dos posiciones. Para obtener una contribución destinada a la hoja C , se promedia el eje espacial como se muestra en la Ecuación (5.18):

$$h_{CX} = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix} \implies \mathcal{R}_{CX \rightarrow C}(h_{CX}) = \begin{pmatrix} \frac{1+3}{2} \\ \frac{2+4}{2} \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \end{pmatrix}. \quad (5.18)$$

De este modo, un patrón distribuido por la cuadrícula se convierte en un resumen por color. En la implementación completa se suman las reducciones de todas las hojas compatibles, se normaliza el resultado y se incorpora mediante la proyección residual.

Normalización final

Cada bloque termina con *normalize*. Esta función se aplica de forma independiente a cada hoja y a cada canal, pero calcula sus estadísticos sobre todos los demás ejes de la hoja. Primero resta la media y después divide por la raíz de la media cuadrática de los valores centrados. No introduce parámetros entrenables de escala o sesgo y, a diferencia de las siete operaciones anteriores, no forma una rama residual.

Si I_k es el conjunto de N posiciones pertenecientes al canal k , la operación se define en la Ecuación (5.19):

$$\begin{aligned} \mu_k &= \frac{1}{N} \sum_{i \in I_k} h_{i,k}, \\ \mathcal{N}(h)_{i,k} &= \frac{h_{i,k} - \mu_k}{\sqrt{\varepsilon + \frac{1}{N} \sum_{j \in I_k} (h_{j,k} - \mu_k)^2}}, \quad \varepsilon = 10^{-8}. \end{aligned} \quad (5.19)$$

Ejemplo simplificado. Para una hoja con un único canal y cuatro posiciones se toman los estadísticos de la Ecuación (5.20):

$$H = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad \mu = 2,5, \quad \frac{1}{4} \sum_{i=1}^4 (H_i - \mu)^2 = 1,25. \quad (5.20)$$

Al sustituir estos valores se obtiene el resultado de la Ecuación (5.21):

$$\mathcal{N}(H) = \frac{H - 2,5}{\sqrt{1,25 + 10^{-8}}} \simeq \begin{pmatrix} -1,342 & -0,447 \\ 0,447 & 1,342 \end{pmatrix}. \quad (5.21)$$

La matriz resultante tiene media nula y una magnitud cuadrática media próxima a uno. Esto impide que las sumas residuales de los cuatro bloques aumenten progresivamente la escala de las activaciones y proporciona al bloque siguiente una entrada numéricamente estable.

En conjunto, *share_up* y *share_down* conectan distintos niveles de detalle; *softmax* introduce selección; *cummax* y *shift* realizan propagación espacial; *direction_share* integra orientaciones. SiLU aporta capacidad no lineal y *normalize* mantiene estable el flujo. La repetición de esta secuencia durante cuatro bloques permite componer estas operaciones elementales para aproximar transformaciones más complejas de las tareas ARC-AGI.

5.1.4. Limitaciones

El autor Liao y Gu (2025) deja claro en su trabajo lo que el modelo sabe y no sabe hacer. Las habilidades del modelo se pueden resumir del siguiente modo:

- Completar patrones espaciales (extensión de líneas, propagación de colores).
- Llenar regiones cerradas.
- Detectar y completar simetrías como espejos, rotaciones y traslaciones.
- Agrupar, reagrupar y reordenar objetos.
- Aplicar transformaciones afines simples a regiones.
- Identificar el objeto distinto en una colección uniforme.

También existen numerosas limitaciones arquitectónicas y de capacidad del modelo que restringen su desempeño en tareas más complejas o abstractas:

- Existen algunas ausencias claras de habilidades primitivas, como la capacidad de contar. El modelo no es capaz de contar regiones ni cuántos píxeles de un color hay en la cuadrícula. Esta habilidad simplemente no está presente en una función del modelo y podría ser implementada dentro de la capa del núcleo de descompresión.
- Limitaciones que requieren recurrencia en el tiempo, como simulaciones de píxeles que se mueven con reglas específicas, como sucede en la tarea 2dd70a9a.

- Ausencia de planificación a largo plazo, reglas con múltiples pasos secuenciales no se suelen descubrir.
- Se ha observado aleatoriedad en la semilla. Distintas semillas son capaces de resolver o no ciertas tareas. Existen ciertos casos en los que algunos tensores críticos pueden llegar recomponerse tras una caída en el error de manera abrupta.
- Entrenar el conjunto total de tareas de ARC-AGI dura en torno a 300 horas, lo que equivale a 20 minutos por tarea.
- El uso de la CPU mediante el entrenamiento en paralelo es demasiado alto. A pesar de poder ejecutar diversos modelos al mismo tiempo gracias a la capacidad de la tarjeta gráfica, el alto uso de CPU produce una especie de “cuello de botella” que limita el rendimiento global. Ya sea de forma secuencial o paralela, la velocidad de entrenamiento rara vez supera las 2 iteraciones por segundo.
- Destacar que existen limitaciones estructurales de capas neuronales densas. La arquitectura está fijada como una constante en 4 capas para todas las tareas. El objetivo es mantener baja la longitud de la descripción del modelo. La consecuencia es que pueden existir tareas que requieren más de 4 capas o directamente que necesitan iterar más veces un mismo operador. Esto deja claro una evidente ausencia de recurrencia y capacidad adaptativa profunda. Otros modelos, como los de razonamiento jerárquico, sí presentan mecanismos de razonamiento iterativo y adaptativo.
- Limitaciones operacionales de réplica de formas. Ciertas tareas que requieren traslación, rotación, reflexión y escalado de una forma completa se quedan sin solución. Esta es probablemente la mayor categoría de fallos que se pueden resolver con mejoras arquitectónicas.
- En muchos casos ocurre un colapso del espacio latente. Generalmente suelen ser 4 tensores los que sostienen el aprendizaje dentro del bucle iterativo. 14 de los 18 tensores nunca llegan a recuperarse de un colapso tras 200 iteraciones. Si alguno de los tensores colapsados es requerido para codificar información crítica, la tarea se queda sin solución y no existe método de rescate. Fortalecer la robustez del espacio latente parece ser fundamental.

- Se ha observado que el código implementa un mecanismo de puntuación de los diversos candidatos mediante variables constantes.

5.1.5. Núcleos de conocimiento

Según Chollet (2019b), la inteligencia no depende únicamente de la cantidad de experiencia disponible, sino también de los conocimientos previos que orientan la búsqueda de una solución. Estos conocimientos previos, o sesgos inductivos, permiten reducir el espacio de hipótesis que debe explorarse y hacen posible generalizar a partir de un número reducido de ejemplos. En el contexto de ARC-AGI, Chollet identifica cuatro sistemas de conocimiento fundamentales, relacionados con la objetualidad, la agencia, los números y la geometría elemental junto con la topología. No se trata de cuatro etapas de procesamiento independientes, sino de capacidades previas que condicionan la forma en que se representan los problemas y se interpretan las transformaciones.

El análisis de *CompressARC* permite determinar hasta qué punto estos cuatro núcleos están presentes en su arquitectura. El modelo representa cada tarea mediante un sistema de tensores con cinco ejes, correspondientes a ejemplos, colores, direcciones, filas y columnas. Esta decisión incorpora de forma explícita algunas relaciones espaciales, pero no proporciona una representación completa de todos los conceptos que pueden aparecer en una tarea de ARC-AGI. Por tanto, el rendimiento del modelo no solo depende de la optimización de sus parámetros, sino también de que la transformación buscada pueda expresarse mediante los conocimientos previos que la arquitectura ya contiene.

Objetualidad

La objetualidad se comprende como la capacidad de percibir entidades como objetos cohesionados, persistentes y diferenciables del fondo. En muchas tareas de ARC-AGI la regla no se aplica a cada celda de manera aislada, sino a una figura completa. El tamaño, la forma, la posición, el color o la relación de una figura con otras entidades determinan la transformación que debe realizarse.

CompressARC no incluye una representación explícita de objetos. Sus operaciones trabajan sobre tensores densos y sobre las combinaciones de ejes del sistema Multitensor. La información sobre una figura puede mantenerse de forma implícita mediante los valores de sus celdas, las máscaras y las operaciones espaciales, pero el modelo no detecta componentes conexas como unidades independientes ni reserva un estado específico pa-

ra sus propiedades. Tampoco existe un mecanismo dedicado para comparar dos objetos, seleccionar el de mayor tamaño o agruparlos según su forma.

Esta carencia explica que algunas tareas de selección y agrupación de objetos solo sean accesibles cuando la regla puede aproximarse mediante operaciones píxel a píxel y relaciones direccionales. Cuando la solución exige conservar la identidad de varias figuras y operar sobre cada una de ellas de manera independiente, la representación actual debe codificar esa información en activaciones distribuidas. El modelo posee, por tanto, una objetualidad implícita y débil, pero no un núcleo de conocimiento dedicado a la detección y manipulación de objetos.

Orientación hacia objetivos

El segundo núcleo describe la capacidad de representar entidades que actúan, persiguen objetivos y modifican su entorno. Aunque no todas las tareas de ARC-AGI requieren un agente explícito, este conocimiento resulta relevante cuando la solución depende de una secuencia de acciones, de una interacción entre entidades o de la distinción entre un estado inicial y un estado objetivo.

En *CompressARC* no existe una representación de agentes, acciones ni objetivos intermedios. El modelo recibe los ejemplos de entrada y salida, transforma sus representaciones internas y optimiza directamente la correspondencia con la salida esperada. No mantiene un estado de planificación ni dispone de un mecanismo que evalúe qué acción debe ejecutarse a continuación. En consecuencia, las tareas que pueden resolverse como una transformación global de la cuadrícula son compatibles con el diseño actual, mientras que las que requieren simular una interacción o encadenar decisiones dependen de que dicha secuencia quede codificada indirectamente en las capas disponibles.

La ausencia de este núcleo no constituye una limitación para todas las tareas de ARC-AGI, puesto que muchas de ellas son transformaciones estáticas. Sin embargo, reduce la capacidad del modelo para representar reglas con varios pasos consecutivos y contribuye a explicar la falta de planificación a largo plazo observada en los experimentos.

Números y conteo

El tercer núcleo corresponde al conocimiento de los números y a la capacidad de contar. En una tarea de ARC-AGI, contar no implica únicamente sumar valores, sino identificar unidades discretas, conservar su cardinalidad y comparar cantidades. Ejemplos de este tipo

son determinar cuántos objetos aparecen, comparar el número de celdas de dos regiones o repetir una operación tantas veces como indique una cantidad observada.

La arquitectura analizada no contiene una primitiva explícita de conteo ni una representación entera de las cantidades. Los colores se tratan como categorías y las activaciones internas son variables continuas. Las normalizaciones, las agregaciones y las proyecciones pueden producir magnitudes relacionadas con el número de celdas activas, pero esas magnitudes no tienen garantizada una interpretación discreta ni una correspondencia estable con los números naturales. Del mismo modo, una comparación de intensidades no equivale a una comparación exacta de cardinalidades.

Esta limitación hace que las tareas basadas en conteo de píxeles, regiones u objetos queden fuera del sesgo inductivo principal del modelo. El sistema puede aprender una correlación aproximada cuando la cantidad está asociada a un patrón espacial sencillo, pero no dispone de un operador que garantice el resultado para tamaños, colores o configuraciones nuevas. La ausencia de este núcleo es, por ello, una restricción representacional y no únicamente un problema de entrenamiento.

Geometría elemental y topología

El cuarto núcleo es el que se encuentra más desarrollado en CompressARC. La arquitectura incorpora una cuadrícula bidimensional con ejes de filas y columnas, ocho orientaciones espaciales y operaciones que propagan información en esas direcciones. Las funciones *shift* y *cummax* permiten comunicar activaciones entre celdas vecinas y acumular señales a lo largo de recorridos, mientras que *direction_share* intercambia información entre orientaciones. Además, la parametrización compartida de las direcciones introduce una equivarianza asociada a las simetrías del cuadrado, es decir, al grupo D_4 .

Este conjunto de mecanismos proporciona un sesgo adecuado para transformaciones espaciales elementales, como prolongar líneas, propagar colores, reconocer relaciones de vecindad y tratar simetrías compatibles con rotaciones de 90 grados y reflexiones. La representación direccional también facilita que una misma operación se aplique de forma coherente en diferentes orientaciones, lo que reduce la cantidad de parámetros que debe aprender el modelo.

La geometría incorporada, no obstante, es parcial. El grupo D_4 no cubre rotaciones de ángulo arbitrario, escalados ni transformaciones afines generales. Tampoco existe una primitiva específica para copiar, trasladar o replicar una figura completa conservando

simultáneamente su estructura. Por otra parte, el componente topológico está poco desarrollado. El modelo no representa de forma explícita la conectividad, los agujeros, los contornos ni la relación de inclusión entre regiones. En consecuencia, puede capturar regularidades geométricas locales, pero tiene dificultades cuando la solución depende de una propiedad topológica o de una transformación global de un objeto.

Síntesis de los núcleos presentes

La comparación anterior muestra que *CompressARC* materializa con mayor claridad el conocimiento geométrico y direccional. El conteo carece de una operación propia. Esta distribución de capacidades es coherente con los resultados observados en las tareas experimentales. El modelo resuelve con mayor facilidad las transformaciones que pueden expresarse como propagaciones, simetrías o relaciones espaciales sobre una cuadrícula, pero encuentra dificultades cuando debe identificar objetos como entidades persistentes, contar elementos o ejecutar una secuencia de transformaciones.

Desde esta perspectiva, el límite de rendimiento no se explica únicamente por el número de capas o por la duración del entrenamiento. También refleja el alcance del conjunto de conocimientos previos incorporado en el diseño. Aumentar la capacidad de cálculo puede mejorar la optimización dentro de este espacio, pero no garantiza que el modelo pueda resolver una tarea cuyo concepto central no esté representado. Por ello, los ejes de mejora deben considerar la incorporación de operaciones de copia y manipulación de objetos, primitivas de conteo y mecanismos de iteración adaptativa, manteniendo al mismo tiempo las ventajas de las representaciones direccionales y de la equivarianza espacial.

5.2. Ejes de mejora

Debido a las limitaciones observadas en los experimentos previos, se proponen los siguientes ejes de mejora para abordar los problemas identificados en el modelo:

1. **Eje H: Reducción de saturación de procesos en paralelo.** El objetivo es estabilizar los problemas de saturación de la CPU sobre todo para el entrenamiento de tareas en paralelo. También se deben desarrollar herramientas para obtener métricas comparativas cuantitativas.
2. **Eje D: Eficiencia computacional y aprovechamiento de recursos.** Acelerar

el proceso de entrenamiento aprovechando la capacidad computacional medida en TFLOPs y VRAM.

3. **Eje B: Mejora de robustez del espacio latente.** Tratar de contener el colapso del espacio latente para mejorar la robustez en el proceso de entrenamiento.

5.2.1. Eje H: Reducción de saturación de procesos en paralelo

Las primeras pruebas ejecutadas en el equipo mostraron una saturación de la CPU de alrededor del 80 % con diversos procesos en paralelo. Esto indicaba una saturación en la comunicación entre la CPU y la GPU. Como consecuencia del uso intensivo, también se produjeron serios problemas de temperatura durante procesos prolongados. En muchas ocasiones, los diversos procesos en paralelo han provocado temperaturas por encima de 90 °C, lo que representa un riesgo para la viabilidad técnica del proyecto. Dado que la mayor parte de las operaciones son realizadas por la GPU, no tiene sentido tal problema de sincronización. El código tampoco requiere operaciones complejas relacionadas con la CPU.

Para controlar, al menos temporalmente, este problema, se implementó un argumento opcional para limitar el número de hilos de procesamiento cada vez que se entrenaba en paralelo. El número de hilos de procesamiento prácticamente equivale al número de tareas de entrenamiento al mismo tiempo.

El objetivo principal de este eje es la optimización del tiempo de ejecución. No se busca una mejora en el modelo ni que razone mejor, sino que cada tarea requiera una menor carga de la CPU con los mismos procesos. La mayoría de las operaciones recaen en la GPU, pero existen numerosas operaciones pequeñas que son realizadas por el procesador:

- Llamadas frecuentes a la CPU.
- Operaciones tensoriales reducidas.
- Lanzamientos de *kernels* en la GPU.
- Bucles de procesamiento.
- Conversiones de datos entre la CPU y la GPU. La CPU carga información constantemente en la GPU.
- Cálculos de recolección de datos para el error.

El problema era que la CPU debía coordinar demasiadas operaciones pequeñas. Cada proceso consumía CPU preparando operaciones, esperando resultados y comunicándose con la GPU. Una operación como `.cpu().numpy()` es especialmente importante. Las operaciones del *software* de bajo nivel como ROCm suelen ejecutarse de forma asíncrona, Python deja operaciones en la cola de la GPU y continúa. Sin embargo, cuando el programa solicita los datos en CPU, la CPU debe esperar a que la GPU termine. Esa espera se denomina sincronización GPU y CPU.

Eliminación de sincronizaciones innecesarias

Los entrenamientos de tareas requieren una gran cantidad de iteraciones para encontrar la solución. Este número de iteraciones oscila entre 1500 y 2000. En cada iteración se realizan diversas operaciones, como el cálculo del error acumulado, el error de reconstrucción y la pérdida total. Estos datos se estaban registrando mediante objetos en memoria. Aunque cada valor podría ser solamente un escalar, la operación obligaba a registrarlo en memoria por medio de la CPU. El problema radica en la cantidad de sincronizaciones necesarias en los diferentes componentes del sistema y el coste de detener el proceso y esperar a que finalicen las operaciones de la GPU puede ser mucho mayor. Muchas de estas sincronizaciones se utilizaban para generar registros de análisis y visualización, lo cual no interesa cuando se trata de realizar trabajos en paralelo.

Se realizaron una serie de cambios para que, de manera opcional, este tipo de operaciones de registro no se realizaran en procesos largos y en paralelo. No se fuerza la sincronización en cada iteración, ya no es necesario hacerlo. Toda esta transferencia de datos se conservaría en la memoria de la GPU y posteriormente de manera opcional se transfiere a la CPU en caso de análisis o visualización.

Se introdujo también un componente denominado *profiler* que se encargaría de lanzar los procesos de manera independiente y realizar mediciones para posteriormente analizar y comparar métricas.

Reducción de la frecuencia de post-procesamiento

Durante el entrenamiento no es necesario generar curvas, gráficos ni análisis detallados en cada iteración. Por ello, el post-procesamiento se realiza con menor frecuencia y se concentra al finalizar el entrenamiento o en puntos concretos de supervisión. De este modo, la GPU se dedica principalmente al cálculo del modelo y se reducen las transferencias de

datos y las esperas de la CPU.

Esta mejora no modifica la pérdida ni los parámetros aprendidos. Solo evita repetir tareas de registro y visualización que no son necesarias para cada paso, lo que disminuye el tiempo total de ejecución, especialmente cuando se entrenan varias tareas en paralelo.

Vectorización del cálculo de la pérdida

El cálculo del error de reconstrucción generaba muchas operaciones pequeñas en la CPU. Para cada ejemplo se probaban distintas posiciones de la cuadrícula objetivo y se calculaba una pérdida para cada recorte. Se agrupan estas operaciones para procesarlas de forma vectorizada en la GPU.

En primer lugar se calculan las puntuaciones de todas las posiciones posibles mediante una suma prefija de la máscara m . Si $P(k)$ es la suma de los valores anteriores a la posición k , la puntuación de un desplazamiento o puede escribirse como:

$$P(k) = \sum_{i=0}^{k-1} m_i, \quad \ell(o) = 2P(o+L) - 2P(o) - P(n), \quad 0 \leq o \leq n-L. \quad (5.22)$$

Así, la suma de cada región de la máscara se obtiene reutilizando los valores de P , en lugar de repetirla para cada desplazamiento. Todas las puntuaciones se calculan a la vez y se convierten en probabilidades mediante *logsumexp*:

$$\log p(o \mid m, L) = \ell(o) - \text{logsumexp}_{o'}(\ell(o')). \quad (5.23)$$

La segunda parte de este procedimiento vectoriza la reconstrucción de los colores. La operación *unfold* extrae todos los recortes posibles de los *logits* y los organiza como un lote. Para cada posición (x, y) , el tensor resultante contiene los valores:

$$\mathcal{A}_{x,y,c,i,j} = a_{c,x+i,y+j}, \quad \text{CE}_{x,y} = \text{CE}(\mathcal{A}_{x,y}, G). \quad (5.24)$$

La entropía cruzada se calcula entonces para todos los recortes en una sola llamada. Una pérdida de clasificación mayor implica una menor probabilidad de reconstrucción. Por ello, la probabilidad final suma todos los desplazamientos posibles con *logsumexp*:

$$\log p(G \mid a, L_x, L_y) = \text{logsumexp}_{x,y} [\log p(x) + \log p(y) - \text{CE}_{x,y}]. \quad (5.25)$$

El objetivo de entrenamiento no cambia:

$$\mathcal{L} = \text{total_KL} + 10 \text{reconstruction_error}. \quad (5.26)$$

Los bucles externos sobre ejemplos, cuadrículas y tamaños posibles se mantienen porque cada caso tiene máscaras y dimensiones diferentes. La vectorización se aplica a la parte con más candidatos, es decir, a los desplazamientos y a los recortes espaciales. De este modo se reducen las llamadas de Python, los lanzamientos de *kernels* y las sincronizaciones entre CPU y GPU, sin cambiar la función de pérdida salvo por pequeñas diferencias de redondeo.

Se realizaron una serie de pruebas con pequeños conjuntos de tareas de entrenamiento para comprobar realmente que existen mejoras cuantitativas.

Métrica	Base	Optimizada	Variación
Tiempo total de ejecución	7929,4 s	7140,2 s	−10,0 %
Duración media por tarea	7101,4 s	6379,4 s	−10,2 %
Uso medio de CPU	84,8 %	80,4 %	−4,4 p.p.
Tiempo con CPU > 90 %	74,3 %	65,0 %	−9,3 p.p. (−12,5 % relativo)
Utilización media de GPU	99,5 %	99,6 %	+0,1 p.p.
Tareas resueltas	5 de 12 (41,7 %)	5 de 12 (41,7 %)	Sin variación

Tabla 5.2: Resultados comparativos de la línea base y la versión optimizada del Eje H en 12 tareas de entrenamiento en paralelo.

Las métricas se han calculado con únicamente las 12 tareas reales, sin incluir el proceso auxiliar de gestión. En conjunto, las optimizaciones redujeron el tiempo de ejecución y la saturación de la CPU, sin disminuir la utilización de la GPU ni la precisión observada en las tareas.

5.2.2. Eje D: Eficiencia computacional y aprovechamiento de recursos

El objetivo principal de este eje de mejoras es acelerar las operaciones realizadas en los procesos de entrenamiento. Principalmente se requiere optimizar los procesos en paralelo y reducir el tiempo de entrenamiento lo máximo posible. No se requiere modificar el núcleo principal del modelo.

El modelo base realiza los entrenamientos con una precisión predeterminada de 32 bits de punto flotante (FP32). Se realizaron pruebas utilizando formato de punto flotan-

te *bfloat16* (BF16) para acelerar el entrenamiento sin reducir la capacidad del modelo. Tras probarlo, se pudo apreciar un impacto en el rendimiento considerablemente peor, descartando reducir la precisión de los datos de manera definitiva.

La tarjeta gráfica Radeon RX 9070 XT dispone de cómputo aritmético suficiente como para poder realizar operaciones de entrenamiento a un mayor ritmo. Algo que se pudo detectar es que el trabajo realizado por la gráfica estaba por debajo de sus capacidades. Tras inspeccionar el código se pudo comprobar que una de las posibles causas es que la CPU estaba solicitando continuamente muchos *kernels* pequeños y el tiempo de lanzar y coordinar tal número de micro-códigos podía ser mayor que el tiempo utilizado para realizar las operaciones.

Para resolver este problema se diseña un nuevo componente denominado *inductor*. Este componente mejora la eficiencia de las operaciones mediante compilación de *kernels*, construyendo una versión optimizada de las regiones que PyTorch puede analizar. Internamente, el código de bajo nivel construye un grafo para conectar las diversas operaciones en lugar de ejecutar inmediatamente cada operación. Las diversas operaciones de las capas se agrupan en bloques más grandes, por ejemplo, combinando la normalización, activación y vuelta a la normalización en un solo bloque. La mejora radica principalmente en reducir el número de llamadas y, por tanto, de solicitudes a la GPU.

Con este método, al principio del entrenamiento de la tarea existe un tiempo de preparación en el que todavía no se realiza ninguna operación. Durante este periodo se realiza la compilación y se cargan los *kernels* en la memoria RAM de la CPU. Cuando la compilación ha terminado, se lanza el entrenamiento y las operaciones se ejecutan mucho más rápidas. En algunos momentos del entrenamiento en paralelo se ha conseguido una mejora significativa incrementando la iteración por segundo de hasta 4 veces. Para comprobar que esta técnica funciona, se debe medir el tiempo completo de ejecución, desde el tiempo de compilación hasta que ha terminado el entrenamiento.

Tras realizar una serie de pruebas, se pudo comprobar una reducción significativa del tiempo de entrenamiento por tarea de alrededor del 66,8 %. Esta mejora, sumada al entrenamiento en paralelo, resulta todavía más considerable. Los tiempos se gestionan de manera más eficiente porque se combina la compilación con el entrenamiento entre tareas. Esta optimización mantiene intacta la arquitectura y no añade parámetros. Tampoco es necesario cambiar el número de iteraciones.

En la siguiente tabla se presentan los resultados comparativos entre la línea base y la

optimización mediante compilación. Se debe tener en cuenta que en los resultados aparece una tarea resuelta adicionalmente. Esto es debido a la variabilidad de algunas tareas dependiendo de la semilla. No se considera una mejora del modelo. También requiere remarcar que esta comparativa se ha realizado con entrenamientos de 1500 iteraciones por tarea, mientras que, para hacer una comparativa justa respecto al modelo base, en los resultados finales se realizará la comparación con 2000 iteraciones por tarea.

Métrica	Línea base	Compilación	Variación
Tiempo total (50 tareas)	40 550,7 s (11,3 h)	13 476,0 s (3,7 h)	−66,8 %
Velocidad de entrenamiento en fase 2 (pasos/s)	1,85	5,67	+206,5 %
Uso medio de CPU	79,4 %	61,2 %	−22,9 %
Utilización media de GPU	99,9 %	98,8 %	−1,1 p.p.
Pico de VRAM	15 768 MB	9 417 MB	−40,3 %
Pico de RAM del sistema	21,9 %	89,5 %	+307 %
Energía por 1.000 pasos (Wh)	14,08	5,08	−64,0 %
Tareas resueltas (<i>pass@2</i>)	18 de 50 (36 %)	19 de 50 (38 %)	+1 tarea

Tabla 5.3: Resultados de la comparación final del Eje D sobre 50 tareas con 1500 iteraciones por tarea.

5.2.3. Eje B: Mejora de robustez del espacio latente y multi-semilla

El objetivo del eje B es mejorar la estabilidad del proceso de entrenamiento. *CompressARC* puede resolver algunas tareas de manera “afortunada” dependiendo de la semilla inicial. Esto lleva a algunos resultados inconsistentes. Sumado al problema del colapso del espacio latente, es necesario analizar una estrategia para estabilizar el espacio latente distribuido en las 18 hojas del Multitensor. Cada una de estas hojas representa una combinación de diferentes dimensiones, como ejemplos, colores, direcciones, filas y columnas (píxeles).

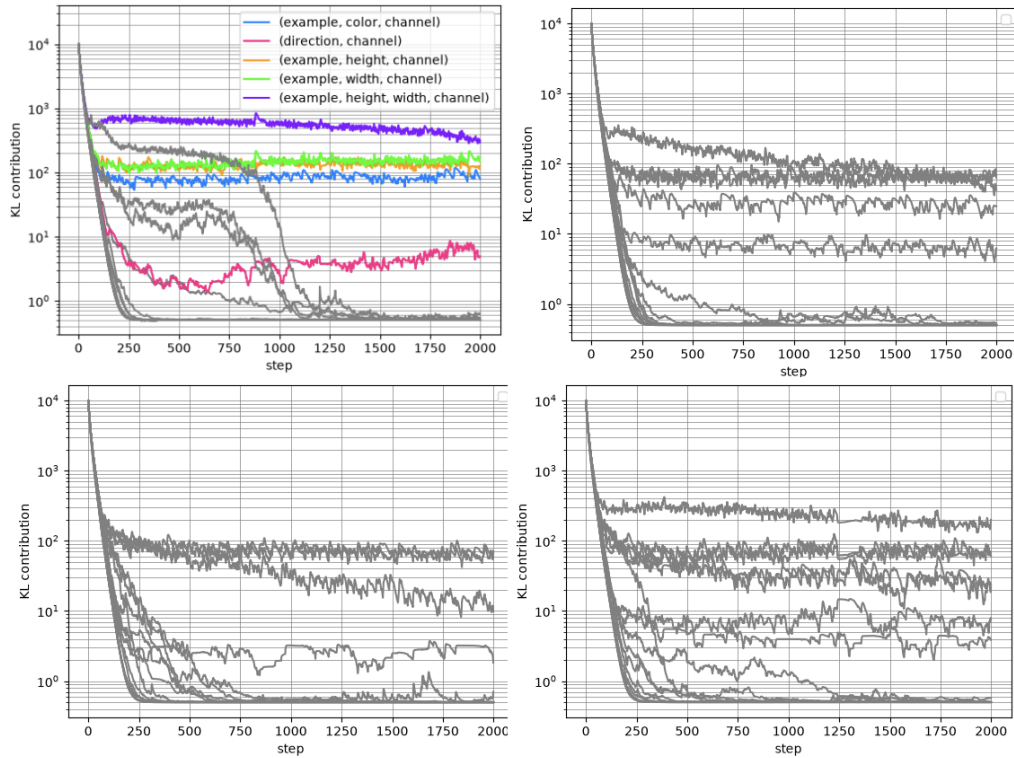


Figura 5.10: Ejemplos de colapso posterior. Tareas 6cdd2623, ecdecbb3, 2bee17df y 007bbfb7.

Fuente: Elaboración propia.

Durante el entrenamiento, cada hoja aporta una cantidad de información mediante su divergencia de Kullback-Leibler. Sin embargo, se observó que muchas hojas reducían rápidamente su contribución. Alrededor de las iteraciones 200-300, muchas hojas colapsaban hasta valores próximos a 0. Este fenómeno se conoce como *posterior collapse*, ya que es el propio espacio latente el que es incapaz de aportar información significativa al modelo. Cuando una hoja colapsa, deja de transportar información relevante sobre la tarea. Si la hoja afectada era necesaria para representar la regla del problema, el modelo podía quedar atrapado en una solución incorrecta sin disponer de un mecanismo para recuperarla.

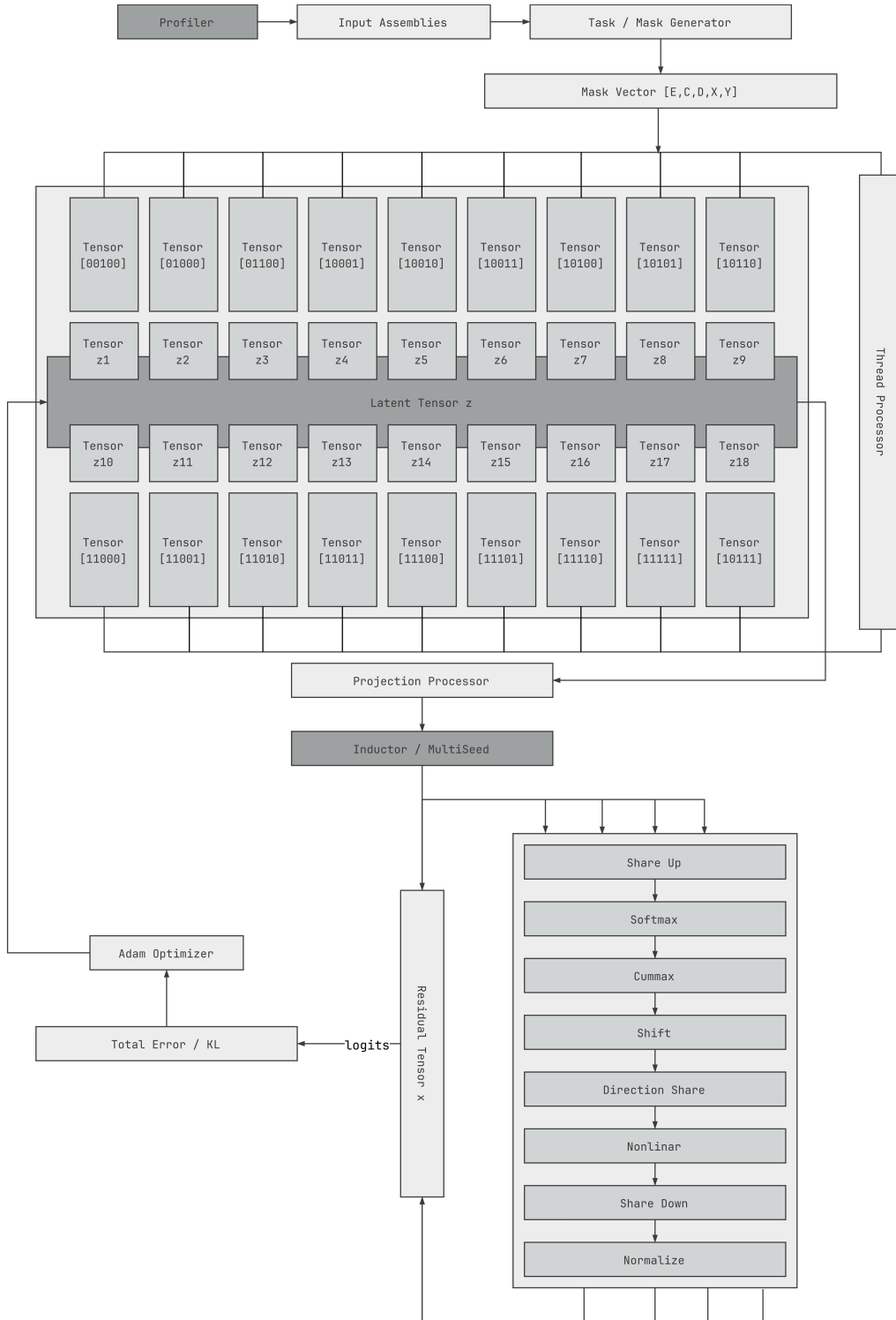


Figura 5.11: Arquitectura *CompressARC* con ejes de mejora H/B/D.

Fuente: Elaboración propia.

Además, el entrenamiento depende de la inicialización aleatoria de los pesos, de los posteriores latentes y de las muestras de ruido utilizadas por el decodificador. Por este motivo, dos ejecuciones de la misma tarea pueden seguir trayectorias diferentes y producir soluciones distintas. Existen casos en los que una hoja ha colapsado y posteriormente se ha recuperado, por lo que el colapso no siempre es irreversible si se recupera a tiempo.

1. Umbral de información libre aplicado a cada hoja latente.
2. Aprendizaje adaptativo que concede mayor importancia a los ejemplos de entrenamiento más difíciles.
3. Varias ejecuciones independientes mediante diferentes semillas durante las primeras etapas de la fase de entrenamiento y una fusión final de sus candidatos.

Estos mecanismos no modifican la arquitectura de *CompressARC*. No se añaden capas, parámetros ni nuevas operaciones al decodificador. El cambio se aplica exclusivamente al flujo de optimización, a la generación de trayectorias independientes y a la selección final de las predicciones.

La inicialización de diferentes semillas independientes por tarea y, por tanto, el entrenamiento independiente durante las primeras etapas tienen como consecuencia un coste computacional evidente. Con esta implementación, cada tarea necesita ejecutarse varias veces de forma independiente en las etapas iniciales y de forma simultánea. Esto obliga a disponer de suficiente capacidad de paralelización y a lanzar las múltiples instancias en lotes. La estrategia utilizada para gestionar este caso es lanzar solamente dos tareas de manera simultánea. Cada tarea se divide en 4 procesos independientes lo que da un total de 8 procesos simultáneos. Una vez que terminan las etapas iniciales, se procede a la fusión de los candidatos generados por cada ejecución independiente. Se ha especificado en código un límite de 2 tareas y 8 procesos simultáneos.

La configuración utilizada en los experimentos principales fue la siguiente:

- Cuatro semillas independientes: 0, 1, 2, 3.
- Umbral inicial de *free-bits* de 2 nats por hoja.
- Decaimiento lineal del umbral hasta cero.
- Aprendizaje adaptativo de ejemplos activado.

- Intensidad máxima del aprendizaje adaptativo igual a 1.
- Decaimiento de la dificultad histórica igual a 0,9.
- Postprocesamiento de candidatos cada cuatro iteraciones.

La configuración se valida antes de iniciar los procesos. Esto evita comparar ejecuciones que utilicen mecanismos de optimización diferentes y que, por tanto, no puedan atribuirse correctamente al eje estudiado.

Umbral de información libre por hoja latente

La primera mejora se basa en aplicar un umbral de información libre a cada una de las 18 hojas del Multitensor. En la configuración original, la pérdida incluye directamente la suma de todas las divergencias KL:

$$\mathcal{L}_{\text{base}} = \sum_{\ell=1}^{18} K_{\ell,t} + 10R_t, \quad (5.27)$$

donde $K_{\ell,t}$ representa la divergencia KL total de la hoja ℓ en la iteración t , y R_t representa el error de reconstrucción de los datos observables.

Cada $K_{\ell,t}$ se obtiene sumando todos los valores del tensor KL asociado a una hoja:

$$K_{\ell,t} = \sum_j \text{KL}_{\ell,j,t}. \quad (5.28)$$

El Eje B sustituye temporalmente esta contribución por una versión limitada inferiormente:

$$K_{\ell,t}^{\text{efectiva}} = \max(K_{\ell,t}, \tau_t), \quad (5.29)$$

donde τ_t es el umbral de información libre en la iteración t . La divergencia KL utilizada en la pérdida queda definida como:

$$K_t^{\text{efectiva}} = \sum_{\ell=1}^{18} \max(K_{\ell,t}, \tau_t). \quad (5.30)$$

El umbral sigue un calendario lineal:

$$\tau_t = \tau_0 \left(1 - \frac{t}{T-1}\right), \quad \tau_0 = 2, \quad (5.31)$$

donde T es el número total de iteraciones. En una ejecución de 2000 iteraciones, el valor comienza en 2 nats y termina en 0:

$$\tau_0 = 2, \quad \tau_{1999} = 0. \quad (5.32)$$

La pérdida completa utilizada durante el entrenamiento es, por tanto:

$$\mathcal{L}_t = K_t^{\text{efectiva}} + 10R_t. \quad (5.33)$$

La interpretación del mecanismo es importante. Si una hoja tiene una KL inferior al umbral, su contribución a la pérdida se sustituye por el valor constante τ_t . En consecuencia, el optimizador deja de recibir el gradiente que empuja esa hoja hacia una KL todavía menor. Por ejemplo, si una hoja tiene $K_{\ell,t} = 0,5$ y el umbral es $\tau_t = 2$, la hoja aporta 2 nats a la pérdida, pero el gradiente asociado a la operación $\max(K_{\ell,t}, 2)$ es cero en esa región.

Este mecanismo no obliga a la hoja a aprender información. Su función es más limitada: simplemente evita que el proceso de compresión la reduzca de manera anticipada a 0. De este modo, la hoja conserva una oportunidad de recibir gradientes procedentes del error de reconstrucción y de recuperar una representación útil.

El umbral disminuye progresivamente hasta desaparecer. Por ello, al final del entrenamiento se recupera la pérdida KL original. Durante el entrenamiento se registran por separado diferentes métricas:

- KL original, sin aplicar el umbral.
- KL efectiva utilizada en la pérdida.
- Valor del umbral en cada iteración.
- Número de hojas cuya KL se encuentra por debajo del umbral.

Aprendizaje adaptativo de demostraciones

La segunda mejora modifica la importancia relativa de los ejemplos de entrenamiento. Una tarea de ARC-AGI contiene varios pares de demostración. Cada par está compuesto por una entrada y una salida conocida. En el modelo base, todos los pares contribuyen a la reconstrucción con el mismo peso. Cabe destacar que, en los datos de evaluación, el

número de ejemplos para la fase de entrenamiento suele ser menor y, en ocasiones, mucho menor.

El Eje B mantiene un historial de dificultad para cada ejemplo. En cada iteración se calcula la pérdida de reconstrucción del par y se divide por el número de píxeles observables:

$$r_{i,t} = \frac{R_{i,t}^{\text{entrada}} + R_{i,t}^{\text{salida}}}{N_i}, \quad (5.34)$$

donde i identifica la demostración, N_i es el número de píxeles de sus cuadrículas y $r_{i,t}$ representa el error medio por píxel.

El uso de la pérdida por píxel evita favorecer a los ejemplos que poseen cuadrículas más grandes. Sin esta normalización, una demostración con mayor tamaño tendría una pérdida superior aunque el modelo la resolviera proporcionalmente mejor.

El código calcula el error de reconstrucción de cada ejemplo, lo divide por el número de píxeles y conserva una media móvil de ese valor. Por tanto, una demostración tiene mayor dificultad cuando el modelo lleva varias iteraciones reconstruyéndola peor que las demás.

Al principio del entrenamiento no existe todavía un historial de dificultad y $\beta_0 = 0$. En ese momento todas las demostraciones tienen peso uno. A medida que avanza el entrenamiento, los ejemplos con mayor pérdida histórica reciben un peso superior y los ejemplos ya dominados reciben un peso inferior.

La reconstrucción utilizada por el modelo puede expresarse como:

$$R_t = R_t^{\text{test input}} + \sum_{i=1}^{n_{\text{train}}} w_{i,t} \left(R_{i,t}^{\text{entrada}} + R_{i,t}^{\text{salida}} \right). \quad (5.35)$$

Las entradas de prueba siguen participando con peso uno porque son datos observables. En cambio, las salidas de prueba permanecen ocultas y nunca participan en la pérdida. Esta condición impide que el currículo utilice información de las soluciones reales de evaluación.

Por tanto, el mecanismo no aprende cuál es la respuesta correcta del test. Únicamente reorganiza el esfuerzo de optimización sobre los pares de entrenamiento conocidos.

Entrenamiento independiente mediante varias semillas

La tercera parte del Eje B aborda la variabilidad producida por la inicialización aleatoria. Para cada tarea se ejecutan cuatro modelos independientes, uno por cada semilla.

Cada combinación de tarea y semilla se considera un trabajo independiente. Por ejemplo, para una tarea llamada 007bbfb7 se crean los trabajos:

- 007bbfb7_seed0
- 007bbfb7_seed1
- 007bbfb7_seed2
- 007bbfb7_seed3

Cada proceso inicializa de forma independiente:

- Generador aleatorio de Python.
- Generador de NumPy.
- Generador de PyTorch.
- Generador aleatorio de la GPU.
- Pesos del modelo.
- Posteriores latentes.
- Optimizador Adam.
- Estado del currículum.
- Registrador de candidatos.

No se comparten pesos, gradientes, momentos del optimizador, valores KL, historiales del currículum ni logits entre las semillas. La semilla no representa un modelo que aprenda entre tareas. Solo determina una trayectoria independiente para resolver una tarea concreta.

Al principio del entrenamiento de un conjunto de tareas en paralelo, se realiza la fase 1, que mide la cantidad de memoria de VRAM requerida para cada tarea y se guarda en un archivo temporal. La medición inicial de memoria no se repite para cada semilla. La fase de medición ejecuta solamente dos pasos por tarea, porque las cuatro semillas utilizan la misma geometría y el mismo consumo aproximado de memoria. La fase de entrenamiento completa sí se expande en cuatro trabajos independientes por tarea.

Selección de candidatos dentro de cada semilla

Cada semilla conserva el selector original de *CompressARC*. En cada paso se consideran dos candidatos:

1. Predicción actual del modelo.
2. Predicción suavizada mediante una media móvil exponencial.

El postprocesamiento completo se realiza cada cuatro pasos en las ejecuciones paralelas. El paso cero siempre se procesa para garantizar que exista un candidato inicial. Esta reducción de frecuencia pertenece al Eje H y busca disminuir sincronizaciones entre CPU y GPU sin cambiar el criterio matemático de selección.

Fusión de las soluciones obtenidas por las semillas

Cuando las cuatro semillas de una tarea han terminado, sus evidencias se combinan para construir las dos respuestas finales exigidas por ARC-AGI.

Se utiliza un consenso entre semillas agregando puntuaciones a diferentes trayectoras. Por otro lado, se busca diversidad normalizando puntuaciones dentro de cada semilla. Si se utilizara únicamente el consenso, una semilla que hubiera descubierto una solución correcta pero diferente podría quedar oculta por una solución repetida por las demás.

La fusión no utiliza las soluciones reales de las tareas. Aunque se ejecuten cuatro modelos internos, el sistema continúa entregando únicamente dos respuestas por tarea y sigue siendo compatible con la métrica *pass@2*. Esto es especialmente importante porque, de no ser así, las pruebas no serían representativas del rendimiento real del sistema.

El conjunto de las soluciones correctas obtenidas por las cuatro semillas no puede utilizarse directamente como métrica. Esa operación sería una selección *oracle*, porque necesitaría conocer de antemano cuál de las soluciones es correcta. El sistema real debe elegir solo dos candidatos utilizando sus puntuaciones internas.

Se realizaron varios experimentos con reducidos conjuntos de tareas. Los experimentos evidenciaron que el modelo no había empeorado e incluso consiguió una ligera mejora aproximada del 4% de tareas resueltas. Estas mejoras no garantizan que resuelva todas las tareas, al contrario, el modelo sigue careciendo de habilidades primitivas necesarias para resolver ciertas tareas.

El Eje B mejora la robustez mediante una combinación de estabilización, adaptación y diversidad. El umbral de información libre protege temporalmente las hojas latentes. El uso de 4 semillas permite explorar trayectorias independientes y conservar dos soluciones complementarias mediante una fusión basada en consenso y diversidad.

Herramienta de visualización de procesos

Para facilitar la monitorización del progreso, comportamiento general del sistema e identificar tareas en entrenamiento, se ha implementado una interfaz sencilla que permite visualizar el estado de las tareas en entrenamiento y el comportamiento general del sistema. Esta interfaz permite realizar observaciones que son muy necesarias cuando se realizan ejecuciones de entrenamiento prolongadas.

La interfaz proporciona información activa de entrenamiento, velocidad de iteraciones por segundo y algunas métricas adicionales.

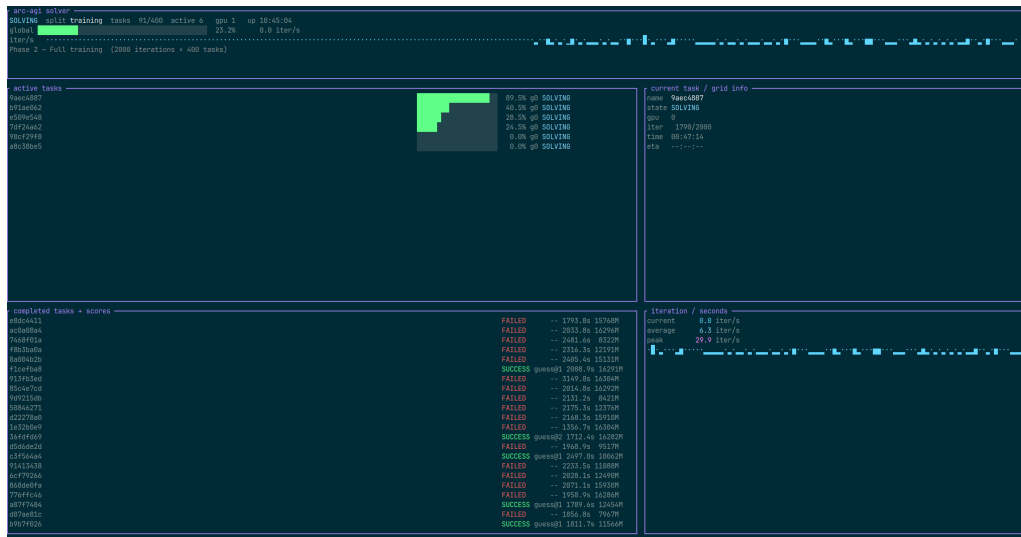


Figura 5.12: Panel de monitorización del progreso y comportamiento del sistema.

5.2.4. Problemas encontrados

Durante el desarrollo se han ido encontrando diversos problemas relacionados con la puesta en marcha del *software* del entorno, entre otros:

- A pesar de los grandes avances de ROCm, todavía se encuentran errores de controladores en medio de los procesos de entrenamiento. Este era un problema especialmente preocupante al inicio del desarrollo porque podía provocar que todo el conjunto de

entrenamiento de las tareas, que duraba muchas horas, se viera afectado. Se tuvieron que implementar protocolos y excepciones de código para controlar de forma aislada aquellos entrenamientos que provocaban fallos en todo el sistema.

- Se encontraron problemas del *software* que controlaban diversos sistemas del equipo durante largas horas de entrenamiento. Los ventiladores del equipo podían quedarse sin funcionar en cualquier momento.
- Problemas de falta de memoria gráfica durante el entrenamiento de diversas tareas en paralelo. Se tuvo que recurrir a una especie de fase previa en la que la GPU carga los modelos en memoria, calcula el consumo y se guarda en un archivo temporal para que luego el programa supiera de alguna manera el tamaño de reserva antes de cargarlo.
- La compilación de *kernels* también presentaba problemas en ciertas circunstancias. Los millones de archivos que se generan con el tiempo aumentaban el espacio del disco. Se tuvo que recurrir al uso de un dispositivo de almacenamiento adicional para recoger los 842 GB de documentos creados tras el entrenamiento de las 800 tareas (conjunto de entrenamiento y evaluación).
- Además de optimizar el flujo de entrenamiento y el uso de la CPU, los problemas de temperatura continuaron. Se cambió el sistema de refrigeración y se añadió un marco de contacto de CPU, además de aplicar un menor voltaje por medio de la BIOS.

6. Resultados

Los resultados de la ejecución base se han obtenido de la documentación oficial del modelo *CompressARC*. Las métricas del autor original se han utilizado como referencia, para comparar los resultados obtenidos en este proyecto.

Se ha realizado una ejecución completa de los datos de entrenamiento y de evaluación tras la implementación de cada eje de mejora. La comparativa se establece entre el modelo base, el eje de mejora H/B y el eje de mejora acumulado H/B/D. El eje de mejora H/B hace referencia a las optimizaciones relacionadas con el *hardware* y la memoria, mientras que el eje de mejora D se centra en mejorar el modelo. El eje de mejora acumulado H/B/D combina todas las optimizaciones anteriores.

Las implementaciones H/B no mejoran en las métricas de evaluación, pero es la implementación más eficiente en términos de uso de recursos y tiempo de ejecución.

Por otro lado, el acumulado H/B/D permite mejorar tanto la eficiencia en el uso de recursos como las métricas de evaluación, combinando las ventajas de las optimizaciones de *hardware*, memoria y modelo.

Métrica	Base	Eje H/B	Eje H/B/D
Tiempo total de ejecución (evaluación)	138 h	35,6 h	64,3 h
<i>pass@2</i> Entrenamiento	139 de 400 (34,75 %)	139 de 400 (34,75 %)	176 de 400 (44 %)
<i>pass@2</i> Evaluación	80 de 400 (20 %)	80 de 400 (20 %)	101 de 400 (25,25 %)
Uso medio de CPU	79,4 %	61,2 %	63,7 %
Coste por tarea (aproximación)	0,086 \$	0,02225 \$	0,040 \$

Tabla 6.1: Resultados finales tras la ejecución del modelo base respecto a ejes de mejora.

Con los resultados obtenidos, el modelo renombrado *SuperCompressARC* demuestra una ligera mejora en las métricas de coste por tarea y puntuación de evaluación. El modelo se acerca al rendimiento del modelo compacto HRM, aunque resulta menos exigente desde el punto de vista computacional y más barato.

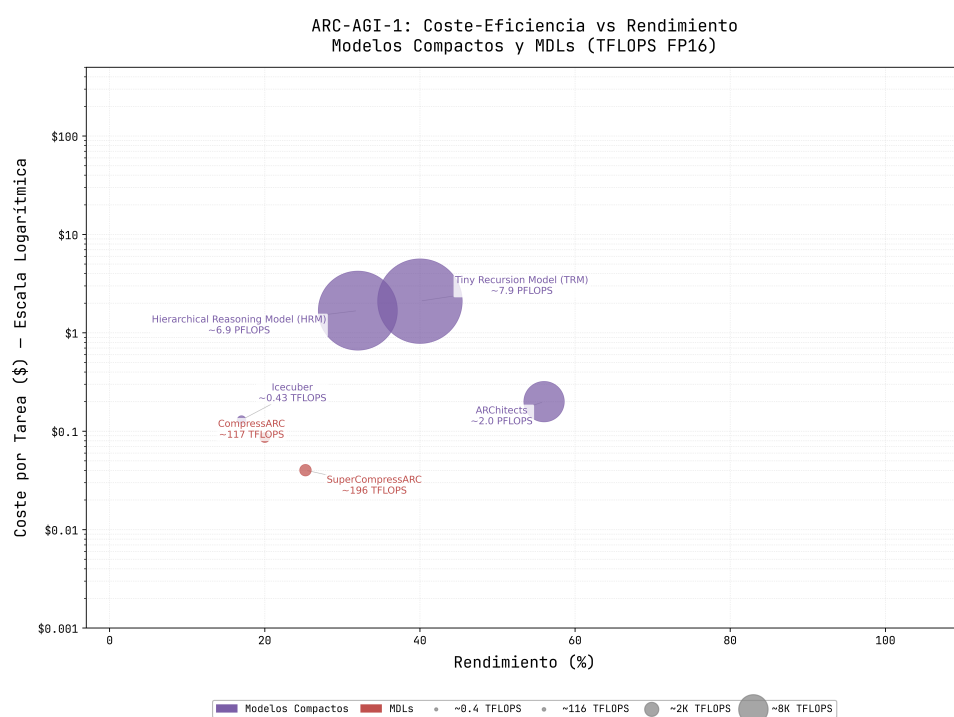


Figura 6.1: Comparativa del modelo renombrado *SuperCompressARC* respecto a otros modelos compactos.

Fuente: Elaboración propia. Datos obtenidos de la página web oficial de ARC-AGI. Imagen completa:

https://github.com/UNIR-TUC/arc-agi/blob/main/src/arc_agi_data_scraper/images/compact_tflops_cost_light_final.png

7. Ética del proyecto

En diversas ocasiones, las decisiones que se toman en cualquier aspecto de la vida pueden implicar alguna consideración ética. Teniendo en cuenta el mercado y la evolución de las tecnologías, cada vez se puede considerar de mayor importancia plantear dimensiones éticas ante decisiones que se tomen.

En este caso, el proyecto utiliza infraestructuras de computación local y esto conlleva diversas reflexiones éticas. Cada vez más entidades están dispuestas a delegar sus propios servicios de computación a proveedores externos que ofrecen servicios de inteligencia artificial a un precio relativamente asequible. De este modo, también se delega en estos proveedores la responsabilidad de mantener los servicios de computación sin desatender las cuestiones éticas del ámbito medioambiental. Los servicios de computación en la nube necesitan indirectamente materias primas y suministro eléctrico, por lo que se deben tener en cuenta la responsabilidad y el impacto derivados del gasto energético.

Cuidar el medioambiente y reducir el impacto sobre él es responsabilidad de cada ciudadano. Por tanto, en este proyecto se puede realizar un análisis ético desde dos perspectivas fundamentales: como individuo y como profesional de la ingeniería del *software*.

Desde una dimensión deontológica, se debe impulsar que cada individuo se informe sobre las consecuencias medioambientales de convertirse en un abonado de un proveedor de servicios de cualquier tecnología de inteligencia artificial. Se debe conocer en qué condiciones y cuáles son los principios que defiende la entidad ante un problema que afecta a todos los ciudadanos. Todo ciudadano tiene derecho a disfrutar y a beneficiarse del mundo globalizado, pero se debe tener en cuenta que las generaciones posteriores también tienen el mismo derecho y, por tanto, se debe preservar el planeta en las mejores condiciones posibles.

Como profesional de la ingeniería del *software*, se debe impulsar el uso de tecnologías que favorezcan la eficiencia energética y el uso de infraestructuras propias, evitando depender excesivamente de servicios externos. El uso de infraestructura propia permite mayor control y libertad al ciudadano sin tener que depender de ciertas compañías.

Se deben apoyar tecnologías que promuevan una generalización real de la inteligencia sin “comprarla” mediante computación y datos masivos a costa del impacto medioambiental mencionado.

8. Conclusiones y Trabajo Futuro

La industria de la inteligencia artificial es un mercado cambiante y exigente. Es un mercado que exige innovaciones constantes en busca de una inteligencia artificial general. En este contexto, es importante tener en cuenta la responsabilidad ética y medioambiental que conllevan el estudio y el desarrollo de tecnologías y modelos.

Este trabajo ha estudiado el diseño y la optimización de un modelo compacto orientado a la resolución del *benchmark* ARC-AGI-1. El objetivo principal consistía en analizar hasta qué punto era posible mejorar el rendimiento de un sistema con una capacidad computacional limitada, priorizando el entrenamiento en tiempo de inferencia y la relación entre precisión y coste de ejecución.

Los resultados obtenidos muestran que este objetivo se ha alcanzado parcialmente. Se ha logrado optimizar el modelo base y posteriormente mejorar ligeramente la capacidad de resolver tareas de ARC-AGI-1. Esto supone una mejora desde el 34,75 % hasta el 44 % en entrenamiento y desde el 20 % hasta el 25,25 % en evaluación. Por tanto, se observa que las estrategias de estabilización del espacio latente, la utilización de varias semillas y la selección de candidatos pueden mejorar la robustez de las predicciones.

Los recientes avances también evidencian que los modelos compactos siguen siendo una opción para la sociedad al margen de grandes compañías y granjas de computación. El uso de tecnología propia y modelos compactos proporciona una mayor libertad a la ciudadanía. Los avances tecnológicos no pueden estar ligados a quienes pueden comprar la inteligencia mediante computación infinita.

No todos los objetivos específicos se han alcanzado completamente. El entrenamiento específico por tarea y la adaptación en tiempo de inferencia sí se han integrado en el flujo experimental. También se ha desarrollado una estrategia de diversidad basada en varias semillas y en la selección de candidatos. Sin embargo, el modelo continúa utilizando una profundidad fija y no dispone de un mecanismo plenamente adaptativo de *early stopping*. Del mismo modo, los bucles de refinamiento implementados no constituyen todavía un sistema recursivo capaz de decidir dinámicamente la profundidad necesaria para cada tarea, como sí realizan modelos como HRM y TRM.

En conjunto, el trabajo confirma que los modelos compactos pueden ofrecer una alternativa viable para investigar el razonamiento abstracto bajo restricciones de *hardware*. La reducción del tamaño o del tiempo de ejecución, por sí sola, no es suficiente para alcanzar

una generalización robusta en ARC-AGI-1.

Como continuación de este trabajo, se propone incorporar una profundidad de razonamiento adaptativa que permita detener el proceso cuando la predicción haya convergido o prolongarlo cuando la tarea requiera transformaciones adicionales. También sería conveniente introducir representaciones explícitas de objetos, operaciones de conteo y mecanismos de copia, traslación y replicación de formas.

Referencias

- Ainooson, J., Sanyal, D., Michelson, J. P., Yang, Y., y Kunda, M. (2023). A neurodiversity-inspired solver for the abstraction & reasoning corpus (arc) using visual imagery and program synthesis. *arXiv preprint arXiv:2302.09425*.
- Alford, S. (2021). *A neurosymbolic approach to abstraction and reasoning* (Tesis Doctoral no publicada). Massachusetts Institute of Technology.
- Aristotle, y Boeri, M. D. (2010). *Acerca del alma*. Colihue Buenos Aires.
- Chollet, F. (2019a). *GitHub - fchollet/ARC-AGI: The Abstraction and Reasoning Corpus* — *github.com*. <https://github.com/fchollet/ARC-AGI>. ([Accessed 20-05-2026])
- Chollet, F. (2019b). On the measure of intelligence. *arXiv preprint arXiv:1911.01547*.
- Chollet, F., Knoop, M., Kamradt, G., Landers, B., y Pinkard, H. (2025). Arc-agi-2: A new challenge for frontier ai reasoning systems. *arXiv preprint arXiv:2505.11831*.
- Franzen, D., Disselhoff, J., y Hartmann, D. (2024). *The llm architect: Solving arc-agi is a matter of perspective*.
- Gendron, G., Bao, Q., Witbrock, M., y Dobbie, G. (2024). *Large language models are not strong abstract reasoners*. Descargado de <https://arxiv.org/abs/2305.19555>
- González, R. (2011). Descartes: las intuiciones modales y la inteligencia artificial clásica. *Alpha (Osorno)*(32), 181–198.
- Hernández-Orallo, J. (2017). Evaluation in artificial intelligence: from task-oriented to ability-oriented measurement. *Artificial Intelligence Review*, 48(3), 397–447.
- Hurbans, R. (2020). *Grokking artificial intelligence algorithms: Understand and apply the core algorithms of deep learning and artificial intelligence in this friendly illustrated guide including exercises and examples*. Manning.

- Jolicoeur-Martineau, A. (2025). Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*.
- Kahneman, D. (2011). Thinking, fast and slow. *Farrar, Straus and Giroux*.
- Legg, S., Hutter, M., y cols. (2007). A collection of definitions of intelligence. *Frontiers in Artificial Intelligence and applications*, 157, 17.
- Li, W., Xu, Y., Sanner, S., y Khalil, E. B. (2024). Tackling the abstraction and reasoning corpus with vision transformers: the importance of 2d representation, positions, and objects. *arXiv preprint arXiv:2410.06405*.
- Liao, I., y Gu, A. (2025). *Arc-agi without pretraining*. Descargado de https://iliao2345.github.io/blog_posts/arc_agi_without_pretraining/arc_agi_without_pretraining.html
- Minsky, M. (1968). *Matter, mind, and models in: Ml minsky (ed.): Semantic information processing*. MIT Press.
- Newell, A., Shaw, J. C., y Simon, H. A. (1962). The processes of creative thinking. En *Contemporary approaches to creative thinking, 1958, university of colorado, co, us; this paper was presented at the aforementioned symposium*.
- Ouellette, S. (2024). Towards efficient neurally-guided program induction for arc-agi. *arXiv preprint arXiv:2411.17708*.
- Perez, D., Samothrakis, S., y Lucas, S. (2014). Knowledge-based fast evolutionary mcts for general video game playing. En *2014 ieee conference on computational intelligence and games* (pp. 1–8).
- Spelke, E. S., y Kinzler, K. D. (2007). Core knowledge. *Developmental science*, 10(1), 89–96.
- Świechowski, M., Godlewski, K., Sawicki, B., y Mańdziuk, J. (2023). Monte carlo tree search: A review of recent modifications and applications. *Artificial Intelligence Review*, 56(3), 2497–2562.
- Vahdati, S., Aioanei, A., Suresh, H., y Lehmann, J. (2026). The arc of progress towards agi: A living survey of abstraction and reasoning. *arXiv preprint arXiv:2603.13372*.
- Wang, G., Li, J., Sun, Y., Chen, X., Liu, C., Wu, Y., . . . Yadkori, Y. A. (2025). Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*.
- Waxman, W. (2014). *Kant’s anatomy of the intelligent mind*. OUP USA.
- Wind, J. S. (2020). Dsl solution to the arc challenge. URL <https://github.com/top-quarks/ARC-solution>.